

CURSO DE JAVA



Iván Rodríguez Ruiz

ÍNDICE

1 INTRODUCCIÓN

- 1.1 ¿Que es Java?
- 1.2 Principales características
- 1.3 Máquina Virtual de Java
- 1.4 Ediciones de Java
- 1.5 Primeros Pasos
 - 1.5.1 El JDK
 - 1.5.2 Configurar Variables de Entorno
 - 1.5.3 Instalación de NetBeans 8
 - 1.5.4 Otros entornos de desarrollo (IDEs)
- 1.6 Creación del primer programa Java
- 1.7 Conceptos básicos de Programación en Java
 - 1.7.1 Objetos
 - 1.7.2 Clases
 - 1.7.3 Métodos y campos
 - 1.7.4 Métodos y campos estáticos
 - 1.7.5 El método main()

2 SINTAXIS DEL LENGUAJE

- 2.1 Sintaxis Básica
- 2.2 Secuencias de escape
- 2.3 Tipos de datos primitivos
- 2.4 Variables
 - 2.4.1 Tipos de datos de una variable
 - 2.4.2 Declaración de variables
 - 2.4.3 Asignación
 - 2.4.4 Literales
 - 2.4.5 Ambito de Variables
 - 2.4.6 Valores por defecto de una variable
 - 2.4.7 Conversiones de tipo
 - 2.4.8 Conversiones implícitas
 - 2.4.9 Conversiones Explícitas
- 2.5 Operadores
 - 2.5.1 Aritméticos
 - 2.5.2 Asignación
 - 2.5.3 Asignación de referencias y asignación de valores
 - 2.5.4 Condicionales
 - 2.5.5 Comparación de tipos básicos
 - 2.5.6 Igualdad de Objetos
 - 2.5.7 Lógicos
 - 2.5.8 Operadores a nivel de bits
 - 2.5.9 Operador instanceof
 - 2.5.10 Operador condicional
- 2.6 El Recolector de Basura

2.7 Instrucciones de Control

- 2.7.1** Condicional If
- 2.7.2** Instrucción switch
- 2.7.3** Bucle for
- 2.7.4** Bucle while
- 2.7.5** Salida forzada del bucle

2.8 Arrays

- 2.8.1** Declaración
- 2.8.2** Dimensionado de un array
- 2.8.3** Acceso a los elementos de un array
- 2.8.4** Paso de un array como argumento
- 2.8.5** Array como tipo de devolución de un método
- 2.8.6** Recorrido de arrays con for each
- 2.8.7** Arrays Multidimensionales

3 CLASES DE USO GENERAL

3.1 Los paquetes

- 3.1.1** Ventajas del uso de paquetes
- 3.1.2** Importar clases y paquetes de clases
- 3.1.3** Paquetes de uso general
- 3.1.4** El API J2SE

3.2 Gestión de cadenas: String

- 3.2.1** Creación de String
- 3.2.2** Inmutabilidad de los objetos String
- 3.2.3** Métodos de la clase String

3.3 La clase Math

- 3.3.1** Constantes públicas
- 3.3.2** Métodos
- 3.3.3** Importaciones estáticas

3.4 Uso de fechas

- 3.4.1** Clase Date
- 3.4.2** Clase Calendar

3.5 Clases de Envoltorio

- 3.5.1** Encapsulamiento de un tipo básico
- 3.5.2** Conversión de cadena a tipo numérico
- 3.5.3** Autoboxing

3.6 Clases de Entrada y Salida

- 3.6.1** Salida de datos
- 3.6.2** Salida con formato
- 3.6.3** La clase Scanner

3.7 Colecciones

- 3.7.1** ArrayList
- 3.7.2** Creación de un ArrayList
- 3.7.3** Métodos del ArrayList

4 PROGRAMACIÓN ORIENTADA A OBJETOS

4.1 Empaquetado de clases

4.2 Modificadores de acceso

4.3 Encapsulación

4.3.1 Protección de datos

4.3.2 Facilidad en el mantenimiento de la clase

4.3.3 JavaBeans

4.4 Sobrecarga de métodos

4.5 Constructores

4.5.1 Definición y uso

4.5.2 Constructores por defecto

4.6 Herencia

4.6.1 Concepto

4.6.2 Ventajas

4.6.3 Nomenclatura y reglas

4.6.4 Relación “Es un”

4.6.5 Creación de Herencia

4.6.6 Ejecutar constructores con Herencia

4.6.7 Métodos y atributos protegidos

4.6.8 Clases finales

4.6.9 Sobreescritura de métodos

4.7 Clases abstractas

4.7.1 Definición

4.7.2 Sintaxis y características

4.8 Polimorfismo

4.8.1 Asignar objetos a variables de la Superclase

4.8.2 Definición

4.8.3 Ventajas

4.9 Interfaces

4.9.1 Definición

4.9.2 Implementación

4.9.3 Interfaces y polimorfismo

4.9.4 Interfaces en el J2SE

- 5 ACCESO A DATOS EN JAVA (JDBC)**
 - 5.1 Java Database Connectivity (JDBC)**
 - 5.2 El driver JDBC**
 - 5.3 Estructura y funcionamiento**
 - 5.4 Tipos de driver JDBC**
 - 5.4.1 Driver JDBC-ODBC**
 - 5.4.2 Driver nativo**
 - 5.4.3 Driver intermedio**
 - 5.4.4 Driver puro-Java**
 - 5.5 Lenguaje SQL**
 - 5.6 El API JDBC**
 - 5.7 Uso de JDBC para acceso a Datos**
 - 5.7.1 Conexión a Base de Datos**
 - 5.7.2 Ejecución de consultas**
 - 5.7.3 Cierre de la conexión**
 - 5.7.4 Manipulación de registros**
 - 5.7.5 Información sobre los datos**
- 6 APLICACIONES BASADAS EN ENTORNO GRÁFICO (SWING)**
 - 6.1 Swing**
 - 6.2 Contenedores**
 - 6.2.1 Creación de una ventana**
 - 6.2.2 Personalización de ventanas**
 - 6.3 Uso de Layouts**
 - 6.3.1 Layout Null**
 - 6.3.2 Layout FlowLayout**
 - 6.3.3 Layout BorderLayout**
 - 6.3.4 Layout GridLayout**
 - 6.3.5 Layout BorderLayout**
 - 6.4 Agregar controles al contenedor**
 - 6.4.1 JDialog**
 - 6.4.2 JLabel y JButton**
 - 6.4.3 JTextField, JScrollPane, JTextArea**
 - 6.4.4 JComboBox**
 - 6.4.5 JCheckBox**
 - 6.4.6 JRadioButton**
 - 6.4.7 JMenuBar, JMenu y JMenuItem**
 - 6.5 El modelo de gestión de Eventos en Java**

1 INTRODUCCIÓN

1.1 ¿Que es Java?

El lenguaje de programación Java fue originalmente desarrollado por James Gosling de Sun Microsystems y publicado en 1995 como un componente fundamental de la plataforma Java de la misma compañía, que sería posteriormente comprada por Oracle. Su sintaxis deriva en gran medida de C y C++, pero tiene menos acceso de bajo nivel que cualquiera de ellos. Las aplicaciones de Java son generalmente compiladas a bytecode (clase Java) que puede ejecutarse en cualquier máquina virtual Java (JVM) sin importar la arquitectura de la máquina subyacente.

La compañía Sun desarrolló la implementación de referencia original para los compiladores de Java, máquinas virtuales, y librerías de clases en 1991 y las publicó por primera vez en 1995. A partir de mayo de 2007, en cumplimiento con las especificaciones del Proceso de la Comunidad Java, Sun volvió a licenciar la mayoría de sus tecnologías de Java bajo la Licencia Pública General de GNU. Otros también han desarrollado implementaciones alternas a estas tecnologías de Sun, tales como el Compilador de Java de GNU y el GNU Classpath.

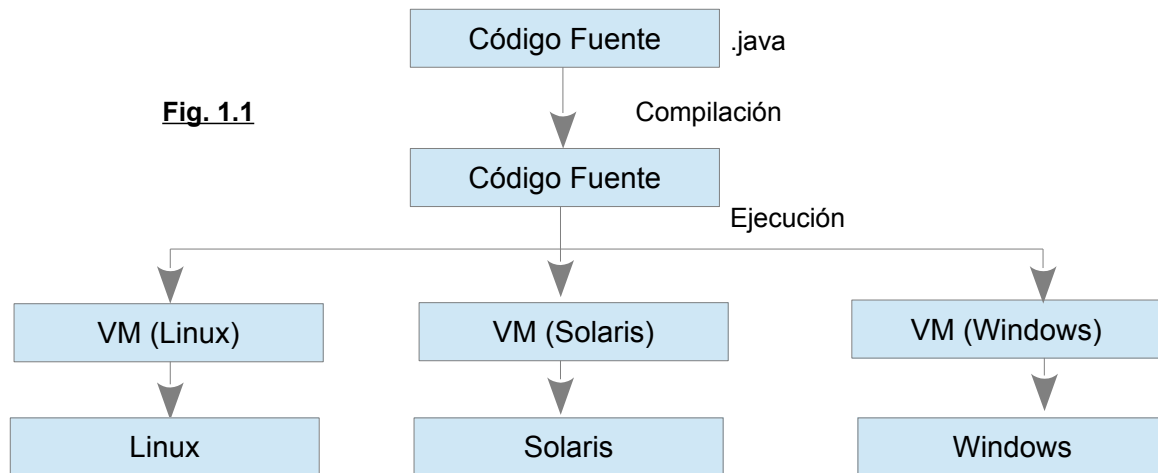
1.2 Principales características

Los principales elementos que posee el lenguaje son los siguientes:

- **Orientado a Objetos:** incluye todos los conceptos que tiene esta técnica: Abstracción, Encapsulamiento, Modularidad, Polimorfismo, Herencia o Recolección de basura
- **Amplio conjunto de librerías:** programar con Java no sólo incluye el uso del juego de instrucciones propio del lenguaje, sino que además permite usar un amplio conjunto de clases que Oracle/Sun pone a disposición del desarrollador con los que es posible desarrollar cualquier tipo de aplicación. Algunos ejemplos son: interfaces gráficas, gestión de redes, multitarea o acceso a bases de datos que ya veremos posteriormente.
- **Aplicaciones Multiplataforma:** Una vez compilado el programa, este puede ser ejecutado en diferentes sistemas operativos sin necesidad de efectuar cambios en el código fuente y sin que haya que volver a compilarlo, es decir: *“Compila una vez y ejecutar en cualquier plataforma”*.
- **Introduce una capa de seguridad:** Para empezar el lenguaje no posee instrucciones que permitan un acceso directo a la memoria (los potentes y peligrosos punteros de C/C++). Por otro lado, la máquina virtual impone ciertas restricciones a las aplicaciones para garantizar una ejecución segura.
- **Amplio soporte de fabricantes software:** Al no estar Java vinculada a ninguna plataforma existe una amplia variedad de productos de software de multitud de fabricantes y distintos entornos (Linux, Windows, Solaris, MacOS).

1.3 Máquina Virtual de Java

La Máquina Virtual Java (JVM – Java Virtual Machine) es el entorno de ejecución de las aplicaciones Java, cuya principal finalidad es adaptar los programas Java compilados a las características del sistema operativo donde se van a ejecutar. En la Figura 1.1 tenemos un gráfico que explica todo el proceso de compilación y ejecución de aplicaciones.



Toda aplicación Java está organizado en clases. Éstas se codifican en archivos de texto con extensión .java. Cada archivo de código fuente .java puede, a su vez, contener una o varias clases, aunque lo usual es que haya solo una.

Cuando se compila el archivo .java se genera uno o varios archivos .class de código binario (uno por cada clase), denominados bytecodes, que son independientes de la arquitectura de la máquina. Este hecho supone que no pueden ser ejecutados directamente por el sistema operativo. Así, durante esta fase de ejecución los archivos .class se someten a un proceso de interpretación, consistente en traducir los bytecodes a código ejecutable por el sistema operativo. Esta operación lo realiza un software conocido como Máquina Virtual de Java (JVM).

Cada sistema operativo incluye su propia implementación de la (JVM) y de hecho, hoy en día, dicha máquina es un componente mas del propio sistema.

1.4 Ediciones Java

Java dispone de un amplio conjunto de paquetes (librerías) de clases para la realización de las aplicaciones.

Este compendio de clases se organiza en 3 grandes grupos o Ediciones:

- **Java 2 Standard Edition (J2SE) 1.7:** Paquetes de uso general (cadenas, uso de datos, etc). Incluye también entornos gráficos y aplicaciones para navegadores (Applets).
- **Java 2 Enterprise Edition (J2EE) 1.7:** Paquetes y tecnologías necesarias para la creación de aplicaciones multicapa y entre ellas las ejecutables en entorno web (como JSP).
- **Java 2 Micro Edition (J2ME) 1.7:** Incluye paquetes y especificaciones para aplicaciones ejecutables en dispositivos electrónicos limitados (móviles, tablets, etc). De estos deriva el mundialmente conocido Android.

1.5 Primeros Pasos

1.5.1 El JDK

El Java Development Kit (JDK) proporciona el conjunto de herramientas básico para el desarrollo de aplicaciones Java. Se puede obtener de manera gratuita en el siguiente enlace:

<http://www.oracle.com/technetwork/es/java/javase/downloads/index.html>

Java SE Development Kit 7u71		
You must accept the Oracle Binary Code License Agreement for Java SE to download this software.		
☐ Accept License Agreement ☒ Decline License Agreement		
Product / File Description	File Size	Download
Linux x86	119.44 MB	 jdk-7u71-linux-i586.rpm
Linux x86	136.76 MB	 jdk-7u71-linux-i586.tar.gz
Linux x64	120.81 MB	 jdk-7u71-linux-x64.rpm
Linux x64	135.63 MB	 jdk-7u71-linux-x64.tar.gz
Mac OS X x64	185.84 MB	 jdk-7u71-macosx-x64.dmg
Solaris x86 (SVR4 package)	139.36 MB	 jdk-7u71-solaris-i586.tar.Z
Solaris x86	95.48 MB	 jdk-7u71-solaris-i586.tar.gz
Solaris x64 (SVR4 package)	24.68 MB	 jdk-7u71-solaris-x64.tar.Z
Solaris x64	16.36 MB	 jdk-7u71-solaris-x64.tar.gz
Solaris SPARC (SVR4 package)	138.74 MB	 jdk-7u71-solaris-sparc.tar.Z
Solaris SPARC	98.62 MB	 jdk-7u71-solaris-sparc.tar.gz
Solaris SPARC 64-bit (SVR4 package)	23.94 MB	 jdk-7u71-solaris-sparcv9.tar.Z
Solaris SPARC 64-bit	18.35 MB	 jdk-7u71-solaris-sparcv9.tar.gz
Windows x86	127.78 MB	 jdk-7u71-windows-i586.exe
Windows x64	129.52 MB	 jdk-7u71-windows-x64.exe

Elegimos la plataforma

correspondiente (x86 de 32 Bits o x64 de 64Bits) y lo instalamos en el equipo. De todos modos, para equipos linux recomendamos el uso de los repositorios correspondientes.

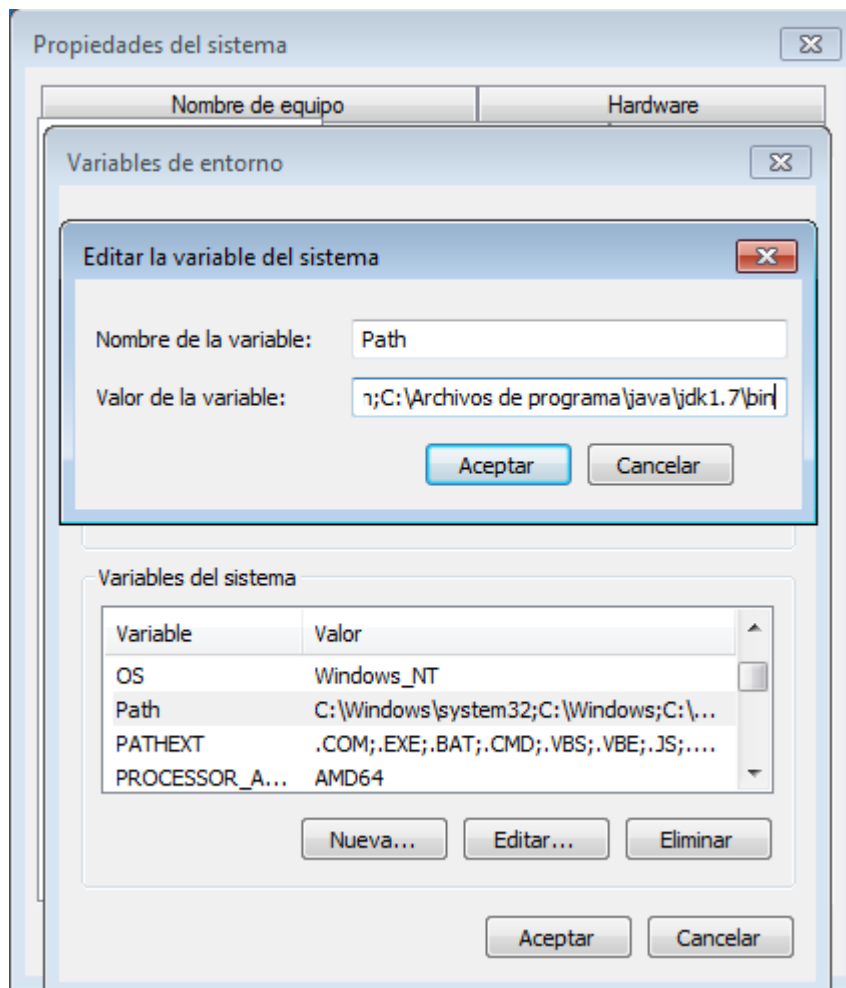
Una vez instalado, el JDK proporciona lo siguiente:

- La implementación de la JVM (Máquina Virtual) para el sistema operativo indicado durante el proceso de descarga del JDK.
- Herramientas para la compilación y ejecución de programas. Estos deben ser usados a través de la consola de comandos. La relación es la siguiente:
 - El Compilador: **javac.exe**
 - El Depurador: **jdb.exe**
 - El Intérprete: **java.exe** y **javaw.exe**
 - El visualizador de applets: **appletviewer.exe**
 - El generador de documentación: **javadoc.exe**
 - El desensamblador de clases: **javap.exe**
 - El generador de archivos fuente y de cabecera para clases nativas c: **javah.exe**
- Paquetes de clases del J2SE

1.5.2 Configurar Variables de entorno

Para configurar las variables de entorno, en Windows7, hacemos el siguiente proceso:

1. Copiamos la dirección donde se encuentra el SDK que suele ser:
C:\Archivos de programa\java\jdk1.7\bin
NOTA: En función del JDK que usemos la carpeta jdk1.7 puede cambiar.
2. Botón derecho sobre Equipo > Propiedades > Configuración avanzada del sistema.
3. Le damos a [Variables de entorno...] y en Variable de sistema (en la zona inferior) hacemos doble clic en Path.
4. Al final de Valor de la variable, ponemos un punto y coma (";") y añadimos la ruta del SDK.

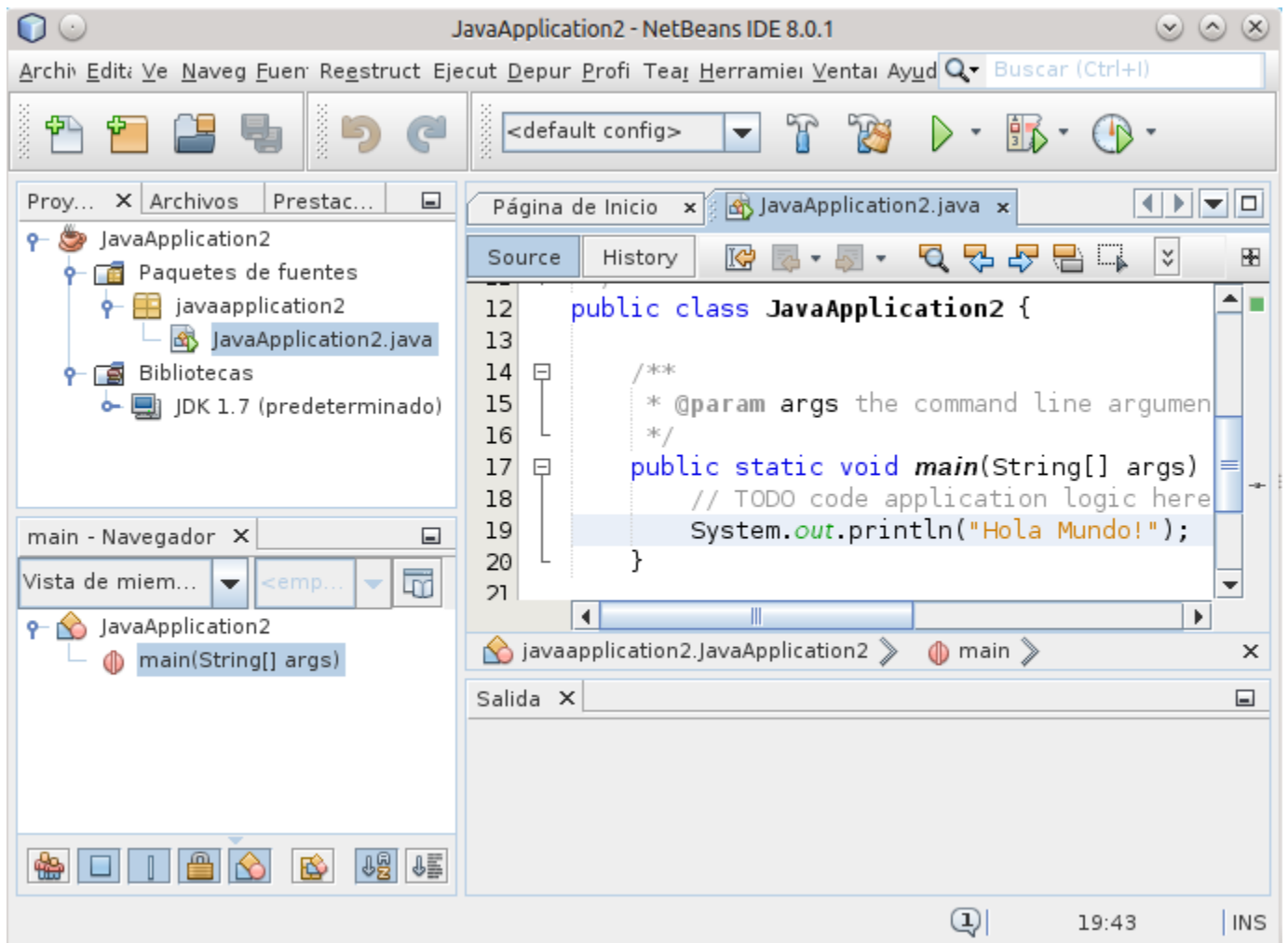


1.5.3 INSTALACIÓN DE NETBEANS 8

Los pasos para hacer la instalación de NETBEANS son los siguientes:

1. Nos vamos a su página de descargas: <https://netbeans.org/downloads/>
2. Descargamos el paquete propio de nuestra plataforma e idioma (en mi caso he elegido Linux e Idioma Español). Elegimos el [Download] de la columna ALL
3. Se descarga un archivo sh (para Linux) o un DMG para MacOS o bien un EXE para Windows. Al terminar, en Linux, debemos dar permisos de ejecución al archivo:
sudo chmod +x (Nombre del archivo).sh
4. Y por último lo ejecutamos:
sudo sh (Nombre del Archivo).sh
5. Durante la instalación, pulsamos en Personalizar y añadimos Apache Tomcat.
6. Una vez instalado tendremos la interfaz de NETbeans lista para ser usada.

NOTA: Para la instalación de NETBEANS en Windows se realiza mediante un instalador que guiará todo el proceso y que no necesita ejecutar ningún comando vía terminal.



1.5.4 Otros entornos de Desarrollo (IDEs)

Existen en el mercado, además de NETBeans (proyecto iniciado por el propio Sun) otros entornos de desarrollo igualmente válidos y algunos gratuitos, entre los que destacan:

- Eclipse: El entorno mas usado a nivel profesional, completo y lleno de funcionalidades. Incluye la posibilidad de instalar plugins para aumentar la funcionalidad.
<https://www.eclipse.org/>
- JBuilder: Entorno comercial muy orientado a las aplicaciones gráficas.
<http://www.embarcadero.com/products/jbuilder>
- BlueJ: Entorno muy sencillo y fácil de manejar, empleados en algunos círculos académicos para la enseñanza de Java (por ejemplo en la UNED).
<http://www.bluej.org/>

1.6 Creación del Primer programa Java

Los pasos para crear nuestro primer programa Java serán los siguientes:

1. Iniciamos NetBeans y pulsamos en Archivo > Proyecto Nuevo
2. En Categoría elegimos Java y en Proyectos, Java Application. Clic en [Siguiente]
3. Como Ubicación del proyecto, dejamos puesto NetBeansProject (o bien nos creamos una carpeta donde guardaremos el resto de ejemplos).
NOTA: Recomiendo guardar los ejemplos por temas. Para este caso, **Tema01**.
4. Como nombre de proyecto, ponemos Ejemplo01_HolaMundo. Clic en [Terminar]
5. Veremos que se nos abre el archivo Ejemplo01_HolaMundo.java.
6. Por ahora, que no conocemos los elementos del lenguaje, ponemos lo siguiente (en negrita):

```
package ejemplo01_holamundo;
/**
 *
 * @author ivan
 */
public class Ejemplo01_HolaMundo {
    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        // TODO code application logic here
        System.out.println("Hola Mundo!");
    }
}
```

NOTA: Por convención se pone los nombres de las clases la primera letra en mayúsculas.

7. Para ejecutar el proyecto actual, pulsamos Ejecutar > Ejecutar proyect o bien F6. Veremos el resultado de la ejecución en la zona inferior derecha del entorno de desarrollo.

1.7 Conceptos básicos de programación en java

Java está totalmente orientado a objetos. Como tal, todo programa Java debe estar escrito en una o varias clases, dentro de las cuales se podrán usar un amplio surtido de paquetes prediseñados. Aunque trataremos el concepto de Programación Orientada a Objetos mas adelante, vamos a explicar algunos conceptos básicos para seguir con el aprendizaje del lenguaje.

1.7.1 Objetos

Desde el punto de vista de la programación, entendemos como objeto a una entidad propia, opaca al exterior, que expone una serie de operaciones (métodos) que pueden ser usados por otros programas para la realización de tareas mayores, imitando el comportamiento de objetos reales del mundo real. Así, por ejemplo, si tenemos el objeto “**Camión**”, vamos a exponer al exterior los métodos **encender()**, **cambiarmarcha()**, **acelerar()** para permitir que un usuario emplee dicho objeto.

Para poder invocar a los métodos de un objeto desde fuera del mismo es necesario tener la **referencia del objeto**. Normalmente esta se guarda mediante el operador **PUNTO (“.”)**, empleándose del siguiente modo:

```
referenciaObjeto.metodo1();  
referenciaObjeto.metodo2(argumento1, argumento2,...)
```



```
referenciaCamion.encender();  
referenciaCamion.cambiarMarcha(“subir”, 1);
```

Algunos métodos necesitan que se les proporcione una serie de datos (argumentos de llamada) para poder realizar su función. Los argumentos deben ser aportados en la llamada al método, situados entre **PARENTESIS ()** y separados por **COMAS (“,”)**, a continuación del nombre del método tal y como se refleja en el ejemplo anterior, tras arrancar el camión, subimos una marcha en una unidad.

```
referenciaCamion.cambiarMarcha(“subir”, 1);
```

Es importante reseñar que aquellos métodos que no necesitan argumentos, la sintaxis de Java les obliga a utilizar parentesis en la llamada a los mismos:

```
referenciaCamion.encender();
```

1.7.2 Clases

Las clases contienen la definición de los objetos, es decir, es el lugar donde se codifican los métodos que van a exponer los objetos de una clase. Siguiendo con el simil del camión, una marca y modelo de camión sería la clase (cada modelo de camión define sus propios métodos distintos a otros modelos de camión), mientras que un camión concreto de esa marca y modelo sería el objeto (en tal caso estamos hablando de una **instancia del objeto**).

En Java se define una clase de la siguiente manera:

```
[Public] class NombreClase  
{  
    // Declaración de campos o atributos  
    // Declaración de métodos  
}
```

Opcionalmente la palabra class puede estar precedida por el modificador de acceso public, que será explicado mas adelante. Por ahora, basta decir que una clase definida como public debe ser almacenada en un archivo con extensión **.java** cuyo **nombre debe ser exactamente el mismo que el de la clase**.

Una vez definida la clase con sus métodos, los programadores podrán crear objetos (instancias) de las mismas para poder usar sus métodos. Las instancias u objetos de una clase se deben crear con el operador **new**, que crea la instancia, la almacena en memoria y devuelve una referencia a la misma que normalmente se guarda en una variable para, posteriormente, invocar a los métodos del objeto.

Variable que almacena la referencia al objeto

Creación del objeto

nombreClase v = **new** nombreClase();
v.metodo1();

Llamada a los métodos del objeto

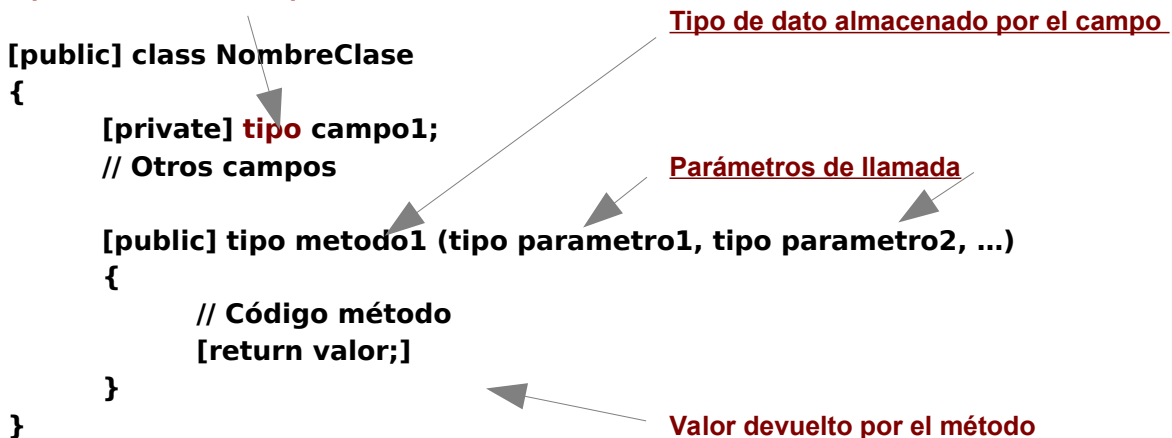
Veamos un ejemplo práctico:

```
Camion camionVolvo = new Camion();  
camionVolvo.encender();
```

1.7.3 Métodos y Campos

Los métodos definen los comportamientos de los objetos de una clase. Estos métodos pueden hacer uso de campos o atributos para almacenar información sobre un objeto y que puede ser usada posteriormente por cualquiera de los métodos del objeto. Por ejemplo, la clase **Camion** debería tener un campo **marcha** donde almacenar la marcha actual del camion, de este modo el método **cambiarMarcha**, podrá usar este campo para mantener actualizado en cada momento el valor de la marcha del camión (evidentemente, si está a 0 no empieza a andar).

Tipo de dato devuelto por el método



En Java los métodos se implementan mediante funciones y los campos mediante variables. La palabra `tipo` representa un tipo de dato válido para Java y puede ser un tipo de dato primitivo (que veremos en el próximo capítulo) o un tipo objeto y debe ser utilizado para indicar el tipo de dato de los campos, los parámetros de métodos y el valor de devolución de estos.

Los métodos de una clase Java pueden recibir determinados datos (argumentos) en la llamada. Los valores recibidos por el método se declaran en la cabecera de este como parámetros de llamada. La declaración de parámetros de un método sigue el formato de la declaración de variables de Java.

Cuando se invoca a un método utilizando argumentos, estos son copiados en los parámetros declarados en la cabecera del método. Así pues, los argumentos de llamada deben coincidir en número y tipo con los parámetros declarados.

Llamada al método

```
objeto.metodo(argumento1, argumento1);
```

Declaración del método

```
tipo metodo(tipo parametro1, tipo parametro2) { // ... }
```

Opcionalmente, un método puede devolver un resultado al punto de llamada, para lo cual se utiliza la palabra `return` en alguna parte del cuerpo del método (normalmente al final), seguida del valor a devolver. El tipo de devolución del método se indica en la definición del mismo, delante del nombre. En caso de que no devuelva ningún resultado el tipo de devolución será **void**.

Por ejemplo:

```
int marchaCamion = referenciaCamion.cambiarMarcha("subir", 1);
```

Aquí damos por sentado que `cambiarMarcha` devuelve un entero con la marcha final del camión. Dentro del método tomaremos la marcha actual, veremos la opción elegida (subir o bajar), así como el número de marchas a subir y bajar y actuar en consecuencia.

Por tanto, cuando se hace una llamada a un método que devuelve un valor, este deberá ser almacenado en una variable del tipo correspondiente o utilizado en una expresión:

tipo variable = instancia.metodo();

Otro aspecto a destacar de la definición de una clase son los modificadores de acceso. Como ya se vio antes, una clase puede tener un modificador `public` que a su vez se puede emplear en la definición de los métodos para permitir el acceso de los mismos desde el exterior de la clase.

En el caso de los campos o atributos, suele usarse el modificador `private` para impedir que puedan ser usados directamente desde el exterior, forzando a que el acceso a los mismos se haga a través de métodos específicos de la clase (los getters y setters). Este mecanismo de protección se conoce en programación orientada a objetos como encapsulación, que ya veremos con más detenimiento más adelante.

Siguiendo el ejemplo del camión, la implementación del campo `marchaCamion` y el método `cambiarMarcha` quedaría como se indica en este código:

```
public class Camion
{
    private int marchaCamion;
    public void cambiarMarcha (String modo, int valor)
    {
        if (modo.equals("subir"))
            marchaCamion += valor;
        else
            marchaCamion = valor;
    }
}
```

En el siguiente código, creamos una instancia del camión y llamamos al método `cambiarMarcha`:

```
Camion camionVolvo = new Camion ( );  
camionVolvo.cambiarMarcha("subir",1);
```

1.7.4 Métodos y campos estáticos

Según lo visto anteriormente, para poder invocar a un método es necesario en primer lugar crear un objeto o instancia de la clase correspondiente. A partir de ahí, puede utilizarse la variable que almacena la referencia al objeto para llamar a los métodos.

Esto es así porque, normalmente, los métodos y los campos que estos utilizan están asociados a los objetos. Así, por ejemplo, el campo `marchaCamion` y el método `cambiarMarcha()` están asociados a un objeto camión particular (cada camión tendrá su forma de iniciar la marcha).

Sin embargo no todos los métodos y campos de una clase tienen que estar asociados a un objeto particular de la misma. Puede haber métodos genéricos cuya tarea a realizar no dependa de un objeto particular. Por ejemplo, el objeto `camion` podría tener un método `obtenerPrecio()` cuya ejecución generaría el mismo resultado para cualquier objeto de la clase, pues todos los objetos `camion` tendrían el mismo precio (suponemos un único modelo).

A este tipo de métodos se les conoce como métodos estáticos y puesto que no dependen de un objeto en particular de la clase, se les puede invocar sin necesidad de crear objetos utilizando la sintaxis siguiente:

NombreClase.metodo();



Camion.obtenerPrecio();

De la misma que tenemos métodos estáticos, también podemos tener campos o variables estáticas, cuyos valores no están asociados a objetos concretos de la clase. Por ejemplo, un campo estático para camion sería numRuedas, ya que este valor no depende de un camión específico en particular, sino de una marca y modelo. Este tipo de campos son también llamados **variables de clase**. La definición de un método y campo estático en Java tendrá la siguiente sintaxis:

```
[public] class NombreClase
{
    static tipo campo1;

    static tipo metodo1(parametros)
    {

        // Código método

    }
}
```

Volviendo al ejemplo del camión:

```
public class Camion
{
    public static int numRuedas = 6;
    public static float obtenerPrecio ()
    {

        // Instrucciones para devolver el precio

    }
}
```

A diferencia de los campos estándar, que suelen ser privados para uso interno de la clase, los campos estáticos deben llevar un modificador de acceso que permita su uso desde el exterior de la misma. Este puede ser public, protected o no llevar ninguno (ya se verá que significa cada uno).

La implementación de un método estático impone algunas restricciones en el código:

- **Un método estático no puede hacer referencia a elementos no estáticos de su misma clase.** Lógicamente si un método estático no depende de ningún objeto particular de la clase, no puede hacer referencia dentro del código a campos y métodos que sí dependan de los objetos. Por ejemplo, dentro del método obtenerPrecio() no se podría hacer referencia al campo marchaCamion, dado que este tendría un valor específico para cada objeto.
- **Un método estático no puede hacer uso de super y this.** Estas palabras reservadas se estudiarán más adelante en el capítulo dedicado a la Programación Orientada a Objetos.

1.7.5 El método main ()

Toda aplicación Java está compuesta por al menos una clase; incluso una sencilla aplicación que genere el saludo Hola Mundo. En alguna de la clases que componen una aplicación, que ha de ser declarada con el modificador de acceso public, debe existir un método estático llamado main(), cuyo formato debe ser el siguiente:

```
public static void main (Strings [ ] args)  
{  
    // Código del método  
}
```

El método main () debe cumplir con las siguientes características:

- Ha de ser público
- Ha de ser un método estático
- No puede devolver ningún resultado (tipo devolución void)
- Ha de declarar un array de cadena de caracteres en la lista de parámetros o un número variable de argumentos.

El método main () es el punto de arranque de una aplicación Java, cuando se invoca al comando java.exe desde la línea de comandos, el JVM busca en la clase indicada un método main () con el formato indicado anteriormente. Dentro del código main () pueden crearse objetos de otras clases e invocar a sus métodos. En general, se pueden incluir cualquier tipo de lógica que respete las restricciones indicadas para los métodos estáticos.

Es posible suministrar parámetros al método main() a través de la línea de comandos. Para ello, los valores a pasar deberán especificarse a continuación de la clase, separados por un espacio.

```
> java nombre_clase arg1 arg2 arg3
```

Los datos llegarán al método main() en forma de un array de cadenas de caracteres (que ya trataremos mas adelante). Por ejemplo, dada la siguiente clase:

Nos creamos un nuevo proyecto de NetBeans llamado **Ejemplo02_UsoMain**

```
public class Ejemplo02_UsoMain {  
    public static void main(String[] args) {  
        System.out.println(args[0]);  
        System.out.println(args[1]);  
        System.out.println(args[2]);  
        System.out.println(args[3]);  
    }  
}
```

Para ejecutarlo tendremos que pasarle los 4 argumentos al compilador.

Para hacerlo con NetBeans, botón derecho sobre el nombre del proyecto, Propiedades > Ejecutar.

En argumentos ponemos los 4 argumentos: Hola Curso de Java

Al ejecutar la aplicación veremos como queda una palabra encima de la otra.

2 SINTAXIS DEL LENGUAJE

2.1 Sintaxis Básica

Empecemos por ver algunos aspectos sintácticos generales:

- **El Lenguaje es sensible a mayúsculas y minúsculas.** Así, no es lo mismo poner public como modificador de acceso que Public (daría en este caso un error). Esto se aplica no solo a las palabras reservadas, sino también a variables y métodos.
- Las sentencias acaban con **PUNTO Y COMA (";")**. Al igual que otros lenguajes como C++.
- Los bloques de instrucciones se delimitan con **LLAVES { }**.
- Tenemos tres tipos de comentarios:
 - Comentarios de una línea: van precedidos de **DOBLE BARRA //**
 - Comentarios multilínea: empieza con **BARRA ASTERISCO /*** y terminan con **ASTERISCO BARRA */** (es decir **/* */**)
 - Comentarios para JavaDoc que son **/**** y cierre con ***/**

2.2 Secuencias de Escape

Hay determinados caracteres en Java que o bien no tienen representación explícita o bien no pueden ser usados directamente por el hecho de tener un significado especial para el lenguaje. Para poder utilizar estos caracteres dentro de una aplicación Java se usan Secuencias de Escape.

Una secuencia de escape está formado por el caracter **BARRA INVERTIDA ("\"**) seguido de una letra, en el caso de caracteres no imprimibles, o de un carácter especial. En la siguiente tabla mostramos todas las secuencias de escape de Java:

Secuencia de Escape	Significado
\b	Retroceso
\n	Salto de Línea
\t	Tabulación Horizontal
\\	Barra Invertida \
\'	Comilla Simple
\"	Comilla Doble

Veamos un ejemplo (creamos un proyecto en NetBean dentro de una carpeta llamada Tema02):

```
public class Ej01_SecuenciasEscape {  
    public static void main(String[] args) {  
        System.out.println("Contenido del Curso: ");  
        System.out.println(" Mod1> \t \"MySQL\" \n Mod2 > \t \"Java\"");  
        System.out.println(" Mod3> \t \"JSP\" \n Mod4 > \t \"PHP\"");  
    }  
}
```

2.3 Tipos de Datos Primitivos

Toda la información que se maneja en una aplicación Java puede estar representada bien por un objeto o bien por un dato básico o de tipo primitivo. Java soporta 8 datos primitivos que se indican en la tabla inferior.

Entre dichos tipos primitivos no está ninguno que represente una cadena de caracteres. El motivo es que Java los trata como objetos, concretamente objetos de la clase String. En el próximo capítulo lo veremos con mas detalle.

Tipo	Tamaño	Ejemplo	Rango de Valores
byte	8 bits	byte a;	-128 : 127
short	16 bits	short b, c=3;	-32,768 : 32,767
int	32 bits	int d = -30; int e = 0xC125;	-2,147,483,648 : 2,147,483,647
long	64 bits	long b = 434123; long b=5L; /* L indica Long */	-9,223,372,036,854,775,808 9,223,372,036,854,775,807
char	16 bits	char car1 = 'c'; char car2=99; /* Ambos son iguales */	'\u0000' : '\uffff'
float	32 bits	float pi = 3.1416; float pi = 3.1416F; /* F indica float */ float medio = 1/2F; /* 0.5 */	-3,4·10 ⁻³⁸ +3,4·10 ⁺³⁸
double	64 bits	double millon=1e6; /* 1x10 ⁶ */ double medio 1/2D; /* D indica double */	-1,7·10 ⁻³⁰⁸ +1,7·10 ³⁰⁸
boolean	JVM	boolean bool = true;	

Los tipos primitivos se pueden organizar en 4 grupos:

- **Numéricos enteros.** Son los tipos byte, short, int y long. Los cuatro representan números enteros con signo.
- **Carácter.** El tipo char representa un carácter codificado en el sistema unicode.
- **Numérico decimal:** Los tipos float y double representan números decimales de coma flotante.
- **Lógicos:** El tipo boolean es el tipo de dato lógico. Los dos únicos valores permitidos son true o false. En Java no existe equivalencia entre estos valores (ademas que son palabras reservadas) y valores numéricos (como el 0 y 1). En cuanto al tamaño en bits del tipo boolean, este dependerá de la máquina virtual.

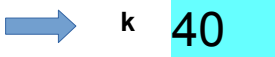
2.4 Variables

Una variable es un espacio físico dentro de la memoria donde un programa puede almacenar un dato para su posterior uso.

2.4.1 Tipos de Datos de una Variable

En Java las variables pueden usarse para tratar los tipos de datos indicados anteriormente:

- Tipos primitivos: Las variables que se declaran de un tipo primitivo almacenan el dato real según la figura inferior:

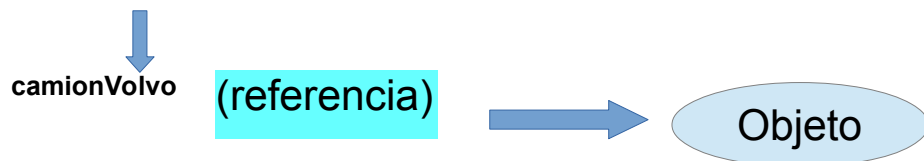
`int k = 40;`  `k` 40

- Tipo Objeto: Como ya se mencionó previamente, los objetos en Java también se tratan como variables, solo que a diferencia de los datos primitivos, una variable de tipo objeto no contiene el objeto como tal, sino una referencia al mismo. No debe importarnos el valor exacto del contenido de la variable. Tan solo tenemos que saber que a través del operador “.” podemos acceder con ella a los métodos del objeto referenciado, utilizando la expresión:

`variable_objeto.metodo();`

Ej:

`Camion camionVolvo = new Camion ( );`



2.4.2 Declaración de Variables

Una variable se declara de la siguiente forma:

tipo_dato nombre_variable

Un nombre de variable debe cumplir con las siguientes reglas:

- Debe comenzar con un carácter alfabético
- No puede contener espacios, signos de puntuación o secuencias de escape.
- No puede utilizarse como nombre de variable una palabra reservada Java.

También es posible declarar en una misma instrucción varias variables, siempre que sean del mismo tipo. La sintaxis en tal caso será:

tipo_dato nombre_variable1, nombre_variable2, ..., nombre_variableN;

Válidas	No válidas
<code>int k, cod;</code>	<code>boolean 7q // Comienza por un número</code>
<code>long p1;</code>	<code>int num uno; // Continene un espacio</code>
<code>char cad_2;</code>	<code>long class; // Usa una palabra reservada</code>

2.4.3 Asignación

Una vez declarada una variable se le puede asignar un valor siguiendo el formato:

variable = expresion;

donde expresión puede ser cualquier expresión Java que devuelva un valor acorde con el tipo de dato de la variable. Veamos un ejemplo (tema02/Ej02_AsignacionVariables)

```
public class Ej02_AsignacionVariables {  
    public static void main(String[] args) {  
        int p, k, v;  
        p=30;  
        k=p+20;  
        v=k*p;  
  
        System.out.println("El Valor de la variable p es: "+p);  
        System.out.println("El Valor de la variable k es: "+k);  
        System.out.println("El Valor de la variable v es: "+v);  
    }  
}
```

También es posible asignar un valor inicial a una variable en la misma instrucción de la declaración. Veamos un par de ejemplos:

```
int num=5;      // Declara la variable y la inicializa  
int p, n=7;    // Declara dos variables y solo inializa la segunda.
```

2.4.4 Literales

Un literal es un valor constante que se puede asignar directamente a una variable o puede ser usado en una expresión.

Existen cuatro tipos de literales básicos, coincidiendo con los cuatro grupos en los que se puede dividir los tipos básicos de Java, esto es, numéricos enteros, numéricos decimales, lógicos y carácter.

A la hora de utilizar estos literales en una expresión hay que tener en cuenta lo siguiente:

- **Los literales numéricos enteros se consideran como tipo int.**

Dado que todo literal entero es un tipo int, una operación como:

byte b=10;

intentaría asignar un número de tipo int a una variable de tipo byte, lo que a priori podría provocar un error de compilación. Sin embargo, como veremos mas adelante, en expresiones de este tipo Java realiza una conversión implícita del dato al tipo de destino, siempre y cuando dicho dato entre dentro del tamaño permitido por la variable.

- **Los literales numéricos decimales se consideran como tipo double.** Una asignación del tipo:
float p = 3.14;

provoca un error de compilación al intentar asignar un dato double a una variable float que tiene un tamaño menor. Para evitar el error, se debe usar la letra f a continuación del número:

```
float p = 3.14f
```

lo que provoca una conversión del número double a float.

- **Los literales boolean son true y false** Estas palabras reservadas no tienen equivalencia numérica, por lo que la siguiente instrucción provocará un error de compilación de incompatibilidad de tipos:

```
boolean b = 0;
```

- Los literales de tipo char se escriben entre comillas simples. Se puede usar la representación del carácter o su valor unicode en hexadecimal con el carácter \u por delante. Es decir, son equivalente y correctos:

```
char carácter = 'ñ';
```

```
char laÑ = '\u00F1';
```

Mas información: <http://unicode-table.com/en/#basic-latin>

2.4.5 Ámbito de las variables

En función de donde esté declarada la variable, esta puede ser:

- **Campo o atributo:** Estas variables se declaran al principio de la clase, fuera de los métodos. Son variables que se comparten por todos los métodos de la clase y suelen declararse como private para limitar su uso al interior de la clase, aunque en ocasiones se usarán otros modificadores de acceso para permitir su uso en clases externas. **Las variables atributo pueden ser usadas sin haber sido inicializadas de forma explícita**, ya que se inician automáticamente al crear un objeto de la clase.
- **Local:** Son variables que se declaran dentro de un método. Su ámbito se reduce al interior del propio método y no permiten ningún modificador. Una variable local se crea en el momento de la llamada al método, destruyéndose al acabar la ejecución de este. Una variable local también se puede declarar dentro de un bloque de instrucciones y su uso quedará reducido a ese bloque concreto de instrucciones. **Toda variable local tiene que ser inicializada explícitamente antes de ser usada.**

```
public class factorial
{
    private int resultado;
    public int calculaFactorial (...) {
        int valor = 1;
    }
}
```

2.4.6 Valores por defecto de una variable

Las variables de tipo atributo son inicializadas implícitamente. El valor que toma al ser inicializada se conoce como **valor por defecto o predeterminado** y depende del tipo declarado:

Tipo de Variable	Valor por defecto
byte, short, int, long	0
char	\u0000
float, double	0.0
boolean	false
objeto	null

Las variables locales no son inicializadas de forma implícita, siendo necesario asignarles un valor antes de usarlas. Por ejemplo, el siguiente código generará un error de compilación:

```
void suma () {  
    int rdo, n1, n2;  
    rdo = n1 + n2;           // Error de compilación  
}
```

2.4.7 Conversiones de tipo

Java es un lenguaje fuertemente tipado, lo que significa que es muy estricto a la hora de asignar valores a las variables en función del tipo declarado. De todos modos, en determinadas circunstancias, es posible realizar conversiones que permitan almacenar en una variable un dato diferente al declarado.

Podemos hacer conversiones de tipo entre todos los tipos básicos exceptuando boolean. Dichas conversiones pueden ser implícitas o explícitas.

2.4.8 Conversiones Implícitas

Estas conversiones son automáticas y son realizadas por el compilador antes de almacenar el valor en la variable. Veamos un ejemplo:

```
int i;  
byte b = 10;  
i = b;
```

En este ejemplo, el dato de tipo byte almacenado en la variable b es convertido a int antes de asignarlo a la variable i.

Para que la conversión pueda realizarse de forma implícita el tipo de variable destino debe ser igual o mayor en tamaño al tipo de origen, excepto en estos dos casos:

- Cuando la variable destino es entera y el origen es decimal (float o double) la conversión no podrá ser automática.
- Cuando la variable destino es char y el origen es numérico del tipo que sea, la conversión no podrá ser automática.

Veamos una serie de ejemplos:

```
int k=10, p;  
short s = 5;  
char c = 'ñ';  
float f;  
p = c;           // Conversión implícita de char a int  
f = k           // Conversión implícita int a float  
k = s           // Conversión implícita short a int
```

Sin embargo, con estos ejemplos, tendremos error de compilación:

```
int n;  
long l = 30;  
float f = 2.4f;  
char c;  
byte b = 5;  
n = l; // No se puede convertir de forma implícita long a int (< tamaño)  
c = b; // No se puede convertir de forma implícita byte a char  
n = f; // No se puede convertir de forma implícita float a int;
```

2.4.9 Conversiones Explícitas

Cuando se cumplan las condiciones para una conversión implícita, esta podrá realizarse utilizando la siguiente sintaxis:

variable_destino = (tipo_destino) dato_origen;

En este caso, el compilador convierte en primer lugar el dato_origen al tipo_destino y luego lo asigna, evitando errores de compilación. A esta operación se le conoce como casting o estrechamiento ya que al convertir un dato de un tipo superior en tamaño a uno inferior se realiza un acortamiento que en algunos casos puede provocar pérdida de datos, eso si, sin provocar errores de ejecución. Algunos ejemplos:

```
char c;  
byte b;  
int n = 300;  
double d = 34.6  
c = (char) d;           // Se elimina la parte decimal (truncado)  
b = (byte) n           // Se produce una pérdida de datos, pero no da error.
```

No se puede realizar conversiones entre objetos, aunque se puede asignar un objeto de una clase a una variable de clase diferente. Esto lo veremos mas adelante.

2.5 Operadores

Los operadores son símbolos que permiten realizar operaciones con los datos de un programa. Java dispone de una amplia variedad de operadores que estudiaremos con detenimiento en este apartado, clasificados según la función que realizan.

2.5.1 Aritméticos

Se usan para realizar operaciones aritméticas sobre valores de tipo numérico. Los mas importantes son los siguientes:

Operador	Descripción	Ejemplo
+	Suma dos valores numéricos	int n; n = 6 + 3;
-	Resta dos valores numéricos	int n; n = 6 - 3;
*	Multiplica dos números	int n; n = 6 * 3;
/	Divide dos números. El tipo de resultado depende de los operandos, pues si ambos son enteros, el resultado será entero.	int a=7, b=3; float x,y; x = a / b; // Resultado: 2 y = a / 3.0f; // Rdo: 2,33
%	Calcula el resto de una división entre dos números (el módulo).	int n; n = 7 % 2; // Resultado: 1
++	Incrementa una variable numérica en una unidad y lo asigna a una variable	int n = 5; n++; // Equivale a n=n+1
--	Decrementa una variable numérica en una unidad y lo asigna a una variable	int n = 6; n--; // Equivale a n = n-1

El operador + también se puede usar para concatenar cadenas:

```
System.out.println ("Hola" + " Clase"); // Mostrará Hola clase
```

También puede usarse el operador + para unir texto con el contenido de una variable que no sea de tipo texto, puesto que **Java convierte automáticamente los operandos que no sean cadena** antes de hacer la concatenación. Veamos un ejemplo:

```
public class Ej03_Cuadrado {  
    public static void main(String[] args) {  
        int rdo, n=5;  
        rdo = n * n;  
        System.out.println("El cuadrado de "+n+" es "+rdo);  
    }  
}
```

En cuanto a los operadores de incremento (++) y decremento (--) hay que tener precaución a la hora de su uso, puesto que se pueden situar delante de la variable (prefijo) o después (sufijo), lo que puede condicionar el resultado final de una expresión. Veamos un ejemplo:

```
public class Ej03b_Incremento {
    public static void main(String[] args) {
        int a=3, b=10, rdo;
        rdo = a++;
        System.out.println ("El valor de Resultado es "+rdo);
        System.out.println ("El valor de A es "+a);
        rdo = ++b;
        System.out.println ("El valor de Resultado es "+rdo);
    }
}
```

En la línea 2 el operador de incremento está colocado después de la variable lo que implica que primero asignará el valor actual de la variable “a” a la variable “b”, realizando posteriormente el incremento de “a”. Por otro lado, en la línea 4, dado que el operador está situado delante de la variable, se realizará primero el incremento de las variables “a” y “b” y después se multiplicarán sus contenidos.

Como norma general a la hora de usar los operadores de incremento y decremento en una expresión compleja, debemos tener en cuenta los siguientes puntos:

- Las expresiones de Java se evalúan de izquierda a derecha
- Si el operador se sitúa delante de la variable, se ejecutará primero la operación que representa antes de seguir evaluando la expresión.
- Si el operador se sitúa después de la variable, se utilizará el valor actual de la variable para evaluar la expresión y después se ejecuta la operación asociada al operador.

2.5.2 Asignación

Además del operador de asignación de otros lenguajes, (=), Java dispone de un conjunto de operadores que permiten simplificar el proceso de operar con variables y asignar el resultado de la operación a la misma variable. Son estos:

Operador	Descripción	Ejemplo
=	Asigna la expresión de la derecha al operando situado a la izquierda del operador.	int n; n = 4*5; // Asigna 20 a n
+=	Suma la expresión de la derecha a la variable situada a la izquierda del operador.	int n=4; n+=5; // Equivale a n=n+5
-=	Resta la expresión de la derecha a la variable situada a la izquierda del operador.	int n=5; n-=2; // Equivale a n=n-2
=	Multiplica la expresión de la derecha a la variable situada a la izquierda del operador.	int n=3; n=6; // Equivale a n=n*6
/=	Divide la expresión de la derecha a la variable situada a la izquierda del operador.	int n=8; n/=2; // Equivale a n=n/2
%=	Calcula el resto de la división entre la variable situada a la izquierda y la expresión a la derecha, asignando el resultado a la variable	int n=5; n%=2; // Equivale a n=n%2

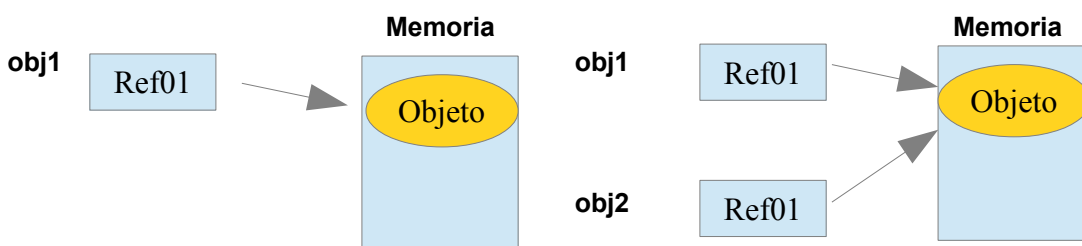
2.5.3 Asignación de referencias y asignación de valores

Existe una diferencia fundamental entre realizar una operación de asignación entre variables de tipo objeto y realizar la misma operación entre variables de tipo básico.

En el caso de tipo básico como por ejemplo un entero, la operación implica que el dato contenido en una variable se copia en otra variable. Esto mismo sucede con una variable de tipo objeto, pero en este caso **lo que se copia es una referencia al objeto, no el propio objeto**. Por tanto nos encontramos que no tenemos dos copias del objeto, sino un único objeto que está referenciado por dos variables. Vamos a verlo gráficamente:

Object obj1 = new Object()

Object obj2 = obj1



Sin embargo hay casos de objetos que son inmutables y que a pesar de compartir la referencia se comportan como objetos independientes llegado el caso: los String. Veamos un ejemplo:

```
public class Ej04_AsignacionReferencias {
    public static void main(String[] args) {
        int n1 = 10;
        int n2;
        n2 = n1; // Se copia el valor
        n1 = 20; // Modifico la primera variable
        System.out.println("Valor n1="+n1+" y valor n2="+n2);

        /*Ahora repitamos el proceso con un objeto String*/
        String s1 = "Hola";
        String s2; // Creo el objeto y lo dejo sin inicializar
        s2 = s1; // Asigno la referencia de s1 a s2
        s1 = "Curso";
        // ¿Que aparecerá como contenido de ambos?
        System.out.println("s1 = "+s1+" y s2="+s2);
    }
}
```

Vemos que, en este caso, el contenido de s1 ha cambiado sin cambiar el de s2.

Para otro tipo de objetos, si veríamos que el contenido sería el mismo, puesto que ambos apuntan, en realidad, al mismo objeto.

2.5.4 Condicionales

Se usan para establecer una condición dentro de un programa que dará lugar un valor booleano, verdadero (true) o falso (false). Estos operadores se usan en instrucciones de control de flujo (if, while y otros) que ya veremos mas adelante.

Operador	Descripción
==	Compara dos valores; si son iguales, el resultado es TRUE
<	Si el operando de la derecha es mayor que el de la izquierda es TRUE
>	Si el operando de la derecha es menor que el de la izquierda es TRUE
<=	Si el operando de la derecha es mayor o igual que el de la izquierda es TRUE
>=	Si el operando de la derecha es menor o igual que el de la izquierda es TRUE
!=	Si el valor de los operandos es diferente, el resultado es TRUE

2.5.5 Comparación de tipos básicos

Los operadores de comparación (<, >, <= y >=) únicamente podrán utilizarse para comparar enteros, puntos flotantes y caracteres. Si estos operadores se usan con referencias a objetos (como en el caso de los String), se produce un error de compilación.

```
String s = "hola";
char c = 'k';
if (c<=20) ...           // Permite comparar por valor unicode; Correcto.
If (s> "adios")...       // Error de compilación. Se debe usar equals
```

2.5.6 Igualdad de Objetos

Los operadores de igualdad "==" y desigualdad "!=" pueden usarse para comparar cualquier tipo de dato primitivo. Sin embargo no son válidos para variables de tipo objeto, ya que **estamos comparando referencias y no los propios objetos**.

Esto implica que tengamos dos variables referenciando a dos objetos iguales y que la condición de igualdad resulte falsa. Para comprobar la igualdad de objetos, las clases proporcionan un método llamado equals en el que cada clase implementa su propio criterio de igualdad. Debemos usar este método y no el operador ==, que únicamente daría TRUE si ambas variables apuntan al mismo objeto.

Veamos un ejemplo (donde hemos usado la instrucción de control if):

```
public class Ej05_UsoEquals {
    public static void main(String[] args) {
        String s1 = "hola";
        String s2 = "hola";
        if (s1.equals(s2))
            System.out.println("Ambas cadenas son iguales!");
    }
}
```

Por otro lado es importante reseñar que el uso del operador “==” únicamente está permitido para referencias del mismo tipo de objeto. Si se comparan dos tipos de objetos diferentes dará un error. Por ejemplo:

```
int i = 20;
String s = "hola";
if (i==s) { ... }    // Error de compilación
```

2.5.7 Lógicos

Operan con valores boolean siendo el resultado del mismo tipo boolean. Estos son los mas importantes:

Operador	Descripción
&&	Operador lógico AND. El resultado es TRUE si ambos operandos son TRUE. En cualquier otro caso será FALSE.
	Operador OR. El resultado es TRUE si alguno de los operandos es TRUE.
!	Operador NOT. Actúa sobre un único operando, dando como resultado el valor contrario que tuviese el operando.

Los operadores && y || funcionan en modo “cortocircuito”, es decir, si el primer operando determina el resultado de la operación, se ignora el segundo. Un ejemplo:

```
public class Ej06_OperadoresLogicos {
    public static void main(String[] args) {
        int i=10, k=20;
        if ( (i>0) || (++k>0) ) {
            i++;
        }
        System.out.println("i="+i);
        System.out.println("k="+k);
    }
}
```

Como puede verse ++k>0 no llega a evaluarse porque la primera condición ya es TRUE y el valor i se incrementa en una unidad.

2.5.8 Operadores a nivel de bits

Existe una forma en la que operadores lógicos no operan en “cortocircuito”: los operadores a nivel de bits, que además, de evaluar los dos operandos de la expresión, lo hacen a nivel de bits, permitiendo evaluar operandos boolean y enteros. Son estos:

Operador	Descripción
&	Operador lógico AND. Realiza la operación AND entre los operandos bit a bit
	Operador lógico OR. Realiza la operación OR entre los operandos bit a bit
^	Operador lógico OR exclusiva. Realiza la operación OR exclusiva entre los operandos bit a bit
~	Operador NOT. Invierte el estado de los bits del operando.

Veamos un ejemplo:

```
public class Ej07_OperadoresBits {
    public static void main(String[] args) {
        int n1=5, n2=7;
        boolean b=true;
        long l=5;

        b=b| (++l>0);
        System.out.println("b = "+b);
        System.out.println("l = "+l);
        System.out.println("n1 y n2 valen "+(n1&n2));
        // En este último caso al evaluarse TRUE pone el primer valor
    }
}
```

2.5.9 Operador instanceof

Se emplea sólo para referencias a objetos y se usa para saber si un objeto pertenece a un determinado tipo. La forma de usarlo es:

referencia_objeto instanceof clase

El resultado es TRUE si el objeto pertenece a la clase especificada o a una de sus subclases. Veamos un ejemplo:

```
public class Ej08_Instanceof {
    public static void main(String[] args) {
        String s = "Hola";
        if (s instanceof String)
            System.out.println("s es una cadena!");
    }
}
```

2.5.10 Operador condicional

Se trata de un operador ternario (consta de 3 operandos) cuya función es asignar un valor entre dos posibles a una variable, en función del cumplimiento o no de una condición. Su formato es:

tipo_variable = (condición)? valor_si_true:valor_si_false

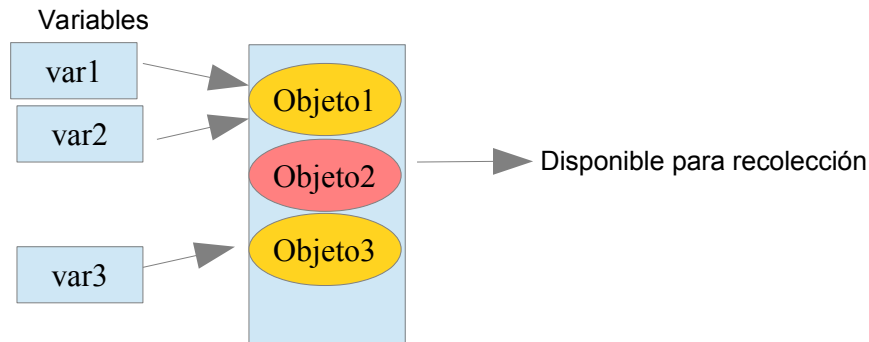
Si la condición es TRUE, se almacenará en la variable el resultado de la expresión valor_si_true; en caso contrario se almacenará el valor_si_false. Un ejemplo:

```
public class Ej09_OperadorCondicional {
    public static void main(String[] args) {
        int n=10;
        String s = (n%2==0)? "n es par": "n es impar";
        System.out.println(s);
    }
}
```

2.6 El Recolector de Basura

El recolector de basura (Garbage Collector) es una aplicación de JVM cuyo objetivo es liberar memoria de los objetos no referenciados. Cuando un objeto deja de estar referenciado, es marcado como basura; en ese momento la porción de memoria que ocupa puede ser liberada por el recolector.

El momento de liberación de la memoria depende de la máquina virtual y no es posible saber su momento exacto, aunque siempre será después de eliminar la última referencia a un objeto.



Si un objeto va a dejar de ser usado por un programa, conviene eliminar las referencias al mismo para que sea marcado como basura. Esto se puede conseguir asignando el valor null a las variables o variables que apuntan al objeto:

```
Integer i = new Integer(30);      // No es demasiado eficiente
```

```
...
```

```
i = null;                          // pierde la referencia
```

Si una variable se sale de ámbito, se pierde también la referencia y se marca como “basura”.

2.7 Instrucciones de Control

Java dispone de un juego de instrucciones para controlar el flujo de ejecución de un programa. Tenemos instrucciones alternativas y repetitivas. Vamos a ver cada una de ellas.

2.7.1 Condicional If

La instrucción if es del tipo alternativa simple. Permite comprobar una condición dentro de un programa. En el caso que la condición devuelva TRUE, ejecutará un determinado número de instrucciones. En caso contrario ejecutará otras instrucciones diferentes o saldrá del programa.

Su sintaxis es:

```
if (condición)      {
    sentencias...
}
else {
    sentencias...
}
```


A la hora de utilizar esta instrucción hay que tener en cuenta lo siguiente:

- La condición puede ser una expresión cuyo resultado debe ser boolean (TRUE o FALSE). En cualquier otro caso dará un error de comprobación. Un ejemplo de esto:
`int n = 10;`
`if (n) { ... } // Error de compilación`
- El bloque else es opcional. Si no existe dicho bloque, el programa seguirá su ejecución en la siguiente instrucción después del cierre de “}” del if.
- Cuando el bloque de sentencias del if o del else sólo tienen una instrucción, el uso de llaves delimitadoras es opcional. De todos modos, por claridad de código, es recomendable usarlo de todos modos.
- Las instrucciones if se pueden anidar. Veamos un ejemplo:

NOTA: En este ejemplo vamos a ver el uso de la clase scanner que veremos con mas detenimiento mas adelante. Su uso en este caso (como en otros) nos permitirá una lectura por teclado obteniendo un mayor abanico de posibilidades en los ejemplos.

Proceso de uso de la clase scanner:

1. En primer lugar importamos la clase al principio de la clase.
`import java.util.Scanner;`
2. Creamos un objeto de tipo Scanner que nos realizará las lecturas.
`Scanner sc = new Scanner(System.in);`
3. Empleamos un println para presentar un mensaje:
`System.out.print("Introduzca dato: ");`
4. Por último, realizamos la lectura y lo metemos en una variable:
`String nombre = sc.nextLine(); // Lectura para String o next()`
`int n = sc.nextInt(); // Lectura para enteros`
`double d = sc.nextDouble(); // Lectur para double`

```
import java.util.Scanner;
public class Ej10_Sentencialf {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int edad;

        System.out.println("Escriba su edad:");
        edad = sc.nextInt();
        if (edad<18)
            System.out.println("Es menor de edad");
        else
            if (edad<65)
                System.out.println("Está en edad de trabajar");
            else
                System.out.println("Está para jubilarse");
    }
}
```

2.7.2 Instrucción switch

Se trata de una instrucción de tipo alternativa múltiple. Permite ejecutar diferentes bloques de instrucciones en función del resultado de una expresión. Veamos su sintaxis:

```
switch (expresion)
{
    case valor1:
        sentencias;
        break;
    case valor2:
        sentencias;
        break;
    default:
        sentencias;
}
```

En el caso que el resultado de la expresión coincida con el valor representado en valor1, se ejecutarán las sentencias de ese bloque, si no coincide, se comparará con valor2 y así sucesivamente. Si el resultado no coincide con ninguno de los valores del case, se ejecutará las instrucciones indicadas en el default.

Sobre el uso de switch hay que tener en cuenta algunas consideraciones:

- Los únicos valores que puede evaluar el switch son números enteros int. Esto incluye además aquellos tipos que pueden ser convertidos a int: byte, char y short. Desde Java5 también permite evaluar String y ENUM.
- Un Switch puede contener un número ilimitado de case, pero no puede haber dos case que tengan el mismo valor.
- El bloque default es opcional y se ejecutará si el resultado de la expresión no coincide con ningún case.
- La sentencia break en cada bloque case es opcional. Se emplea para provocar la finalización del switch al terminar la ejecución del case. Si un case no incluye esta instrucción, al terminar la ejecución de su bloque de sentencias pasará a la ejecución del siguiente case independientemente de que el resultado de la expresión del switch coincida o no con el valor del nuevo case. Veamos un ejemplo:

```
public class Ej11_SentenciaSwitch1 {
    public static void main(String[] args) {
        int n=5;
        switch (n*2) {
            case 10:
                System.out.println("El resultado es 10");
            case 20:
                System.out.println("El número es demasiado alto!");
                break;
            default:
                System.out.println("El resultado no es correcto");
        }
    }
}
```

Desde Java7 está permitido el Switch con String y numerados (ENUM, Java5)

```
import java.util.Scanner;
public class Ej12_SwitchString {
    public static void main(String[] args) {
        String evaluación = "";
        Scanner teclado = new Scanner (System.in);
        System.out.println(" Especifique la evaluación:  ");
        System.out.println(" Opciones: Suspenso, Aprobado,  ");
        System.out.println(" Bien, Notable, Sobresaliente  ");
        System.out.println(" y Matricula  ");
        evaluación = teclado.nextLine();
        switch (evaluación) {
            case "Aprobado":System.out.println(" Nota >5 <6");    break;
            case "Bien":   System.out.println(" Nota >6 <7");    break;
            case "Notable": System.out.println(" Nota >7 <8,5");  break;
            case "Sobresaliente": System.out.println(" Nota >8,5 <10"); break;
            case "Matricula": System.out.println(" Nota =10");    break;
            default:System.out.println(" Nota <5 "); break;
        }
    }
}
```

2.7.3 Bucle for

La instrucción repetitiva for permite ejecutar un conjunto de instrucciones un número determinado de veces. Su sintaxis es la siguiente:

```
for (inicialización; condición; incremento) {
    sentencias...
}
```

La ejecución del for comienza con la instrucción de inicialización que suele inicializar una variable de control, incluyendo su declaración. Después comprueba la condición cuyo resultado debe ser boolean. En el caso de dar TRUE se ejecutan las sentencias delimitadas por las llaves { }. Luego se ejecutará la instrucción de incremento de la variable de control y volverá a comenzar el proceso. En el caso de que la condición devuelva FALSE, el programa saldrá del bucle y se ejecutará la siguiente línea después del cierre de llaves "}".

Sobre el uso del bucle for hay que tener en cuenta lo siguiente:

- **Las instrucciones de control del for son opcionales** (inicialización; condición; incremento). En cualquier caso, el delimitador de instrucciones ";" siempre debe estar presente. Por ejemplo si omitimos las instrucciones de comparación e incremento la cabecera for quedaría así:
for (int i=0;;)

NOTA: Debemos tener precaución con este tipo de situaciones. Podemos provocar la ejecución de un bucle infinito y ser obligados a detener a la fuerza el programa.

- Si se declara una variable en la instrucción de inicialización, esta será accesible únicamente dentro dentro del bloque de instrucciones del for.
- Las llaves sólo son obligatorias en el caso de tener mas de una instrucción en el bloque.

Veamos un ejemplo:

```
import java.util.Scanner;
public class Ej13_buclefor {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int factor=1, factorial=1;
        System.out.println("Escriba factor (entero 1 10):");
        factor = sc.nextInt();

        for (int i=1; i<=factor; i++)
            factorial *=i;
        System.out.println("El factorial de "+factor+" es: "+factorial);
    }
}
```

2.7.4 Bucle while

Permite ejecutar un bloque de instrucciones mientras se cumpla (TRUE) una condición dentro del programa. Permite dos tipos de formatos, cuyas sintaxis son:

while (condicion) {	do {
sentencias;	sentencias;
}	} while (condicion);

En ambos casos se ejecuta el bucle mientras el resultado de la condición sea TRUE. Sin embargo en el segundo caso (do...while) se ejecuta al menos una vez el bloque de instrucciones y luego se evalúa la condición. Solo en el caso que dicha condición devuelva TRUE se ejecutará el bloque por segunda vez y sucesivas veces. Como en otras instrucciones de control, el uso de las llaves es opcional, siendo sólo obligatorio si el bloque de instrucciones contiene mas de una instrucción.

Veamos en primer lugar un ejemplo de do...while.

En este caso pedimos al usuario el ingreso de valores para el calculo del promedio. Termina el ingreso si el usuario pone un 0. Con do...while, al menos realizamos una lectura:

```
import java.util.Scanner;
public class Ej14_bucleDoWhile {
    public static void main(String[] args) {
        Scanner teclado=new Scanner(System.in);
        int suma=0,cant=0,valor,promedio;
        do {
            System.out.print("Ingrese un valor (0 para finalizar):");
            valor=teclado.nextInt();
            if (valor!=0) {
                suma=suma+valor;
                cant++;
            }
        } while (valor!=0);
    }
}
```

```

    if (cant!=0) {
        promedio=suma/cant;
        System.out.print("El promedio de los valores ingresados es:");
        System.out.print(promedio);
    } else {
        System.out.print("No se ingresaron valores.");
    }
}
}

```

Ahora veamos un ejemplo con while.

```

import java.util.Scanner;
public class Ej15_BucleWhile {
    public static void main(String[] args) {
        Scanner teclado=new Scanner(System.in);
        int valor, contador=0;
        System.out.println ("Elija la tabla de Multiplicar ");
        valor=teclado.nextInt();
        System.out.println ("La Tabla de Multiplicar del "+valor);
        while (contador<10)
        {
            contador++;
            System.out.println(" "+valor+"x"+contador+"="+(valor*contador));
        }
    }
}

```

2.7.5 Salida forzada del bucle

Las instrucciones repetitivas for y while, cuentan con instrucciones que permiten interrumpir la ejecución del bloque antes de su finalización. Son **break** y **continue**.

BREAK

Ya hemos visto el uso de break en el condicional multiple switch. También lo podemos emplear para otras sentencias repetitivas. En estas, el uso de break provoca la salida forzada del bucle, continuando la ejecución de la aplicación en la instrucción siguiente del mismo bucle.

Ahora veamos un ejemplo con break:

```

import java.util.Scanner;
public class Ej16_BreakBucles {
    public static void main(String[] args) {
        Scanner teclado=new Scanner(System.in);
        int num;
        System.out.println ("Elija un valor del 1 al 10: ");
        num=teclado.nextInt();
        System.out.println ("\nValores Aleatorios: ");
    }
}

```

```

for (int i=1; i<=10; i++)
{
    int aleatorio = (int)Math.floor(Math.random()*10+1);
    System.out.println(aleatorio);
    if (aleatorio == num)
        break;
}
}
}

```

NOTA: En el ejemplo anterior hemos empleado dos métodos para conseguir un número aleatorio. Son estos:

Math.random: devuelve un double aleatorio entre 0 y 1.

Math.floor: devuelve el double mas cercano del valor pasado por parámetro.

Hacemos una conversión explícita a entero y obtenemos un valor entre 1 y 10.

CONTINUE

La instrucción continue provoca que el bucle detenga la iteración actual y pase, en el caso del for, a ejecutar la instrucción de incremento o en el caso del while a comprobar de nuevo la condición de entrada. Veamos un ejemplo (en el que también usaremos break) :

```

public class Ej17_ContinueBucles {
    public static void main(String[] args) {
        int suma=0;
        while (suma<100)
        {
            int aleatorio = (int)Math.floor(Math.random()*10+1);
            if (aleatorio%2!=0)    // Solo suma pares
                continue;

            System.out.println(aleatorio);
            // Si quitamos el if la suma puede exceder
            // algo de 100...
            if (suma+aleatorio>100) {
                break;
            }
            suma+=aleatorio;
        }
        System.out.println("La suma TOTAL es: "+suma);
    }
}

```

EJERCICIOS COMPLEMENTARIOS

Condicionales

1. Realizar la carga del lado de un cuadrado, mostrar por pantalla el perímetro del mismo (El perímetro de un cuadrado se calcula multiplicando el valor del lado por cuatro)
2. Escribir un programa en el cual se ingresen cuatro números, calcular e informar la suma de los dos primeros y el producto del tercero y el cuarto.
3. Realizar un programa que lea cuatro valores numéricos e informar su suma y promedio.
4. Se debe desarrollar un programa que pida el ingreso del precio de un artículo y la cantidad que lleva el cliente. Mostrar lo que debe abonar el comprador.
5. Realizar un programa que lea por teclado dos números, si el primero es mayor al segundo informar su suma y diferencia, en caso contrario informar el producto y la división del primero respecto al segundo.
6. Se ingresan tres notas de un alumno, si el promedio es mayor o igual a siete mostrar un mensaje "Promocionado".
7. Se ingresa por teclado un número positivo de uno o dos dígitos (1..99) mostrar un mensaje indicando si el número tiene uno o dos dígitos.
Tener en cuenta que condición debe cumplirse para tener dos dígitos, un número entero

Condicionales Anidados

1. Se cargan por teclado tres números distintos. Mostrar por pantalla el mayor de ellos.
2. Se ingresa por teclado un valor entero, mostrar una leyenda que indique si el número es positivo, nulo o negativo.
3. Confeccionar un programa que permita cargar un número entero positivo de hasta tres cifras y muestre un mensaje indicando si tiene 1, 2, o 3 cifras. Mostrar un mensaje de error si el número de cifras es mayor.
4. Un postulante a un empleo, realiza un test de capacitación, se obtuvo la siguiente información: cantidad total de preguntas que se le realizaron y la cantidad de preguntas que contestó correctamente. Se pide confeccionar un programa que ingrese los dos datos por teclado e informe el nivel del mismo según el porcentaje de respuestas correctas que ha obtenido, y sabiendo que:

Nivel máximo:	Porcentaje $\geq$ 90%.
Nivel medio:	Porcentaje $\geq$ 75% y $<$ 90%.
Nivel regular:	Porcentaje $\geq$ 50% y $<$ 75%.
Fuera de nivel:	Porcentaje $<$ 50%.

Condicionales con operadores lógicos

1. Realizar un programa que pida cargar una fecha cualquiera, luego verificar si dicha fecha corresponde a Navidad.
2. Se ingresan tres valores por teclado, si todos son iguales se imprime la suma del primero con el segundo y a este resultado se lo multiplica por el tercero.
3. Se ingresan por teclado tres números, si todos los valores ingresados son menores a 10, imprimir en pantalla la leyenda "Todos los números son menores a diez".
4. Se ingresan por teclado tres números, si al menos uno de los valores ingresados es menor a 10, imprimir en pantalla la leyenda "Alguno de los números es menor a diez".
5. Escribir un programa que pida ingresar la coordenada de un punto en el plano, es decir dos valores enteros x e y (distintos a cero).
Posteriormente imprimir en pantalla en que cuadrante se ubica dicho punto. (1º Cuadrante si $x > 0$ Y $y > 0$, 2º Cuadrante: $x < 0$ Y $y > 0$, etc.)
6. De un operario se conoce su sueldo y los años de antigüedad. Se pide confeccionar un programa que lea los datos de entrada e informe:
 - a) Si el sueldo es inferior a 500 y su antigüedad es igual o superior a 10 años, otorgarle un aumento del 20 %, mostrar el sueldo a pagar.
 - b) Si el sueldo es inferior a 500 pero su antigüedad es menor a 10 años, otorgarle un aumento de 5 %.
 - c) Si el sueldo es mayor o igual a 500 mostrar el sueldo en pantalla sin cambios.
7. Escribir un programa en el cual: dada una lista de tres valores numéricos distintos se calcule e informe su rango de variación (debe mostrar el mayor y el menor de ellos)

Bucles While

1. Escribir un programa que solicite ingresar 10 notas de alumnos y nos informe cuántos tienen notas mayores o iguales a 7 y cuántos menores.
2. Se ingresan un conjunto de n alturas de personas por teclado. Mostrar la altura promedio de las personas.
3. En una empresa trabajan n empleados cuyos sueldos oscilan entre \$100 y \$500, realizar un programa que lea los sueldos que cobra cada empleado e informe cuántos empleados cobran entre \$100 y \$300 y cuántos cobran más de \$300. Además el programa deberá informar el importe que gasta la empresa en sueldos al personal.
4. Realizar un programa que imprima 25 términos de la serie 11 - 22 - 33 - 44, etc. (No se ingresan valores por teclado)
5. Mostrar los múltiplos de 8 hasta el valor 500. Debe aparecer en pantalla 8 - 16 - 24, etc.
6. Realizar un programa que permita cargar dos listas de 15 valores cada una. Informar con un mensaje cual de las dos listas tiene un valor acumulado mayor (mensajes "Lista 1 mayor", "Lista 2 mayor", "Listas iguales")
Tener en cuenta que puede haber dos o más estructuras repetitivas en un algoritmo.
7. Desarrollar un programa que permita cargar n números enteros y luego nos informe cuántos valores fueron pares y cuántos impares.
Emplear el operador `??` en la condición de la estructura condicional:
`if (valor%2==0) //Si el if da verdadero luego es par.`

Bucles for

1. Confeccionar un programa que lea n pares de datos, cada par de datos corresponde a la medida de la base y la altura de un triángulo. El programa deberá informar:
 - a) De cada triángulo la medida de su base, su altura y su superficie.
 - b) La cantidad de triángulos cuya superficie es mayor a 12.
2. Desarrollar un programa que solicite la carga de 10 números e imprima la suma de los últimos 5 valores ingresados.
3. Desarrollar un programa que muestre la tabla de multiplicar del 5 (del 5 al 50)
4. Confeccionar un programa que permita ingresar un valor del 1 al 10 y nos muestre la tabla de multiplicar del mismo (los primeros 12 términos)
Ejemplo: Si ingreso 3 deberá aparecer en pantalla los valores 3, 6, 9, hasta el 36.
5. Realizar un programa que lea los lados de n triángulos, e informar:
 - a) De cada uno de ellos, qué tipo de triángulo es: equilátero (tres lados iguales), isósceles (dos lados iguales), o escaleno (ningún lado igual)
 - b) Cantidad de triángulos de cada tipo.
 - c) Tipo de triángulo que posee menor cantidad.
6. Escribir un programa que pida ingresar coordenadas (x,y) que representan puntos en el plano. Informar cuántos puntos se han ingresado en el primer, segundo, tercer y cuarto cuadrante. Al comenzar el programa se pide que se ingrese la cantidad de puntos a procesar.
7. Se realiza la carga de 10 valores enteros por teclado. Se desea conocer:
 - a) La cantidad de valores ingresados negativos.
 - b) La cantidad de valores ingresados positivos.
 - c) La cantidad de múltiplos de 15.
 - d) El valor acumulado de los números ingresados que son pares.
8. Se cuenta con la siguiente información:
Las edades de 50 estudiantes del turno mañana.
Las edades de 60 estudiantes del turno tarde.
Las edades de 110 estudiantes del turno noche.
Las edades de cada estudiante deben ingresarse por teclado.
 - a) Obtener el promedio de las edades de cada turno (tres promedios)
 - b) Imprimir dichos promedios (promedio de cada turno)
 - c) Mostrar por pantalla un mensaje que indique cual de los tres turnos tiene un promedio de edades mayor.

2.8 Arrays

Un array es un objeto que puede almacenar un conjunto de datos del mismo tipo. Cada uno de los elementos del array tiene asignado un índice numérico según su posición en el array, siendo el primero de ellos el valor 0.

2.8.1 Declaración

Un array debe declararse mediante la siguiente sintaxis:

```
tipo [ ] variable_array;
```

o bien

```
tipo variable_array [ ];
```

En el presente manual el autor ha optado por elegir el segundo formato, donde los corchetes van detrás del nombre de la variable. Veamos algunos ejemplos de arrays:

```
int numeros [ ];
```

```
String cadenas [ ];
```

```
char caracteres [ ];
```

Los arrays pueden declararse en los mismos sitios que las variables estándar: como atributos de clase o como variables locales dentro de un método. Como ocurre con cualquier otro tipo de variable objeto se inicializa implícitamente con el valor null.

2.8.2 Dimensionado de un array

Para asignar un tamaño (de elementos) de un array usaremos la siguiente sintaxis:

```
variable_array = new tipo [tamaño]
```

También podemos asignar el tamaño en la misma línea de declaración de la variable. Veamos algunos ejemplos válidos:

```
array_enteros = new int [10];
```

```
array_caracteres = new char [20];
```

```
String nombres [ ] = new String [5];
```

Al dimensionar un array todos sus elementos son inicializados explícitamente a sus valores por defecto. Existe una forma de declarar, dimensionar e inicializar un array en la misma sentencia. Su sintaxis será: `tipo variable_array = { elemento1, elemento2, ..., elementoN }`

Por ejemplo:

```
int numeros [ ] = { 5, 10, 15, 20 };
```

2.8.3 Acceso a los elementos de un array

Para acceder a los elementos de un array usaremos la siguiente sintaxis:

`variable_array [indice_elemento];`

donde `indice_elemento` expresa la posición del elemento al que se quiere tener acceso y cuyo rango será del 0 hasta `tamaño_array - 1`.

Todos los objetos array contienen un método público llamado `length` que permite conocer el tamaño al que se ha dimensionado el array y que permite recorrer los elementos que componen el array tanto para visualizarlos como para introducirlos.

Por ejemplo, en este caso usaremos el método `length` para introducir números enteros en un array y luego visualizarlos por pantalla.

```
public class Ej18_MeterVerArray {  
    public static void main(String[] args) {  
        int numeros [] = new int [10];  
        // Metemos los elementos en el array  
        for (int i=0; i<numeros.length; i++)  
            numeros[i] = i*2;  
        // Recorremos el array para ver los elementos  
        for (int i=0; i<numeros.length; i++)  
            System.out.println(numeros[i]);  
    }  
}
```

Veamos otro ejemplo: Creamos una aplicación que calcula la suma de todos los números de un array de enteros y además nos muestra el mayor y el menor de sus elementos:

```
public class Ej19_ArrayMayorMenor {  
    public static void main(String[] args) {  
        int mayor, menor, suma=0;  
        int numeros [] = {10, 3, 8, 4, 9, 1};  
        // Por ahora metemos en menor/mayor el primer elemento  
        menor = numeros[0];  
        mayor = numeros[0];  
        for (int i=0; i<numeros.length; i++)  
        {  
            if (numeros[i]>mayor)  
                mayor = numeros[i];  
            if (numeros[i]<menor)  
                menor = numeros[i];  
            suma +=numeros[i];  
        }  
        // Presento los datos  
        System.out.println("La suma es: "+suma);  
        System.out.println("El mayor es: "+mayor);  
        System.out.println("El menor es: "+menor);  
    }  
}
```

2.8.4 Paso de un array como argumento

Además de los tipos básicos, los arrays también pueden usarse como argumentos de llamada de los métodos. Para declarar que un método reciba como parámetro un array, se emplea la sintaxis:

```
tipo metodo(tipo variable_array [ ]) {  
    // Sentencias  
}
```

El siguiente método recibe como parámetro un array de enteros:

```
void metodoEjemplo (int numeros [ ]) {  
    // Sentencias  
    // Podemos recorrer el array (con length)  
}
```

En cuanto a la llamada al método, se usará la siguiente sintaxis:

```
objeto.metodo (variable_array);
```

A modo de ejemplo, vamos a ver la aplicación del apartado anterior usando métodos:

```
public class Ej20_ArraysMetodos {  
    public static void main(String[] args) {  
        int mayor, menor, suma=0;  
        int numeros [] = {10, 3, 8, 4, 9, 1};  
  
        // Por ahora metemos en menor/mayor el primer elemento  
        menor = numeros[0];  
        mayor = numeros[0];  
  
        suma = sumar(numeros);  
        mayor = calculaMayor(numeros);  
        menor = calculaMenor(numeros);  
  
        // Presento los datos  
        System.out.println("La suma es: "+suma);  
        System.out.println("El mayor es: "+mayor);  
        System.out.println("El menor es: "+menor);  
    }  
  
    // Ojo al uso de static. Estamos ante métodos de clase  
    static int sumar (int nums[]) {  
        int sum=0;  
        for (int i=0; i<nums.length; i++)  
            sum += nums[i];  
        return sum;  
    }  
}
```

```

static int calculaMayor (int nums[]) {
    int numMayor = nums[0];
    for (int i=0; i<nums.length; i++) {
        if (nums[i]>numMayor)
            numMayor = nums[i];
    }
    return numMayor;
}

static int calculaMenor (int nums[]) {
    int numMenor = nums[0];
    for (int i=0; i<nums.length; i++) {
        if (nums[i]<numMenor)
            numMenor = nums[i];
    }
    return numMenor;
}
}

```

2.8.5 Array como tipo de devolución de un método

Un array también puede ser devuelto por un método tras la ejecución del mismo. Su sintaxis es:

```

tipo [ ] metodo (parámetros) {
    // Sentencias
    return variable_array;
}

```

Veamos un ejemplo; pedimos un número y nos devuelve un array con los 5 siguientes valores:

```

import java.util.Scanner;
public class Ej21_ReturnArray {
    static int [] devuelveNumeros (int n) {
        int numeros [] = new int [5];
        for (int i=0; i<numeros.length; i++)
            numeros[i] = (n+i)+1;
        return numeros;
    }
    public static void main(String[] args) {
        int valor;
        Scanner teclado=new Scanner(System.in);
        System.out.print("Ingrese un valor para mostrar el array: ");
        valor=teclado.nextInt();
        int nums[] = new int[5];
        nums = devuelveNumeros(valor);

        for (int i=0; i<nums.length; i++)
            System.out.println(nums[i]);
    }
}

```

2.8.6 Recorrido de arrays con for each

Desde la versión 5 de Java se incluye una variante del for para recorrer los arrays: for-each que mejora el recorrido de los arrays y colecciones de objetos (que ya veremos), eliminando la necesidad de usar una variable índice para acceder a los distintos elementos. Aunque identificamos esta instrucción como for-each, se sigue usando la instrucción for con esta sintaxis:

```
for (tipo variable: var_array) {  
    // instrucciones  
}
```

En este caso, tipo variable representa la declaración de la variable auxiliar del mismo tipo del array, variable que irá tomando cada uno de los valores del propio array, siendo var_array la variable que apunta al array. Lo mejor es verlo con un ejemplo:

Tenemos un array de 5 enteros:

```
int numeros [ ] = { 10, 4, 7, 8, 1};
```

Si queremos mostrar por pantalla cada elemento usando el for “tradicional” será así:

```
for (int i=0; i<numeros.length; i++) {  
    System.out.println (numeros [ i ]);  
}
```

Usando la nueva instrucción for-each, lo haríamos del siguiente modo:

```
for (int n: numeros) {  
    System.out.println (n);  
}
```

Como puede observarse, sin acceder de forma explícita a las posiciones del array, cada una de estas es copiada automáticamente a la variable auxiliar n (de tipo entero) al principio de cada iteración, simplificando mucho el código.

Veamos el ejemplo del apartado paso de un array como argumento usando el for-each. Además, hemos incluido la lectura por teclado para introducir los valores del propio array:

```
import java.util.Scanner;  
public class Ej22_ForEach {  
    public static void main(String[] args) {  
        int valor, contador=0;  
        int mayor, menor, suma=0;  
        int numeros [ ] = new int [5];  
        Scanner teclado=new Scanner(System.in);  
        for (int i=0; i<numeros.length; i++) {  
            System.out.print("Meta valor" + (contador+1) + " para array: ");  
            numeros[i]=teclado.nextInt();  
            contador++;  
        }  
  
        menor = numeros[0];  
        mayor = numeros[0];  
  
        suma = sumar(numeros);  
        mayor = calculaMayor(numeros);  
        menor = calculaMenor(numeros);  
    }  
}
```

```

    // Presento los datos
    System.out.println("La suma es: "+suma);
    System.out.println("El mayor es: "+mayor);
    System.out.println("El menor es: "+menor);
}

// Ojo al uso de static. Estamos ante métodos de clase
static int sumar (int nums[]) {
    int sum=0;
    for (int n:nums)
        sum += n;
    return sum;
}

static int calculaMayor (int nums[]) {
    int numMayor = nums[0];
    for (int n:nums) {
        if (n>numMayor)
            numMayor = n;
    }
    return numMayor;
}

static int calculaMenor (int nums[]) {
    int numMenor = nums[0];
    for (int n:nums) {
        if (n<numMenor)
            numMenor = n;
    }
    return numMenor;
}
}

```

2.8.7 Arrays Multidimensionales

Hasta ahora hemos estudiado arrays unidimensionales. Pero Java permite también arrays de dos dimensiones cuya sintaxis será:

tipo variable_array [] [];

De todos modos también está permitido el siguiente formato:

tipo [] [] variable_array;

Por ejemplo, tenemos lo siguiente:

```
int enteros [ ] [ ];
```

A la hora de asignarle un tamaño, procedemos de un modo similar a los arrays unidimensionales:

```
enteros = new int [4] [3];
```

Para acceder a una posición concreta del array procedemos a la siguiente forma:

```
enteros [1] [2] = 10;
```

Recorrido de un Array Multidimensional

Para recorrer un array multidimensional usaremos un for tradicional, empleando tantas variables de control como dimensiones tenga el array. Veámoslo con un ejemplo completo:

```
import java.util.Scanner;

public class Ej23_ArrayMultidimensional {
    public static void main(String[] args) {

        Scanner teclado=new Scanner(System.in);
        int arrayMulti [][] = new int [3][3];
        // Para introducir valores usamos un doble for
        // Con el 2º for, al incluir el índice del primero
        // puedo introducir datos en arrays irregulares
        for (int i=0; i<arrayMulti.length;i++) {
            for (int j=0; j<arrayMulti[i].length;j++) {
                System.out.println ("Escriba Valor: ");
                arrayMulti[i][j]=teclado.nextInt();
            }
        }
        for (int numeros[]: arrayMulti) {
            System.out.println ("");
            for (int entero: numeros)
                System.out.print (""+entero+"\t");
        }
    }
}
```

Arrays Multidimensionales Irregulares

Podemos definir un array multidimensional donde los elementos de la segunda dimensión y sucesivas sea variable, indicando el tamaño únicamente en la primera dimensión:

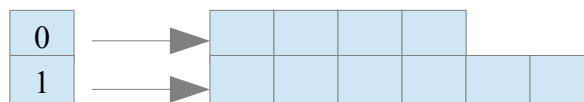
```
int enteros [ ] [ ] = new int [2] [ ];
```

Un array de este tipo equivale a un array de arrays unidimensionales. De este modo, los elementos de la primera dimensión almacenarán su propio array. Siguiendo el ejemplo anterior:

```
enteros [0] = new int [4];
```

```
enteros [1] = new int [6];
```

Gráficamente sería:



Veamos un ejemplo donde hemos empleado métodos estáticos para introducir e imprimir este tipo de arrays multidimensionales irregulares:

```
import java.util.Scanner;

public class Ej24_ArrayIrregularMulti {
    static Scanner teclado=new Scanner(System.in);
```



```

static int [][] crearArray () {
    int filas = 0;
    System.out.println (" ARRAY MULTIDIMENSIONAL IRREGULAR  ");
    System.out.println ("Escriba Filas Array: ");
    filas=teclado.nextInt();

    // Defino el array sin poner el segundo corchete
    int arrayMulti [][] = new int [filas][];

    // Ahora definimos el tamaño para cada fila
    for (int i=0; i<arrayMulti.length;i++) {
        System.out.println ("Escriba Tamaño array" +(i+1)+": ");
        int tam = teclado.nextInt();
        arrayMulti [i] = new int [tam];
    }
    return arrayMulti;
}

static int [][] meteValoresArray (int miArray[][]) {
    for (int i=0; i<miArray.length;i++) {
        for (int j=0; j<miArray[i].length;j++) {
            System.out.println ("Escriba Valor ["+i+"]"+"["+j+"]": ");
            miArray[i][j]=teclado.nextInt();
        }
    }
    return miArray;
}

static void imprimeArray (int miArray[][]) {
    for (int numeros[]: miArray) {
        System.out.println ("");
        for (int entero: numeros)
            System.out.print (""+entero+"\t");
    }
}

public static void main(String[] args) {

    int arrayMulti [][] = crearArray (); // Creo el array
    arrayMulti = meteValoresArray(arrayMulti);
    imprimeArray(arrayMulti);
}
}

```

2.9 Uso de la clase Vector

La clase Vector es similar a los Arrays previamente vistos. De todos modos tiene diferencias importantes, que se resumen en:

- a) Se pueden añadir elementos dinámicamente (sin usar índices)
- b) No hace falta dimensionar
- c) Permite crear listados de objetos

Tiene 3 constructores:

- a) El constructor sin argumentos (que crea un vector de capacidad 10)
- b) El constructor con 1 argumento (la dimensión). Si se rebasa, se incrementa en ese mismo valor.

Por ejemplo: `Vector miVector = new Vector (10);`

Si añadimos un elemento mas de 10, la dimensión pasará a ser 20.

- c) El constructor con 2 argumentos (dimensión, incremento).

En este caso, si se rebasa el tamaño, crece según lo puesto en el segundo argumento.

Veamos un ejemplo completo:

```
import java.util.Scanner;
import java.util.Vector;
/** Uso y manejo de Vectores
 * Los vectores son similares a los arrays
 * @author Iván Rodríguez
 */
public class Vectores {
    static Scanner sc = new Scanner(System.in);
    public static Vector leeVector (Vector v, String cadena) {
        v.addElement(cadena);
        return v;
    }

    public static void imprimeVector (Vector v) {
        // El método size devuelve el nº de elementos del vector
        // El método capacity devuelve el tamaño del vector
        System.out.println("Elementos del Vector");
        for (int i=0; i<v.size(); i++)
            System.out.println(v.elementAt(i));
    }

    public static void main(String[] args) {
        Vector miVector = new Vector ();
        String cadena = "";
        while (!cadena.equals("S")) {
            System.out.println("Escriba nombre: ");
            cadena = sc.next();
            miVector = leeVector(miVector, cadena);
            imprimeVector(miVector);
        }
    }
}
```

3 CLASES DE USO GENERAL

En este tema vamos a ver las clases mas importantes de uso general en Java. Es importante reseñar que la edición J2SE tiene mas de 8000 clases (incluyendo interfaces) con sus correspondientes métodos, que nos permitirán desarrollar cualquier tipo de aplicaciones.

Para el estudio de las clases de uso general (las mas empleadas) las dividiremos en 3 grupos:

- Clases básicas
- Clases de envoltorio
- Clases de Entrada / Salida

Posteriormente veremos otro tipo de clases mas específicos, como los de creación de aplicaciones gráficas (Swing) y acceso a base de datos (JDBC).

3.1 Los paquetes

Los paquetes permiten clasificar clases de una manera estructurada y jerárquica. Un paquete es una carpeta donde se almacenan archivos .class con los bytescodes de la clase. Por otro lado, un paquete puede estar compuesto por subpaquetes.

En el caso de J2SE existe un paquete principal llamado java del cual derivan una serie de subpaquetes donde se encuentran las distintas clases que componen la edición. Por ejemplo, tenemos el siguiente paquete para el manejo de cadenas de caracteres:

java.lang.String

Donde java es el paquete principal, lang el subpaquete y String la clase.

3.1.1 Ventajas del uso de paquetes

El uso de paquetes en Java permite tener las siguientes ventajas:

- **Organizar las clases de manera estructurada**
- **Evitar conflictos de nombres.**

Una clase se identifica por un nombre cualificado de clase que se compone del nombre de la clase precedido por los subpaquetes donde se encuentra hasta llegar al paquete principal separados por un punto. De este modo veremos clases con el mismo nombre que al estar en subpaquetes diferentes tendrán funcionalidades diferentes.

3.1.2 Importar clases y paquetes de clases

Para no usar todo el nombre completo de la clase, incluyendo subpaquetes y paquete principal (como hemos visto con String), podemos usar importaciones. De este modo ganamos en claridad y rapidez en el código. Para importar una clase la sintaxis será la siguiente:

```
import nombre_cualificado_clase;
```

Veamos la diferencia entre importar clases y no hacerlo:

```
public class Vectores {  
    public void metodo ( ) {  
        java.util.Vector v;  
    }  
}
```

Y ahora con importación:

```
import java.util.Vector;
public class Vectores {
    public void metodo ( ) {
        Vector v;
    }
}
```

Si se van a emplear varias clases de un mismo paquete, podemos importarlo al completo usando la siguiente sintaxis:

```
import nombre_paquete.*;
```

Por ejemplo:

```
import java.util.*;
```

NOTA: Debemos evitar en la medida de lo posible importar un paquete entero empleando el *.

3.1.3 Paquetes de uso general

Las clases que vamos a ver están incluidas dentro de los siguientes paquetes:

- **java.lang.**
Clases fundamentales de uso para cualquier programa Java. Puesto que son de uso común, el compilador importa el paquete completo de forma implícita, de modo que no tendremos que usar el import.
- **java.io**
Contiene clases para entrada y salida de datos en Java, independientemente del dispositivo que empleemos para hacerlo.
- **java.util**
Aquí encontraremos clases para varias utilidades, como las colecciones de objetos, manipulación de fechas y otros usos.

3.1.4 El API J2SE

Debemos manejar adecuadamente la documentación oficial de las clases Java estándar, o API. Esta especificación incluye las clases de J2SE así como toda la información relacionada: nombre, modificadores, clases de las que hereda, interfaces que implementa, paquetes donde se encuentra, atributos públicos, métodos y constructores.

Es esencial, dado el gran número de clases existentes, consultar dicha API para saber toda la información referente a la clase que vamos a emplear.

En la siguiente página encontraremos el API referente a J2SE 8.0:

<https://docs.oracle.com/javase/8/docs/api/>



La forma de buscar cualquier clase es la siguiente:

1. En el marco superior izquierda buscamos el paquete donde se encuentra la clase a buscar. En el ejemplo buscaremos la clase String que está en java.lang
2. En el marco inferior izquierda pulsaremos la clase: String.
3. En el marco central nos aparece información de la clase: campos (FIELD), Constructores (CONSTR), Métodos (METHOD) e información relacionada.

3.2 Gestión de cadenas: String

En Java, las cadenas de caracteres no son tipos básicos, sino objetos de la clase java.lang.String. Esta clase incluye una amplia variedad de métodos para manipular y tratar cadenas.

3.2.1 Creación de String

Para crear una cadena podemos seguir el procedimiento habitual para crear objetos. Su sintaxis:
String cadena = new String ("Cadena de caracteres");

De todos modos Java permite crear estos objetos de forma similar a un tipo básico. Por ejemplo:
String cadena = "Cadena de caracteres";

Una vez creado el objeto, podemos emplear los métodos definidos en la clase String:
cadena.length();

Además, al igual que los char con los caractere, podemos usar el operador "+" para concatenar cadenas. Veamos un ejemplo:

```
String cadena1 = "Hola";  
String cadena2 = " Curso de Java";  
String cadena1 = cadena1 + cadena2;
```

3.2.2 Inmutabilidad de los objetos String

En Java, los objetos String son inmutables. Esto quiere decir que una vez se ha creado el objeto, este no puede ser cambiado. En el ejemplo anterior, aunque parece que hemos concatenado una cadena2 a cadena1, modificándola, en realidad se ha creado un objeto nuevo, cambiando la referencia a este.

3.2.3 Métodos de la clase String

Los métodos mas importantes para la gestión de cadenas String son los siguientes:

- **int length ()**: Devuelve el número de caracteres de la cadena.
- **boolean equals (String otraCadena)**: Compara una cadena con otra, con distinción de mayúsculas y minúsculas. Debe usar este método en vez del operador ==. Ejemplo:

```
String cadena1 = "Vaca";
String cadena2 = "Baca";
if (cadena1.equals (cadena2)
    System.out.println("Las cadenas son iguales");
else
    System.out.println("Las cadenas son distintas");
```

- **boolean equalsIgnoreCase**: Compara cadenas sin distinguir mayúsculas y minúsculas.
- **Char charAt (int pos)**: Devuelve el carácter que ocupa la posición indicada (siendo la inicial el 0).
Veamos un ejemplo: Ver el número de veces que sale un carácter.

NOTA: En el siguiente ejemplo se usa la clase scanner tanto para lectura de una cadena (de una palabra solo no permite espacios) como del carácter. Se propone modificar el código para aumentar las posibilidades de lectura.

```
import java.util.Scanner;
public class Ej01_UsoCharAt {
    public static void main(String[] args) {
        Scanner teclado=new Scanner(System.in);
        System.out.println("Escriba una palabra: ");
        String cadena = "";
        char caracter;
        int contador=0;
        cadena =teclado.next(); // Lectura de String

        // Para leer un caracter usamos el next() de String
        // y sacamos el primer carácter con charAt(0).
        System.out.println("Escriba Carácter: ");
        caracter = teclado.next().charAt(0);

        for (int i=0; i<cadena.length(); i++) {
            if (cadena.charAt(i)== caracter)
                contador++;
        }

        System.out.println("Veces Carácter "+caracter+": "+contador);
    }
}
```

- **String substring (int inicio, int final):** Devuelve un fragmento de la cadena. Dicho fragmento serán los caracteres que van desde la posición indicada en inicio hasta final-1. Ejemplo:

```
String cadena = "Curso de Java";
System.out.println(cadena.substring(9,13);           // Saldrá Java
```
- **int indexOf (string cadena):** Indica la posición de la cadena del parametro dentro de la cadena principal. Si no aparece, devuelve -1. Existe otra versión de este método en la que se puede indicar la posición del carácter a partir del cual comenzar la búsqueda:

```
public boolean existeCad (String cadPrincipal, String cadBuscada) {
    if (cadPrincipal.indexOf (cadBuscada) != 1)
        return true;
    else
        return false
}
```
- **String replace (char caracterBuscado, char caracterNuevo):** devuelve la cadena resultante de sustituir todas las veces que aparece el carácter buscado por el caracterNuevo. Este método es muy útil si queremos sustituir por ejemplo un separador de decimal por otro:

```
String cadResultado = " ";
String cadDecimal = "3,1416"
cadenaResultado = cadDecimal.replace ("," , ".");           //=3.1416;
```
- **static String valueOf (tipo_basico dato):** Método estático que devuelve como cadena el valor de tipo básico pasado como parámetro.
- **String toUpperCase ():** Pone en mayúsculas la cadena.
- **String toLowerCase ():** Poner en minúsculas la cadena.
- **String [] split (String separador):** Devuelve un array de String resultado de fragmentar la cadena en función del separador indicado como parámetro. Además del carácter separador se pueden usar expresiones regulares. Veamos un ejemplo:

```
import java.util.Scanner;
public class Ej02_Usosplit {

    public static void main(String[] args) {
        String cadena = "";
        String separador = "";
        String arrayCadenas[];

        Scanner teclado = new Scanner (System.in);
        System.out.println("Escriba cadena a separar");
        cadena = teclado.nextLine();
        System.out.println("Escriba el separador");
        separador = teclado.nextLine();

        arrayCadenas = cadena.split(separador);
        for (String palabra: arrayCadenas)
            System.out.println(""+palabra);
    }
}
```

3.3 La clase Math

Esta clase proporciona los métodos mas usuales para realizar operaciones matemáticas. Todos los métodos de la clase Math son estáticos, por lo que no necesitamos ninguna instancia de la clase para su uso (de hecho no podemos crearnos objetos de dicha clase).

3.3.1 Constantes públicas

La clase Math tiene dos atributos públicos estáticos con las constantes matemáticas mas usadas:

- **static double E**: Contiene el valor double mas cercano al número e.
- **static double PI**: Contiene el valor double mas cercano al número PI.

3.3.2 Métodos

Los métodos mas importantes para esta clase son los siguientes:

- **numero max (numero n1, numero n2)**: Devuelve el mayor de ambos números. Hay 4 versiones, para int, long, float y double.
- **numero min (numero n1, numero n2)**: Igual que el anterior, pero devuelve el menor.
- **double ceil (double num)**: Devuelve el entero mayor mas cercano al número indicado por parámetro. Veamos un ejemplo:
`double d = Math.ceil (3.4); // Devuelve 4.0`
- **double floor (double num)**: Devuelve el entero menor mas cercano al número indicado por parámetro. Veamos un ejemplo:
`double d = Math.floor (3.4); // Devuelve 3.0`
- **double round (double num)**: Devuelve el entero mas cercano al número indicado por parámetro. Veamos un ejemplo:
`double d = Math.round (3.4); // Devuelve 3.0`
- **double pow (double base, double exponente)**: Devuelve el resultado de elevar base por el exponente. Veamos un ejemplo:
`double potencia = Math.pow (3, 4); // Devuelve 81.0`
- **double random ()**: Devuelve un double aleatorio mayor o igual que 0.0 y menor que 1.0. Veamos un ejemplo para generar 10 números aleatorios:

```
public class Ej03_NumAleatorios {  
    public static void main(String[] args) {  
        int aleatorios [] = new int [10];  
  
        for (int i=0; i<aleatorios.length; i++)  
            aleatorios[i] = (int)(Math.random()*100+1);  
        for (int valor:aleatorios)  
            System.out.println(valor);  
    }  
}
```


3.3.3 Importaciones estáticas

La importación estática es una característica añadida en la versión 5 de Java. Consiste en la posibilidad de importar todos los miembros estáticos de la clase, de modo que podamos usarlos sin necesidad de emplear el nombre de la clase delante del método o atributo.

Esta característica puede ser muy útil por ejemplo con la clase Math, puesto que sus métodos son estáticos. Por ejemplo, para usar el método pow pondríamos:
pow (3,4) en vez Math.pow (3,4)

Para importar los elementos estáticos de una clase debemos usar la siguiente sintaxis:

```
import static paquete.clase.*
```

Para el caso de la clase Math deberíamos hacer lo siguiente:

```
import static java.lang.Math.*;
```

De este modo, el ejemplo del apartado anterior quedaría:

```
import java.util.Scanner;
// Con esto importo los miembros estáticos de la clase Math
import static java.lang.Math.*;
```

```
public class Ej04_ImportEstaticos {

    public static void main(String[] args) {
        int aleatorios [];
        int numAleatorios=0;

        Scanner teclado = new Scanner (System.in);
        System.out.println("Escriba número de aleatorios: ");
        numAleatorios = teclado.nextInt();
        aleatorios = new int [numAleatorios];

        // Dejo que la máquina meta los aleatorios
        for (int i=0; i<numAleatorios; i++)
            aleatorios[i]= (int)(Math.random()*100+1);
        for (int numero:aleatorios)
            System.out.println(""+numero);
    }
}
```

3.4 Uso de fechas

Para el manejo de fechas, Java dispone de dos clases: Date y Calendar.

3.4.1 Clase Date

Un objeto definido a partir de la clase Date representa la fecha y hora con precisión de un milisegundo. No ofrece soporte para internacionalización, por lo que a partir del JDK 1.1 se incorporó Calendar que permite mas posibilidades. Muchos métodos de Date están obsoletos (deprecated) aunque su constructor sigue siendo útil. Su sintaxis es:
Date fecha = new Date;

Mediante el método toString() podemos conseguir la fecha y hora actuales.
El resultado será algo similar a: Tue Nov 11 23:25:42 CET 2014

```
import java.util.*;
public class Ej05_UsoDate {
    public static void main(String[] args) {
        Date fecha = new Date();
        System.out.println (fecha.toString());
    }
}
```

El objeto guarda información de la fecha y hora de los milisegundos transcurridos desde el 1 de Enero de 1970 hasta el momento de la creación del objeto. Dicho número podemos obtenerlo mediante el método getTime().

3.4.2 Clase Calendar

Esta clase cubre las lagunas que tiene la clase Date y apareció en el SDK 1.4

Creación de un objeto Calendar

La clase calendar es abstracta, por lo que debemos usar getInstance en vez new para crear un objeto de una subclase de Calendar (gracias al polimorfismo, la clase usada es transparente al programador). Su sintaxis es:

```
Calendar cal = Calendar.getInstance ();
```

Usando el método get() podemos obtener cada uno de los campos que componen la fecha. La clase Calendar define una serie de constantes para los valores de los campos Fecha y Hora.

NOTA: Los meses comienzan a contar a partir del 0.El rango de enteros será de 0-11.

```
import java.util.*;
public class Ej06_UsoCalendar {
    public static void main(String[] args) {
        Calendar cal = Calendar.getInstance();
        String fecha = "";
        fecha += cal.get (Calendar.DAY_OF_MONTH)+"/";
        fecha += cal.get (Calendar.MONTH)+1+"/";
        fecha += cal.get (Calendar.YEAR);
        System.out.println("Hoy es: "+fecha);
    }
}
```

Métodos de la clase Calendar

A parte del método get(), Calendar tiene estos métodos:

- **void set (int año, int mes, int día):** Modifica la fecha actual para el objeto Calendar con los valores pasados como parámetros. Existe un método igual en el que se añaden los parámetros para agregar la hora. Hay que recordar que los meses comienzan por 0.
- **void setTime (Date fecha):** Establece la fecha y hora a partir de un objeto date.
- **Date getTime ():** Devuelve la fecha y hora como un objeto date.
- **void add (int campo, int cantidad):** Realiza una modificación relativa en alguno de los campos de fecha y hora, añadiendo o quitando tiempo al campo especificado. Por ejemplo para restar 15min a la hora actual:
`cal.add (Calendar.MINUTE, -15);`

Por ejemplo, si queremos saber la fecha y hora de hace 38 años, 5 meses, 16 días, 10 horas y 40 minutos, haremos lo siguiente:

```
import java.util.*;
public class Ej07_UsoCalendarAdd {
    public static void main(String[] args) {
        Calendar fecha = Calendar.getInstance();
        fecha.add(Calendar.YEAR, 38);
        fecha.add(Calendar.MONTH, 5);
        fecha.add(Calendar.DAY_OF_MONTH, 16);
        fecha.add(Calendar.HOUR_OF_DAY, 10);
        fecha.add(Calendar.MINUTE, 40);

        String fechaPasada = "";
        fechaPasada += fecha.get (Calendar.DAY_OF_MONTH)+"/";
        fechaPasada += fecha.get (Calendar.MONTH)+1+"/";
        fechaPasada += fecha.get (Calendar.YEAR)+" ";
        fechaPasada += fecha.get (Calendar.HOUR_OF_DAY)+":";
        fechaPasada += fecha.get (Calendar.MINUTE);
        System.out.println("Hoy es: "+fechaPasada);
    }
}
```

El resultado será el 26 de Mayo de 1976 a las 13:10.

- **void roll (int campo, int cantidad):** funciona igual que el add con la diferencia que el resto de campos no se ve afectado. Así, si tengo el 31 de Diciembre de 2014 y añado 1 día saldrá 01/12/2014, es decir, el único campo afectado es el propio día.

3.5 Clases de Envoltorio

Java proporciona una clase que representa como un objeto a los distintos tipos de datos básicos. Estas clases son conocidas como Clases de Envoltorio y se usan para:

1. Encapsular un dato básico (primitivo)
2. Conversión de cadena a un tipo básico

Existen ocho clases de envoltorio, tantas como tipos básicos de Java (entre paréntesis): Byte (byte), Short (short), Integer (int), Long (long), Float (float), Double (double), Character (char) y Boolean (boolean)

3.5.1 Encapsulamiento de un tipo básico

Todas las clases de envoltorio permiten crear un objeto a partir de uno de los 8 tipos básicos.

El procedimiento será el siguiente:

tipo variable1 = valor;

Clase_envoltorio variable2 = new Clase_envoltorio (variable1);

Por ejemplo:

```
int n = 10;
```

```
// n lo envolvemos en el objeto num
```

```
Integer num = new Integer (n);
```

A excepción de character, el resto de clases de envoltorio pueden crear objetos desde la representación de la cadena como dato. El procedimiento es el siguiente:

String cadena = valor;

Clase_envoltorio variable = new Clase_envoltorio (cadena);

Por ejemplo:

```
String cadena = "4.65f";
```

```
Float flotante = new Float (cadena);
```

Como cualquier otro objeto, una instancia creada desde una clase envoltorio tiene algunos métodos entre los cuales se encuentra uno para recuperar el valor del dato encapsulado. Dicho método tiene el formato xxxValue() donde xxx será uno de los ocho tipos de datos básicos. Siguiendo con los dos ejemplos anteriores, para imprimir los valores tendríamos:

```
System.out.println("Valor de num: "+num.intValue());
```

```
System.out.println("Valor de flotante: "+flotante.floatValue());
```

3.5.2 Conversión de cadena a tipo numérico

Las clases numéricas contienen un método estático llamado parseXxx (String) que permiten convertir la representación numérica en forma de cadena en el correspondiente tipo numérico y donde xxx será dicho tipo. El procedimiento será el siguiente:

String cadena = valor;

tipo variable = Clase_Envoltorio.parse<tipo> (cadena).

En el siguiente ejemplo vemos todo lo visto en este apartado y en el anterior.

```
public class Ej08_ClasesEnvoltorio {  
    public static void main(String[] args) {  
        int n = 10;
```

```

// n lo envolvemos en el objeto num
Integer num = new Integer (n);
System.out.println("Valor de num: "+num.intValue());

// Podemos convertir una cadena en float creando un objeto
// a partir de la representación como cadena del dato
String cadena = "4.65f";
Float flotante = new Float (cadena);
System.out.println("Valor de flotante: "+flotante.floatValue());

// Podemos convertir una cadena a tipo numérico
// El proceso usa clases de envoltorio
String cadena1 = "50", cadena2="3.1416";
int n1 = Integer.parseInt(cadena1);
double d1 = Double.parseDouble(cadena2);
System.out.println("Valor de n1: "+n1);
System.out.println("Valor de d1: "+d1);
}
}

```

3.5.3 Autoboxing.

Introducido en Java5 consiste en la encapsulación automática de un tipo básico en objeto de envoltorio mediante el uso de un operador de asignación. Siguiendo con el ejemplo anterior, si queremos encapsular un entero:

```
int n = 10;
```

```
Integer num = new Integer (n);
```

Mediante el autoboxing, lo anterior se realiza de una manera mas sencilla:

```
int n = 10;
```

```
Integer num = n;
```

Además, para recuperar el dato no tenemos que usar el **intValue**, sino que se haría así:

```
int valorN = num;
```

A esta última operación se le conoce como autounboxing y también apareció en Java5.

Con lo anterior, se mejora la legibilidad del código y facilitando el trabajo al programador.

```
// Autoboxing / AutoUnboxing
```

```
n1 = 20;
```

```
Integer num1 = n1;
```

```
System.out.println("Valor de num: "+num1);
```

La utilidad del encapsulamiento de datos primitivos (usando clases de envoltorio) es por si queremos, por ejemplo, hacer un array dinámico de datos primitivos.

3.6 Clases de Entrada y Salida

El paquete java.io incluye una serie de clases para la gestión de entrada y salida de datos en un programa, sea cual sea el dispositivo empleado. Resumiendo, tenemos:

Salida -> `PrintStream`

Entrada -> `InputStream`

De este, a su vez deriva el `BufferedReader` y el `InputStreamReader`.

A partir del SDK5, Java incluye una nueva clase llamada `Scanner` que permite realizar las entradas de un modo menos engorroso y mucho mas sencillo.

3.6.1 Salida de datos

Para realizar el envío de datos al exterior usaremos la clase `PrintStream`.

El proceso para realizar el envío consta de los siguientes pasos:

1. Obtención del objeto `PrintStream`. La forma de crear objetos de esta clase dependerá del dispositivo de salida. La clase `System` proporciona un atributo estático llamado `out` que contiene una referencia al objeto anterior asociado a la salida estándar, que es la consola.
2. Envío de datos al `Stream`. La clase `PrintStream` dispone de dos métodos estándar para la salida de datos, `print` y `println`, ambos con salida de cadenas. El segundo incluye un salto de línea. Así, para mandar un mensaje usaremos algo así:

```
System.out.println ("Texto de Ejemplo");
```

Tanto `print` como `println` se encuentran "sobrecargados" (algo que ya veremos en el tema siguiente), es decir existen versiones iguales de los mismos pero cambiando los datos que se envían al exterior. La sintaxis general será:

```
objeto_printstream.println (dato)            o objeto_printstream.print (dato)
```

Donde `dato` puede ser `String`, `int`, `float`, `double`, `char` o `boolean`.

3.6.2 Salida con formato (printf)

A partir de la versión 5 del SDK, Java permite aplicar formato a la cadena de caracteres que procede del `print` anterior. En este caso usaremos el método `printf` y su sintaxis es:

```
printf (String formato, object... datos)
```

El primer argumento, `formato`, consiste en una cadena de caracteres con las opciones de formato aplicados a los datos que se van a mostrar.

El segundo argumento (y sucesivos) es la información a la que se va a aplicar el formato del primer argumento. El número de estos es ilimitado y variable. Veamos un ejemplo:

```
import static java.lang.Math.*;
```

```
public class Ej09_SalidaPrintf {  
    public static void main(String[] args) {  
        double piCuadrado = Math.PI * Math.PI;  
        System.out.printf ("El Cuadrado de %1$.4f es %2$.2f",  
            Math.PI,piCuadrado);  
    }  
}
```

Sintaxis de la cadena de formato

La cadena consta de un texto fijo (en el ejemplo “El cuadrado de “) mas una serie de especificadores de formato que rigen la forma en la que los datos son presentados (en el ejemplo %1\$.4f es %2\$.2f). Su sintaxis será:

% [posicion_argumento \$] [indicador]
[mínimo] [.num_decimales] conversion

El significado de cada elemento anterior (que debe ponerse seguido) es el siguiente:

- [posicion_argumento \$] Es la posición del argumento sobre el que se aplicará formato. El primer argumento ocupa la posición 1. Su uso es opcional.
- [indicador] Consiste en un conjunto de caracteres que determina el formato de salida. Su uso es opcional. Los mas importantes son los siguientes:
 - '-' El resultado aparece alineado a la izquierda
 - '+' El resultado debe incluir el signo (solo para argumentos numéricos)
- [mínimo] N úmero mínimo de caracteres que serán presentados. Su uso es opcional.
- [.num_decimales] Número de decimales presentados; solo aplicable a float y double. Su uso es opcional. Este valor debe ir precedido de un punto.
- Conversion. Consiste en un carácter que indica cómo deber ser formateado el argumento según esta tabla.

Carácter	Función
's', 'S'	Si es null se formatea como null. Si no será argumento.toString ()
'c', 'C'	El resultado será un carácter unicode
'd'	El resultado será un entero con notación decimal
'x', 'X'	El resultado será un entero con notación hexadecimal
'e', 'E'	El resultado será un decimal en notación científica
'f'	El resultado será un número decimal

Para las fechas, la sintaxis será:

% [posicion_argumento \$] [indicador] [mínimo] conversion

Cada elemento es igual a lo visto previamente, excepto conversión que es una secuencia de dos caracteres: el primero es 't' o 'T', siendo el segundo carácter el que indica el formato según estas tablas (de fechas u horas).

Carácter	Función
'H'	Hora del día con dos dígitos del 00 al 23
'I'	Hora del día con dos dígitos del 01 al 12
'M'	Minutos de la hora actual, con dos dígitos del 00 al 59
'S'	Segundos de la hora actual, con dos dígitos del 00 al 59

Carácter	Función
'B'	Nombre completo del mes
'b'	Nombre abreviado del mes
'A'	Nombre completo del día de la semana
'a'	Nombre abreviado del día de la semana
'y'	Últimos dos dígitos del año
'e'	Día del mes con un rango del 1 al 31

Veamos un ejemplo usando varios elementos de las tablas anteriores:

```
import java.util.Calendar;
public class Ej10_FechaHoraPrintf {
    public static void main(String[] args) {
        Calendar fechaHora = Calendar.getInstance();
        System.out.println ("La hora y fechas actuales es: ");
        System.out.printf("%1$tH:%1$tM:%1$tS %1$td de %1$tB" +
            " de 20%1$ty \n", fechaHora);
    }
}
```

3.6.3 La clase Scanner

La clase Scanner apareció en el SDK5 de Java para superar las limitaciones y el engorro de usar el `InputStreamReader` y `BufferedReader`. Permite la lectura tanto de dispositivos de entrada (como un teclado) como de un fichero externo, tanto en forma de caracteres como de cualquier tipo básico.

Creación de un objeto Scanner

Para tener acceso a los datos de entrada, debemos crearnos un objeto Scanner asociado al `InputStream` del dispositivo que puede ser, por ejemplo, un teclado. Previamente debemos importar la clase Scanner del paquete util de Java:

```
import java.util.Scanner;
```

Así tendremos:

```
Scanner teclado = new Scanner (System.in);
```

El objeto Scanner leerá la cadena de caracteres hasta la pulsación de **ENTER**, dividiendo las cadenas por un separador (que suele ser el espacio en blanco) en **tokens** (equivalente a palabras). Mediante los métodos de Scanner podemos recuperar secuencialmente estos tokens e incluso convertirlos implícitamente en cualquier tipo de datos. Es mas, podemos recoger un grupo de tokens (hasta la pulsación de ENTER) y hacerlos actuar como un único token (con métodos como `nextLine` que vemos ahora).

Métodos de la clase Scanner

Entre los métodos mas importantes de Scanner destacamos los siguientes:

- **String next ()**: Devuelve el siguiente token.
- **String nextLine ()**: Devuelve el siguiente grupo de tokens hasta pulsar ENTER.
- **boolean hasNext ()**: Indica si existe un nuevo token para leer.
- **xxx nextXxx ()**: Devuelve el siguiente token como un tipo básico, siendo Xxx ese tipo básico que puede ser entero (int – nextInt) o decimal (float – nextFloat).
- **boolean hasNextXxx ()**: Indica si existe el siguiente token del tipo especificado.
- **void useDelimiter (String delimitador)**: establece un nuevo separador para los tokens.

Veamos un ejemplo sencillo en la que pedimos un String y un entero:

```
import java.util.Scanner;
public class Ej11_UsoScanner {
    public static void main(String[] args) {
        String nombre;
        int dni;
        Scanner teclado = new Scanner (System.in);

        // Realizamos las lecturas
        System.out.println("Escriba nombre completo: ");
        nombre = teclado.nextLine();
        System.out.println("Escriba DNI (sin letra): ");
        dni = teclado.nextInt();

        // Presentamos los datos
        System.out.println(" ");
        System.out.println(" Nombre: "+nombre);
        System.out.println(" DNI: "+dni);
    }
}
```

3.7 Colecciones

Es un objeto que almacena un conjunto de referencias a otros objetos. Podríamos decir, por tanto que una colección es array de objetos, pero a diferencia de los Arrays que ya hemos visto, es dinámico: no tiene tamaño fijo y permite añadir y eliminar objetos en tiempo de ejecución. Esta funcionalidad le otorga una gran importancia en la elaboración de aplicaciones. De hecho ya veremos como mas adelante lo emplearemos con bases de datos y aplicaciones gráficas.

El paquete **java.util** surte de un amplio conjunto de clases para la creación y tratamiento de colecciones. Todas ellas contienen métodos básicos para el tratamiento de colecciones:

- Insertar objetos a la colección
- Eliminar objetos de la colección
- Obtener un objeto de la colección
- Localizar un objeto de la colección
- Iterar (recorrer) los elementos de una colección

3.7.1 ArrayList

La clase ArrayList permite crear una colección basado en índices. Cada objeto de la misma tiene asociado un número (índice) según la posición que ocupa en la colección, similar a los arrays básicos, donde el primer elemento tiene el índice 0.

3.7.2 Creación de un ArrayList

Para crear un ArrayList tenemos la siguiente sintaxis:

```
ArrayList variable_objeto = new ArrayList ();
```

Donde variable_objeto contiene la referencia al objeto ArrayList creado. Un par de ejemplos:

```
ArrayList colección = new ArrayList ();
```

```
List <ObjetoEquipo> equipos = new ArrayList <ObjetoEquipo> ();
```

Una vez que hemos creado el objeto ArrayList, podemos usar sus métodos asociados para realizar las operaciones habituales de una colección.

3.7.3 Métodos del ArrayList

Los métodos mas importantes de la clase ArrayList son:

- **boolean add (Object o):** Añade un objeto a la colección y lo pone al final de la misma.
- **void add (int índice, Object o):** Añade el objeto en la posición especificada por índice desplazando el resto de objetos de la colección hacia adelante.
- **Object get (int indice):** Devuelve el objeto que ocupa la posición indicada.
- **Object remove(int indice):** Elimina de la colección el objeto con el índice especificado.
- **Void clear ():** Elimina todos los elementos de la colección
- **int indexOf (Object o):** Localiza el objeto indicado y devuelve su posición
- **int size ():** Devuelve el número de elementos almacenados en la colección

En el **ANEXO IV** encontraremos un ejemplo completo de ArrayList con un Objeto definido.

4 PROGRAMACIÓN ORIENTADA A OBJETOS

Java está completamente orientado a objetos, por lo que podemos aplicar sobre sus aplicaciones todas las características y beneficios de este modelo de programación, como el encapsulamiento, herencia y polimorfismo. En este capítulo veremos conceptos fundamentales aplicables tanto para J2SE como para J2EE.

4.1 Empaquetado de clases

La organización de clases en paquetes facilita mucho su uso en otras clases. Para crear nuestra propia estructura en paquetes debemos proceder con lo siguiente:

- Creación de directorios: Debemos crearnos directorios dentro del directorio de trabajo.
- Empaquetado de las clases: una vez creados los directorios, procedemos al empaquetado de las clases usando la sentencia `package` con la siguiente sintaxis:
`package nombre_paquete;`
Esta sentencia debe ir en primer lugar del `.java` (antes incluso de los `import`).
- Compilación: Situamos los `.java` en su directorio correspondiente y procedemos a su compilación mediante la siguiente sintaxis de consola:
`raiz/paquete > javac archivo_clase.java`
o `raiz > javac paquete/archivo_clase.java`

NOTA: La `/` es para sistemas UNIX. Para windows es `\`

Todo el proceso anterior es realizado por los principales IDEs, como **NetBeans** y **Eclipse**.

Veamos un ejemplo completo (nos creamos una carpeta llamada **tema04**):

NOTA IMPORTANTE: Hasta ahora, durante el curso, nos hemos ido creando un proyecto diferente para cada ejemplo. Aquí el proceso cambia. Nos creamos el proyecto **Ej01_UsoPackage**

```
package ej01_usopackage;  
public class Ej01_UsoPackage {  
    public String getMensaje () {           // No incluimos ningún main  
        return "Hola Curso!";  
    }  
}
```

En el mismo proyecto nos creamos un paquete nuevo (`ej02_otraclasepackage`) y dentro una clase llamada **Ej02_OtraClasePackage**, cuyo contenido será el siguiente (y que ejecutaremos):

```
package ej02_otraclasepackage;  
import ej01_usopackage.Ej01_UsoPackage;  
public class Ej02_OtraClasePackage {  
    public static void main(String[] args) {  
        String cadena = "";  
        Ej01_UsoPackage objeto = new Ej01_UsoPackage();  
        cadena = objeto.getMensaje();  
        System.out.println(cadena);  
    }  
}
```

4.2 Modificadores de acceso

Los modificadores de acceso se usan para definir la visibilidad de los miembros de una clase (atributos y métodos). En Java tenemos 4, ordenados de menor a mayor visibilidad, los siguientes:

- **private:** cuando un atributo o método es definido como private, su uso queda restringido a la propia clase. No puede ser aplicado a la propia clase.
- **(ninguno):** al elemento se le aplica el acceso por defecto, esto es, visibilidad desde el mismo paquete donde se encuentre la clase.
- **protected:** el elemento es visible por clases del mismo paquete y por otras clases que deriven de su clase (herencia) aunque se encuentren en otro paquete.
- **public:** el elemento es visible desde cualquier clase.

4.3 Encapsulación

Una clase está compuesta por métodos que determinan el comportamiento de los objetos de la clase (también llamados funciones) y por atributos que representan las características de esos objetos (también llamados campos). Los **métodos suelen ser públicos**, mientras que los **atributos tienen a ser privados**. Aquí es donde aparece la **encapsulación**: los campos quedan restringidos al máximo con el modificador private, mientras que accedemos a los mismos mediante métodos público, que son llamados Métodos de acceso.

Esto se hace sobre todo por dos razones:

- **Protección de datos** “sensibles” (aumento de seguridad).
- **Facilidad y flexibilidad** en el mantenimiento de aplicaciones.

4.3.1 Protección de datos

Debemos evitar la corrupción de datos precisamente encapsulando los atributos, forzando el uso de los mismos mediante una forma controlada. Veamos un ejemplo:

NOTA: A partir de este tema, todas las clases relacionadas las pondremos en el mismo proyecto, aunque seguiremos numerando incrementalmente para separar cada ejemplo.

Nos creamos el proyecto **Ej03_Rectangulo** y dentro la clase **Ej03_Rectangulo**.

```
package ej03_rectangulo;
public class Ej03_Rectangulo {
    public int alto, ancho;
    public void setAlto (int alto) {
        this.alto = alto;
    }
    public void imprimeDatos () {
        System.out.println (" RECTANGULO  ");
        System.out.println (" Alto = "+this.alto);
        System.out.println (" Ancho = "+this.ancho);
    }
}
```

Ahora, en el mismo paquete del mismo proyecto, nos creamos Ej04_ObjRectangulo:

```
package ej03_rectangulo;

public class Ej04_ObjRectangulo {
    public static void main(String[] args) {
        Ej03_Rectangulo nuevoRectangulo = new Ej03_Rectangulo();
        nuevoRectangulo.setAlto( 5);
        nuevoRectangulo.imprimeDatos();
    }
}
```

Como puede observarse, el alto sale con una cantidad negativa, un disparate.

Para evitarlo, debemos modificar la clase principal **Ej03_Rectangulo** quedando así:

```
package ej03_rectangulo;

public class Ej03_Rectangulo {
    private int alto, ancho;
    public void setAlto (int alto) {
        if (alto>0)
            this.alto = alto;
    }
    public int getAlto () {
        return this.alto;
    }

    public void setAncho (int ancho) {
        if (ancho>0)
            this.ancho = ancho;
    }
    public int getAncho () {
        return this.ancho;
    }

    public void imprimeDatos () {
        System.out.println (" RECTANGULO   ");
        System.out.println (" Alto = "+this.alto);
        System.out.println (" Ancho = "+this.ancho);
    }
}
```

Hemos añadido métodos públicos set y get para, respectivamente, asignar valores y recuperar valores de los atributos privados de la clase. Al añadir una capa adicional de seguridad, no tendremos que usar necesariamente en la clase que crea el objeto el método imprimeDatos.

Por otro lado, hemos empleado la palabra reservada `this` para usar los métodos y atributos del propio objeto dentro de la propia clase (aunque es redundante, porque no es obligatorio). Tan solo es necesario (y fundamental) usar dicha palabra para invocar a un miembro del propio objeto cuando nos encontremos una variable local y un atributo con el mismo nombre, en cuyo caso tendremos que usar la sintaxis:
`this.variable_atributo`

Por último, comentar que la clase donde nos creamos el objeto **Ej04_Rectangulo** podemos modificarlo para añadir la impresión de datos, empleando los métodos get correspondientes. Por tanto, la clase **Ej04_ObjRectangulo** quedará del siguiente modo:

```
package ej03_rectangulo;

public class Ej04_ObjRectangulo {
    public static void main(String[] args) {
        Ej03_Rectangulo nuevoRectangulo = new Ej03_Rectangulo();
        nuevoRectangulo.setAlto( 5);

        // nuevoRectangulo.imprimeDatos();
        // Ahora podemos imprimir los datos desde esta clase...
        System.out.println (" RECTÁNGULO  ");
        System.out.println (" Alto = "+nuevoRectangulo.getAlto());
        System.out.println (" Ancho = "+nuevoRectangulo.getAncho());
    }
}
```

4.3.2 *Facilidad en el mantenimiento de la clase*

Si hemos creada la clase decidimos que queremos cambiar los criterios de asignación de valores en los atributos, podemos hacerlo libremente, sin que los programadores tengan que cambiar ni una línea de código de sus aplicaciones que emplean dicha clase para crear sus objetos. Por ejemplo en el código anterior podemos acotar los valores del alto a un rango concreto:

```
...
public class Ej03_Rectangulo {
    private int alto, ancho;
    public void setAlto (int alto) {
        if (alto<=0)
            this.alto = 1;
        else
            if (alto >10)
                this.alto = 10;
            else
                this.alto = alto;
    }
}
...
```

4.3.3 JavaBeans

En muchos tipos de aplicaciones puede llegar a ser útil la creación de clases cuya única finalidad sea el encapsulamiento de datos asociados a una entidad (como información de una película, libro, etc) dentro de la misma y que conviene tener agrupados para su mejor mantenimiento.

Estas clases se les denomina Clases de Encapsulación o JavaBeans (aunque este término puede ser usado también en otros contextos).

NOTA: En NetBeans podemos sacar automáticamente los Set/Get con botón derecho sobre algún atributo > **Insertar Código > Setter/Getter..** Lo mismo para los constructores.

Su estructura: Campos, Constructores y métodos set/get:

```
package ej05_librojavabean;

public class Ej05_LibroJavaBean {
    // Atributos privados
    private String titulo;
    private long isbn;
    private float precio;

    // El constructor
    public Ej05_LibroJavaBean (String titulo, long isbn, float precio) {
        this.titulo = titulo;
        this.isbn = isbn;
        this.precio = precio;
    }

    // Métodos set/get
    public void setTitulo (String nuevoTitulo) {
        titulo = nuevoTitulo;    }

    public String getTitulo () {
        return titulo;    }

    public void setIsbn (long nuevolsbn) {
        isbn = nuevolsbn;
    }
    public long getIsbn () {
        return isbn;    }

    public void setPrecio (float nuevoPrecio) {
        precio = nuevoPrecio;
    }
    public float getPrecio () {
        return precio;    }
}
```

En el mismo paquete nos creamos la clase **Ej06_ObjetoLibro**:

```
package ej05_librojavabean;
import java.util.Scanner;
public class Ej06_ObjetoLibro {
    public static void main(String[] args) {
        // Los atributos del objeto
        String tituloLibro = "";
        long isbnLibro;
        float precioLibro;

        // Hacemos la lectura
        Scanner teclado = new Scanner (System.in);
        System.out.println("Escriba titulo Libro: ");
        tituloLibro = teclado.nextLine();
        System.out.println("Escriba ISBN Libro: ");
        isbnLibro = teclado.nextLong();
        System.out.println("Escriba precio Libro: ");
        precioLibro = teclado.nextFloat();

        // Ahora creamos el objeto y asignamos los valores tomados
        Ej05_LibroJavaBean libro =
            new Ej05_LibroJavaBean (tituloLibro,isbnLibro, precioLibro);

        // Aunque no hemos usados los set, empleamos los get
        // para imprimir los datos del libro...
        System.out.println(" DATOS LIBRO   ");
        System.out.println(" Titulo: "+ libro.getTitulo());
        System.out.println(" ISBN: "+ libro.getIsbn());
        System.out.println(" Precio: "+ libro.getPrecio());
    }
}
```

Para probarlo podemos poner lo siguiente:

Título del libro: Fundación (Isaac Asimov)

ISBN del libro: 9788497599245

Precio del libro: 8,75

NOTA: Debemos poner el precio con coma (","), Si lo hacemos con punto dará un error.

4.4 Sobrecarga de métodos

Una de las grandes ventajas de la POO es la sobrecarga de métodos, que permite tener en la misma clase varios métodos con el mismo nombre, diferenciándose por sus parámetros (número y tipos). El tipo de devolución puede o no ser el mismo, pero la **diferenciación por argumentos es fundamental**.

Veamos un ejemplo completo de uso de sobrecarga de métodos:

```
package ej07_sobrecargametodos;
```

```
import java.util.Scanner;
```

```
public class Ej07_SobrecargaMetodos {
```

```
    // Definimos unos elementos estáticos
```

```
    static Scanner teclado = new Scanner (System.in);
```

```
    // Definimos los métodos
```

```
    static void pideDatos () {
```

```
        System.out.println("Escriba Figura:");
```

```
        String figura = teclado.next();
```

```
        switch (figura) {
```

```
            case "Cuadrado":
```

```
                System.out.println("Escriba Lado:");
```

```
                int lado = teclado.nextInt();
```

```
                calculaArea(lado);
```

```
                break;
```

```
            case "Rectángulo":
```

```
                System.out.println("Escriba ancho:");
```

```
                int ancho = teclado.nextInt();
```

```
                System.out.println("Escriba alto:");
```

```
                int alto = teclado.nextInt();
```

```
                calculaArea(ancho,alto);
```

```
                break;
```

```
            case "Trapezio":
```

```
                System.out.println("Escriba Base Menor:");
```

```
                int baseMenor = teclado.nextInt();
```

```
                System.out.println("Escriba Base Mayor:");
```

```
                int baseMayor = teclado.nextInt();
```

```
                System.out.println("Escriba Altura:");
```

```
                int altura = teclado.nextInt();
```

```
                calculaArea(baseMenor,baseMayor,altura);
```

```
                break;
```

```
        }
```

```
    }
```

```

static void calculaArea (int ancho) {
    int area = ancho*ancho;
    System.out.println("Área Figura: "+area);
}

static void calculaArea (int nuevoAncho, int nuevoAlto) {
    int area = nuevoAncho * nuevoAlto;
    System.out.println("Área Figura: "+area);
}

static void calculaArea (int baseMayor, int baseMenor, int altura) {
    int area = (int)((baseMayor+baseMenor)*altura)/2;
    System.out.println("Área Figura: "+area);
}

public static void main(String[] args) {
    pideDatos ();
}
}

```

4.5 Constructores

En el apartado de JavaBeans ya vimos un ejemplo de constructor donde pasábamos 3 parámetros (y además de distinto tipo) para construir un nuevo objeto libro.

```

public Ej05_LibroJavaBean (String titulo, long isbn, float precio) {
    // ...
}

```

Ahora vamos a profundizar en el concepto de constructor.

4.5.1 Definición y uso

Un constructor es un método especial que es ejecutado en el momento de crear un objeto de la clase mediante el operador new. Podemos usar un constructor para añadir tareas que tengamos que hacer en el momento de crear el objeto, las mas usadas, iniciar los atributos del propio objeto. En el ejemplo anterior podríamos haber conseguido lo mismo usando lo siguiente:

```

libro.setTitulo = ...
libro.setIsbn = ...
libro.setPrecio = ...

```

Pero podemos omitir algún atributo por descuido y dejarlo sin inicializar.

Para crear un constructor debemos seguir las siguientes reglas:

- El nombre del constructor debe ser el mismo de la clase.
- No puede tener ningún tipo de devolución, ni siquiera void.
- Los constructores se pueden sobrecargar, es decir, podemos tener mas de uno (con el nombre de la clase) pero cambiando los parámetros que se les pasa.
- Toda clase debe tener, al menos, un constructor. Si no lo tiene, tendrá el constructor por defecto (que veremos en el apartado siguiente).

4.5.2 Constructores por defecto

Si no se especifica un constructor, el compilador se encarga de añadir uno cuya sintaxis es:

```
public nombre_clase ( ) {  
}
```

Es decir, sin parámetros y sin código; pero necesario para que la clase pueda compilarse.

Eso si, en caso de especificar un constructor propio, el compilador NO añade ningún por defecto.

Por ello debemos tener cuidado con esta circunstancia. Si hemos puesto un constructor como el del ejemplo del apartado anterior:

```
public Ej05_LibroJavaBean (String titulo, long isbn, float precio) {  
    // ...  
}
```

Si creamos un objeto de este modo:

```
Ej05_LibroJavaBean libro = new Ej05_LibroJavaBean ();
```

Nos dará un error.

4.6 Clases Compuestas

Las clases compuestas son esenciales para construir objetos complejos, porque permite crearlos a partir de otras clases. Están muy relacionadas con la Herencia, pero OJO, el concepto NO ES EL MISMO.

4.6.1 Concepto

Una clase compuesta es aquella que tiene uno o mas atributos que son objetos creados a partir de instancias de otras clases. Evidentemente, podemos tener varios niveles de creación de clases compuestas.

Lo mejor es verlo a partir de un ejemplo:

Clase Precio

```
package auxiliares;  
import java.util.Scanner;  
/** @author Iván Rodríguez  
 */  
public class Precio {  
    Scanner sc = new Scanner(System.in);  
  
    // Atributos  
    private float valor;  
    private String divisa;  
  
    // Constructor por defecto  
    public Precio () {}  
  
    // Constructor definido  
    public Precio(float valor, String divisa) {  
        this.valor = valor;  
        this.divisa = divisa;  
    }  
}
```

```

public Precio leePrecio () {
    System.out.println("\n  Escriba Precio");
    System.out.println("\n Valor:");
    float nuevoValor = sc.nextFloat();
    System.out.println("\n Divisa:");
    String nuevaDivisa = sc.next();    // Mejor que nextLine()
    Precio nuevoPrecio = new Precio (nuevoValor, nuevaDivisa);

    return nuevoPrecio;
}

@Override
public String toString () {
    String cadena = "\n Precio: ";
    cadena += this.valor + this.divisa;
    return cadena;
}
}

```

Clase Dimensiones

```

package Auxiliares;
import java.util.Scanner;

public class Dimensiones {
    Scanner sc = new Scanner(System.in);

    // Atributos
    private int alto;
    private int ancho;
    private int fondo;
    private final String medida = "cms";

    // Constructor por defecto
    public Dimensiones () {}

    public Dimensiones(int alto, int ancho, int fondo) {
        this.alto = alto;
        this.ancho = ancho;
        this.fondo = fondo;
    }

    @Override
    public String toString () {
        String cadena = "\n Medidas: ";
        cadena += this.alto+ this.medida;
        cadena += " x " + this.ancho+ this.medida;
        cadena += " x " + this.fondo+ this.medida;
        cadena += " (Al x An x Fo)";
        return cadena;
    }
}

```

```

public Dimensiones leeDimension () {
    System.out.println("\n  Escriba Dimensiones");
    System.out.println("\n Alto:");
    int nuevoAlto = sc.nextInt();
    System.out.println("\n Ancho:");
    int nuevoAncho = sc.nextInt();
    System.out.println("\n Fondo:");
    int nuevoFondo = sc.nextInt();
    Dimensiones miDimension =
        new Dimensiones(nuevoAlto, nuevoAncho, nuevoFondo);
    return miDimension;
}
}

```

Clase Chasis

```

package elementos;
import auxiliares.Dimensiones;
import java.util.Scanner;

public class Chasis {
    //private auxiliares.Dimensiones dimchasis;

    Scanner sc = new Scanner(System.in);
    // Atributos
    private String tipo;
    private Dimensiones dimchasis = new Dimensiones();

    // Constructor
    public Chasis () {}

    public Chasis(String tipo) {
        this.tipo = tipo;
        dimchasis = dimchasis.leeDimension();
    }

    @Override
    public String toString () {
        String cadena = "\n CHASIS ";
        cadena += "\n Tipo: "+this.tipo;
        cadena += dimchasis.toString();
        return cadena;
    }

    public Chasis leeChasis () {
        System.out.println("\n  Escriba Datos Chasis ");
        System.out.println("\n Tipo:");
        String nuevoTipo = sc.nextLine();
        Chasis miChasis = new Chasis (nuevoTipo);
        return miChasis;
    }
}

```

Clase Motor

```
package elementos;
import java.util.Scanner;

public class Motor {
    Scanner sc = new Scanner(System.in);

    private int caballos;
    private int cilindrada;

    // Constructor
    public Motor () {}

    public Motor(int caballos, int cilindrada) {
        this.caballos = caballos;
        this.cilindrada = cilindrada;
    }

    @Override
    public String toString () {
        String cadena = "\n Motor: ";
        cadena += this.caballos + "cv y ";
        cadena += this.cilindrada + "cc";
        return cadena;
    }

    public Motor leeMotor () {
        System.out.println("\n  Escriba Datos Motor  ");
        System.out.println("\n Caballos:");
        int nuevosCaballos = sc.nextInt();
        System.out.println("\n Cilindrada:");
        int nuevaCilindrada = sc.nextInt();
        Motor miMotor =
            new Motor (nuevosCaballos, nuevaCilindrada);
        return miMotor;
    }
}
```

Y por último, la clase Camion que emplea varias de las clases anteriores:

Clase Camion

```
package componentes;

import auxiliares.Precio;
import elementos.Chasis;
import elementos.Motor;
import java.util.Scanner;

public class Camion {
    Scanner sc = new Scanner(System.in);

    private String modelo;
    private int numRuedasCamion;
```

```

private Motor motorCamion = new Motor();
private Chasis chasisCamion = new Chasis();
private Precio precioCamion = new Precio();

// El Constructor con 9 parámetros
/*
public Camion(String modelo,
    int caballos, int cilindrada,
    String tipo, int alto, int ancho, int fondo,
    float valor, String divisa) {
    this.modelo = modelo;
    this.motorCamion = new Motor (caballos, cilindrada);
    this.chasisCamion = new Chasis (tipo, alto, ancho, fondo);
    this.precioCamion = new Precio (valor, divisa);
} */

// Constructor por defecto
public Camion () {}

// Constructor con 2 parámetros
public Camion(String modelo, int numRuedasCamion) {
    this.modelo = modelo;
    this.numRuedasCamion = numRuedasCamion;
    motorCamion = motorCamion.leeMotor();
    chasisCamion = chasisCamion.leeChasis();
    precioCamion = precioCamion.leePrecio();
}

@Override
public String toString () {
    String cadena = "\n Datos Camión ";
    cadena += "\n Modelo: " + this.modelo;
    cadena += "\n Nº Ruedas: " + this.numRuedasCamion;
    cadena += motorCamion.toString();
    cadena += chasisCamion.toString();
    cadena += precioCamion.toString();
    return cadena;
}

public Camion leeCamion() {
    System.out.println("\n  Escriba Datos Camión  ");
    System.out.println("\n Modelo:");
    String nuevoModelo = sc.next();
    System.out.println("\n Nº Ruedas:");
    int numRuedas = sc.nextInt();
    Camion miCamion = new Camion (nuevoModelo, numRuedas);
    return miCamion;
}
}

```

4.7 Herencia

Es uno de los conceptos mas importantes y potentes del paradigma de la POO.

4.7.1 Concepto

El concepto de Herencia es la capacidad de crear clases que hereden de manera automática los miembros heredables (atributos y métodos) de otras clases que ya existen, que serán llamadas clases Padre o SuperClases, pudiendo añadir, además, atributos y métodos propios.

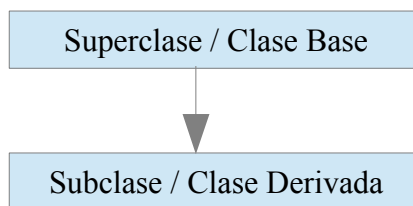
4.7.2 Ventajas

La herencia implica grandes ventajas para el programador de aplicaciones Java. Son:

- Reutilización de código: Se pueden emplear los métodos de la clase de la que se hereda, evitando volver a escribir esos códigos.
- Mantenimiento de aplicaciones existentes: si queremos ampliar la funcionalidad de una clase, no tenemos que reescribir la clase original, sino que hacemos una herencia y en la clase hija o derivada metemos sus funcionalidades añadidas propias.

4.7.3 Nomenclatura y reglas

De manera gráfica, este es el proceso de herencia entre clases:



En Programación Orientada a Objetos, la clase que va a ser heredada se llama superclase o clase base (o clase padre, en alguna literatura sobre Java) y la que hereda se le conoce como subclase o clase derivada (o clase hija).

Las reglas a seguir en la herencia son las siguientes:

- En Java no se permite la herencia múltiple, es decir una clase derivada solo puede heredar de una única clase base (es decir, una clase hija solo puede tener un padre).
- Está permitida la herencia multinivel, es decir una Clase A va a ser heredada por una clase B que a su vez va a ser heredada por una clase C.
- De una clase base pueden heredar varias subclases (para entendernos, una clase padre puede tener varias hijas).

4.7.4 Relación "Es un"

Para entender bien la herencia podemos tener la idea de la relación que se establece entre ambas clases. De este modo, un objeto de la clases "es un" objeto de la superclase. Por ejemplo, si tenemos una Superclase vehiculo y una subclase coche podemos decir "**Coche es un vehiculo**".

4.7.5 Creación de Herencia

Para definir una subclase que hereda de otra clase base se usa la palabra reservada `extends`. Por tanto su sintaxis será la siguiente:

```
public class subclase extends superclase {  
    // Código subclase  
}
```

Todas las clases de Java heredan de alguna clase. Si no es específica nada en el `extends`, se entiende que se hereda implícitamente de `Object`, del paquete `java.lang`, que es, de hecho, la superclase del resto de clases de Java.

Veamos un ejemplo completo.

En primer lugar nos creamos el proyecto `Ej08_SuperVehiculo`. Dentro incluimos `Consola.java` que tenemos en el **ANEXO1** de este mismo manual.

Este será la clase base (de la que heredará `Ej09_SubCamion`).

```
package ej08_supervehiculo;  
public class Ej08_SuperVehiculo {  
    // Atributos (protected, para ser visibles por subclases)  
    protected String color;  
    protected int potencia;  
    // Usamos la Consola para lectura/impresión  
    Consola miConsola = new Consola();  
  
    // Constructor por defecto de la Superclase  
    public Ej08_SuperVehiculo() {  
        color = "blanco";  
        potencia = 60;  
    }  
  
    public void leeVehiculo () {  
        color = miConsola.leeCadena(" Escriba Color: ");  
        potencia = miConsola.leeEntero(" Escriba Potencia: ");  
    }  
  
    public void imprimeVehiculo() {  
        miConsola.imprime(" Color: " + color);  
        miConsola.imprime(" Potencia: " + potencia);  
    }  
}
```

En el mismo paquete, incluimos **Ej09_SubCamion**:

```
package ej08_supervehiculo;  
public class Ej09_SubCamion extends Ej08_SuperVehiculo {  
    // Atributo de la Subclase  
    public boolean remolque;  
    // La consola, en este caso, se hereda de la superclase  
    // Se pone: super.teclado...
```

```

        // Constructor predefinido, derivado de la superclase
public Ej09_SubCamion() {
    super();
    remolque = false; // Nuevo atributo
}

public void imprimeCamion () {
    super.imprimeVehiculo(); // Método de la Superclase
    if (remolque)
        super.teclado.imprimeMensaje("Remolque: Si");
    else
        super.teclado.imprimeMensaje("Remolque: Si");
}

public void leeCamion () {
    miConsola.imprime(" Lecturas de Datos del Camión: ");
    super.leeVehiculo(); // Método de la Superclase

    String respuesta = miConsola.leeCadena(" Remolque? (SI/NO): ");
    if (respuesta.equals("SI"))
        remolque = true;
    else
        remolque = false;
}
}
}

```

Por último nos creamos un objeto de la clase derivada:

```

package ej08_supervehiculo;
public class Ej10_NuevoCamion {
    // Creamos el objeto usando static
    static Ej09_SubCamion miCamion1 = new Ej09_SubCamion();

    public static void main(String[] args) {
        miCamion1.leeCamion();
        miCamion1.imprimeCamion();
    }
}

```

4.7.6 *Ejecutar constructores con Herencia*

Cada vez que Java crea un objeto de una clase derivada, ejecuta, antes del constructor de esta clase, el constructor de la superclase. Siguiendo el ejemplo anterior:

```

public Ej09_SubCamion() {
    super();
    remolque = false; // Nuevo atributo
}

```

Los constructores por defecto también incluyen el `super ( )` como primera línea, puesto que, como hemos visto, toda las clases derivan en última instancia de la clase `java.lang.Object`.

Por otro lado, se puede dar el caso que, desde el constructor de la subclase se llame a otro constructor de la superclase. El procedimiento es muy parecido. Si tenemos en la superclase:

```
public Ej08_SuperVehiculo(String color, int potencia) {
    this.color = color;
    this.potencia = potencia;
}
```

En la subclase podemos llamar a este constructor, añadiendo un parámetro adicional, así:

```
public Ej09_SubCamion(String color, int potencia, boolean remolque) {
    super(color,potencia);
    this.remolque = remolque; // Nuevo atributo de la subclase
}
```

También podemos llamar a un constructor de la propia subclase desde otro constructor diferente, en la misma clase, empleando la palabra reservada `this`:

```
public Ej09_SubCamion(String color, int potencia,
    boolean remolque, String modelo) {
    this(color,potencia, remolque);
    this.modelo = modelo; // Otro atributo de la subclase
}
```

4.7.7 Métodos y atributos protegidos

Se emplea el modificador `protected` y son accesibles desde la propia clase o desde clases derivadas.

Mediante este modificador, podemos acceder, desde la subclase, a un método o atributo de la superclase, pero **NUNCA** hacer referencia a un objeto de la superclase:

```
public class persona {
    persona p = new Persona ();
    protected int getEdad () { ... }
}
public class alumno extends persona {
    imprime (""+ this.getEdad()); // Correcto, accedo a la superclase
    imprime (""+ p.getEdad()); // Acceso al objeto no permitido
}
```

Veamos otro ejemplo en el que importamos de manera estática `Consola` y usamos herencia:

Clase Vehículo

```
package Herencia;
import static Auxiliares.Consola.*;
import Auxiliares.Dimensiones;

public class Vehiculo {
    // Atributos
    protected String marca;
    protected String modelo;
    // Atributo Dimensiones
    protected Dimensiones dimensiones;
```

```

// Constructores
public Vehiculo() {
    dimensiones = new Dimensiones();
}

public Vehiculo(String marca, String modelo) {
    this.marca = marca;
    this.modelo = modelo;
    dimensiones = new Dimensiones();
}

// Método de Lectura
public void leeVehiculo() {
    System.out.println("Lectura Vehiculo");
    this.marca = leeCadena("Escriba Marca:");
    this.modelo = leeCadena("Escriba Modelo:");
    dimensiones = dimensiones.leeDimension();
}

@Override
public String toString () {
    String cadena = "";
    cadena += "\n Marca: "+this.marca;
    cadena += "\n Modelo: "+this.modelo;
    cadena += this.dimensiones.toString();
    return cadena;
}
}

```

Clase Coche

```

package Herencia;
import static Auxiliares.Consola.*;
import Auxiliares.Dimensiones;

public class Coche extends Vehiculo{
    // Atributos propios
    protected int numPuertas;
    protected int plazas;

    // Constructores
    // NO hace falta crear el objeto dimensiones
    // porque YA ESTA CREADO en la clase padre...
    public Coche() {}

    public Coche(String marca, String modelo,
        int numPuertas, int plazas) {
        super(marca, modelo);
        this.numPuertas = numPuertas;
        this.plazas = plazas;
    }
}

```

```

@Override
public void leeVehiculo () {
    System.out.println("Lectura Coche");
    this.marca = leeCadena("Escriba Marca:");
    this.modelo = leeCadena("Escriba Modelo:");
    dimensiones = dimensiones.leeDimension();
}

public void leeCoche() {
    // Si sobreescribimos leeVehiculo, este super no hace falta
    //super.leeVehiculo();
    leeVehiculo();
    this.numPuertas = leeEntero("Escriba Nº Puertas:");
    this.plazas = leeEntero("Escriba Nº Plazas:");
}

@Override
public String toString () {
    String cadena = super.toString();
    cadena += "\n Nº Puertas: "+this.numPuertas;
    cadena += "\n Nº Plazas: "+this.plazas;
    return cadena;
}

public static void main(String[] args) {
    Coche miCoche = new Coche();
    miCoche.leeCoche();
    imprimeMensaje(miCoche.toString());
}
}

```

4.7.8 Clases finales

Si queremos que una clase no sea heredada, debemos usar el modificador final. Sintaxis:

```
public final class nombreClase {  
    // ...  
}
```

Si una clase intenta heredar de esta, dará un error de compilación.

4.7.9 Sobreescritura de métodos

Una subclase tiene la opción de volver a reescribir un método heredado. Es lo que se conoce como sobreescritura de métodos. Para ello debemos seguir las siguientes normas:

- El método sobreescrito debe tener el mismo formato del método de la superclase, es decir, debe llamarse igual, tener los mismos parámetros y devolver el mismo tipo.
- El método sobreescrito en la subclase debe tener un modificador menos restrictivo que el de la superclase. Así, si este es `protected`, podemos poner el mismo o `public` (nunca `private`).
- Para llamar desde el interior de la subclase al método original de la superclase pondremos: `super.nombre_metodo (argumentos)`.
Si no se pone `super`, llamaríamos, en realidad, al método de la subclase.
- Por último, recordar que si no queremos que un método de superclase sea reescrito en una subclase, debemos usar el modificador final:
`public final void metodo () { ... }`

4.8 Clases abstractas

Tras la herencia, vamos a ver las clases abstractas, que son esenciales para entender el polimorfismo, una de las características mas importantes de la programación orientada a objetos.

4.8.1 Definición

Una clase abstracta es aquella en la que alguno o varios de sus métodos está declarado pero no definido; es decir, tiene nombre, parámetros y tipo de devolución, pero no lleva código. Son llamados **Métodos Abstractos**. Se usan junto a la herencia, porque son sus clases derivadas las que, mediante sobreescritura, implementarán dichos métodos y les dotará de funcionalidad.

¿Que conseguimos con las clases abstractas?

Resumiendo, obligar a que las subclases respeten el formato especificado en las superclases.

4.8.2 Sintaxis y características

La sintaxis de una clase abstracta es:

```
public abstract class nombre_clase {  
    public abstract tipo nombre_metodo (argumentos)  
    // Otros métodos  
}
```

Veamos un ejemplo. En primer lugar la superclase que contiene 3 métodos abstractos:

NOTA: Nos creamos un proyecto llamado **Ej11_FiguraAbstracta**

Dentro meteremos consola.java de otro proyecto anterior; eso si, cambiando el package.

```
package ej11_figuraabstracta;
```

```
public abstract class Ej11_FiguraAbstracta {

    // Los atributos de clase
    protected String color;

    // Creamos los constructores, uno con un parámetro
    public Ej11_FiguraAbstracta (String color) {
        this.color = color;
    }

    // Y otro, fundamental, sin parámetros.
    public Ej11_FiguraAbstracta () {
        this.color = "blanco";
    }

    public Ej11_FiguraAbstracta () {
        this.color = "blanco";
    }

    public abstract double area ();
    public abstract void leeDatos();
    public abstract void imprimeDatos();
}
```

De esta clase es importante reseñar, además de los métodos abstractos sin implementar y acabados en ; (sin código) que hemos incluido un constructor con argumentos y otro sin argumentos, para crear luego un objeto con valores predefinidos en los atributos (que luego iremos cambiando si hiciese falta).

Dentro del mismo package creamos una subclase que hereda de la anterior.

```
package ej11_figuraabstracta;
```

```
public class Ej12_SubCirculo extends Ej11_FiguraAbstracta {
    Consola miConsola = new Consola();
    private double radio;

    // Aunque no se emplee, lo tenemos para otra aplicación...
    public Ej12_SubCirculo (String color, double radio) {
        super.color = color;           // También vale: super(color);
        this.radio = radio;
    }
}
```

```

// De nuevo, el constructor por defecto
public Ej12_SubCirculo () {
    super();
    this.radio = 0.0;
}
// Ahora los métodos abstractos reescritos
public double area () {
    double resultadoArea = Math.PI * radio * radio;
    return resultadoArea;
}

public void leeDatos() {
    super.color = miConsola.leeCadena(" Escriba Color: ");
    this.radio = miConsola.leeDouble(" Escriba Radio Circulo: ");
}
public void imprimeDatos(){
    miConsola.imprime("  Datos Círculo");
    miConsola.imprime(" Color: "+super.color);

    // Observese como imprimimos la salida del método
    // sobreescrito abstracto de la superclase
    miConsola.imprime(" Área: "+area());
}

// Dentro del main, en la misma clase, creo el objeto
public static void main(String[] args) {
    Ej12_SubCirculo miCirculo = new Ej12_SubCirculo();
    miCirculo.leeDatos();
    miCirculo.imprimeDatos();
}
}

```

Como hemos visto en el ejemplo anterior, debemos seguir unas normas en la creación y uso de clases abstractas. Son estas:

- **Una clase abstracta puede tener método y/o atributos no abstractos.** Pero basta con que uno solo lo sea para que la clase deba ser declarada como abstracta.
- **No se pueden crear objetos de una clase abstracta.**
- Las subclases que heredan de una superclase abstracta **están obligadas a implementar todos los métodos abstractos que heredan.** Si uno solo no lo hace, obligará a que la subclase sea declarada también como abstracta.
- **Una clase abstracta puede tener constructores.** Si no lo tienen, se les añade uno por defecto, que será heredado por la subclase.

Veamos otro ejemplo aún mas claro:

Clase Figura

```
package Abstractas;
import static Auxiliares.Consola.*;

/**
 * Esta clase (super) implementa atributos que
 * serán heredados y métodos abstractos
 * @author ivan
 */
public abstract class Figura {
    // Atributos
    String color;

    public Figura() {
        color = "blanco";
    }

    public abstract double calculaArea ();
    public abstract double calculaPerimetro ();

    public void leeDatos () {
        this.color = leeCadena("Escriba Color:");
    }

    @Override
    public String toString () {
        String cadena = "\n Color: " + this.color;
        return cadena;
    }
}
```

Clase Triangulo

```
package Abstractas;
import static Auxiliares.Consola.*;
import static java.lang.Math.*;

public class Triangulo extends Figura{
    // Atributos
    double c1, c2, h;

    // Constructor por defecto
    public Triangulo() {}

    // Método de lectura de catetos
    @Override
    public void leeDatos () {
        System.out.println(" LECTURA DEL TRIÁNGULO ");
        super.color = leeCadena("Escriba Color: ");
        this.c1 = leeDouble("Escriba Base o Cateto1:");
        this.c2 = leeDouble("Escriba Altura o Cateto2:");
        h = sqrt(pow(c1,2)+pow(c2,2));
    }
}
```

```

@Override
public double calculaArea() {
    double area = (c1 * c2) / 2;
    return area;
}

@Override
public double calculaPerimetro() {
    double perimetro = c1+c2+h;
    return perimetro;
}

@Override
public String toString () {
    String cadena = "\n Datos del Triángulo ";
    cadena += super.toString();
    cadena += "\n Cateto1 o Base = "+this.c1;
    cadena += "\n Cateto1 o Altura = "+this.c2;
    cadena += "\n Hipotenusa = "+this.h;
    cadena += "\n Área = "+ calculaArea();
    cadena += "\n Perímetro = "+ calculaPerimetro();
    return cadena;
}

public static void main(String[] args) {
    Triangulo miTriangulo = new Triangulo();
    miTriangulo.leeDatos();
    imprimeMensaje(miTriangulo.toString());
}
}

```

Clase Rectángulo

```

package Abstractas;
import static Auxiliares.Consola.*;

public class Rectangulo extends Figura{
    // Atributos
    double base, altura;

    // Constructor
    public Rectangulo() {}

    @Override
    public double calculaArea() {
        double area = base * altura;
        return area;
    }

    @Override
    public double calculaPerimetro() {
        double perimetro = (base + altura) *2;
        return perimetro;
    }
}

```

```

@Override
public void leeDatos () {
    System.out.println(" LECTURA DEL RECTÁNGULO ");
    super.color =leeCadena("Escriba Color: ");
    this.base = leeDouble("Escriba Base:");
    this.altura = leeDouble("Escriba Altura:");
}

@Override
public String toString () {
    String cadena = "\n Datos del Rectángulo ";
    cadena += super.toString();
    cadena += "\n Base = "+this.base;
    cadena += "\n Altura = "+this.altura;
    cadena += "\n Área = "+ calculaArea();
    cadena += "\n Perímetro = "+ calculaPerimetro();
    return cadena;
}

public static void main(String[] args) {
    Rectangulo miRectangulo = new Rectangulo();
    miRectangulo.leeDatos();
    imprimeMensaje(miRectangulo.toString());
}
}

```

4.9 Polimorfismo

El polimorfismo se base en parte en la herencia y el uso de clases abstractas.

4.9.1 Asignar objetos a variables de la Superclase

En Java podemos asignar un objeto de una clase a una variable de su superclase, incluso aunque esta sea abstracta. Por ejemplo, siguiendo el ejemplo anterior, podemos poner:

```
Ej11_FiguraAbstracta figura;
```

Y luego podemos asignar esta variable a un objeto Rectángulo o Trapecio:

```
figura = new Ej13_SubRectangulo ();
```

```
figura = new Ej14_SubTrapecio ();
```

De este modo, podemos usar esta variable para invocar aquellos métodos del objeto que también estén definidos o declarados en la superclase (de ahí la importancia de las clases abstractas) pero no aquellas que solo existan en la clase a la que pertenece el objeto.

4.9.2 Definición

Mediante el polimorfismo, creamos una variable de la superclase que luego asignamos a distintos objetos para luego usar los mismos métodos en cada uno de ellos. Por ejemplo:

```
Ej11_FiguraAbstracta figura;
```

```
// Y ahora, con los mismos métodos, creo un rectangulo
```

```
figura = new Ej13_SubRectangulo ();
```

```
figura.leeDatos();
```

```
figura.imprimeDatos();
```

```
// Y un trapecio...
```

```
figura = new Ej14_SubTrapecio ();
```

```
figura.leeDatos();
```

```
figura.imprimeDatos();
```

El ejemplo completo sería el siguiente (seguimos en el proyecto Ej11_FiguraAbstracta).

NOTA IMPORTANTE: Para probar el ejemplo anterior pusimos en Ej12_SubCirculo un método main. Este método DEBEMOS QUITARLO para poder ver el polimorfismo.

Siguiendo con el ejemplo (tras hacer Ej11_FiguraAbstracta y Ej12_SubCirculo) nos creamos la clase

Ej13_SubRectangulo:

```
package ej11_figuraabstracta;
```

```
/**
```

```
 * @author Iván Rodríguez
```

```
 */
```

```
public class Ej13_SubRectangulo extends Ej11_FiguraAbstracta {
```

```
    Consola miConsola = new Consola();
```

```
    private double base;
```

```
    private double altura;
```

```

// Aunque no se emplee, lo tenemos para otra aplicación...
public Ej13_SubRectangulo (String color, double base, double altura) {
    super(color);
    this.base = base;
    this.altura = altura;
}

// De nuevo, el constructor por defecto
public Ej13_SubRectangulo () {
    super();
    this.base = 0.0;
    this.altura = 0.0;
}

// Ahora los métodos abstractos reescritos
// Obsérvese como hemos añadido un @override (sobreescrito)
@Override
public double area () {
    double resultadoArea = base * altura;
    return resultadoArea;
}

@Override
public void leeDatos() {
    super.color = miConsola.leeCadena(" Escriba Color: ");
    this.base = miConsola.leeDouble(" Escriba Base: ");
    this.altura = miConsola.leeDouble(" Escriba Altura: ");
}

@Override
public void imprimeDatos(){
    miConsola.imprime("  Datos Rectángulo  ");
    miConsola.imprime(" Color: "+super.color);

    // Obsérvese como imprimimos la salida del método
    // sobreescrito abstracto de la superclase
    miConsola.imprime(" Área: "+area());
}
}

```

Y ahora la clase **Ej14_SubTrapezio**:

```
package ej11_figuraabstracta;
public class Ej14_SubTrapezio extends Ej11_FiguraAbstracta{
    Consola miConsola = new Consola();
    private double baseMenor;
    private double baseMayor;
    private double altura;
    // Aunque no se emplee, lo tenemos para otra aplicación...
    public Ej14_SubTrapezio (String color, double baseMenor,
        double baseMayor, double altura) {
        super(color);
        this.baseMenor = baseMenor;
        this.baseMayor = baseMayor;
        this.altura = altura;
    }

    // De nuevo, el constructor por defecto
    public Ej14_SubTrapezio () {
        super();
        this.baseMenor = 0.0;
        this.baseMayor = 0.0;
        this.altura = 0.0;
    }

    // Ahora los métodos abstractos reescritos
    @Override
    public double area () {
        double resultadoArea = (baseMenor+baseMayor)/2 * altura;
        return resultadoArea;
    }
    @Override
    public void leeDatos() {
        super.color = miConsola.leeCadena(" Escriba Color: ");
        this.baseMenor = miConsola.leeDouble(" Escriba Base Menor: ");
        this.baseMayor = miConsola.leeDouble(" Escriba Base Mayor: ");
        this.altura = miConsola.leeDouble(" Escriba Altura: ");
    }

    @Override
    public void imprimeDatos(){
        miConsola.imprime("  Datos Trapecio  ");
        miConsola.imprime(" Color: "+super.color);

        // Observese como imprimimos la salida del método
        // sobreescrito abstracto de la superclase
        miConsola.imprime(" Área: "+area());
    }
}
```

Por último, implementamos la clase **Ej15_Polimorfismo** donde vemos esta característica fundamental de la programación orientada a Objetos.

```
package ej11_figuraabstracta;
public class Ej15_Polimorfismo {
    public static void main(String[] args) {
        // Creo la variable de la superclase
        Ej11_FiguraAbstracta figura;
        // POLIMORFISMO
        // Uso el mismo puntero de la Superclase para referenciar
        // Objetos de diferentes clases derivadas

        // Y ahora, con los mismos métodos, creo un rectángulo
        figura = new Ej13_SubRectangulo ();
        figura.leeDatos();
        figura.imprimeDatos();

        // Y un trapecio...
        figura = new Ej14_SubTrapecio ();
        figura.leeDatos();
        figura.imprimeDatos();
    }
}
```

4.9.3 Ventajas

Mejorar el mantenimiento y, sobre todo, **Reutilización de código**.

4.10 **Interfaces**

Una interfaz es un conjunto de métodos abstractos y constantes públicos definidos en un clase Java. En esencia, una interfaz es una clase abstracta donde todos sus métodos son abstractos. La utilidad de una interfaz es definir el formato que deben tener determinados métodos que serán usados por ciertas clases que heredan de ella.

Un ejemplo de interfaz lo vemos en clases para gestión de entornos gráficos (que veremos mas adelante), donde dichas clases deben capturar ciertos eventos que serán codificados por unos métodos determinados cuyo formato han sido definidos en una interfaz.

Una interfaz no establece lo que un método debe hacer y cómo hacerlo, sino que especifica su formato (nombre, parámetros y tipo de devolución).

4.10.1 Definición

Una interfaz se define con la palabra reservada interface, con la siguiente sintaxis:

```
[public] interface Nombre_Interfaz {
    tipo metodo1 (argumentos);
    tipo metodo2 (argumentos);
    ...
    tipo metodoN (argumentos);
}
```

Si la interfaz usa el modificador public, deberá llamarse igual que el archivo .java que lo contiene. Además, a la hora de crear una interfaz debemos seguir las siguientes reglas:

- **Todos los métodos de la interfaz son públicos y abstractos.** De todos modos, aunque no se pongan, son asignados implícitamente (aunque se recomienda ponerlos).
- **En una interfaz se pueden definir constantes.** Estas son públicas y estáticas de forma implícita (es decir, no hace falta poner public, static o final).
- **Una interfaz no es una clase.** Pueden tener métodos abstractos y constantes, pero no tienen ni código, ni constructores, ni atributos. Y por supuesto, no se pueden crear objetos de ellos.

4.10.2 Implementación

El objetivo de las interfaces es proporcionar un formato común de métodos a las clases que derivan de ella. Para forzar a dichas clases a seguir el formato anterior, debemos usar la palabra reservada **implements**.

Su sintaxis será:

```
public class Nombre_Clase implements Nombre_Interfaz {  
    // ...  
}
```

En la implementación de interfaces debemos seguir las siguientes reglas:

- Cuando una clase implementa una interfaz debe definir el código de los métodos incluidos en la misma, sin excepción. Es el mismo caso que en las clases abstractas.
- Una clase puede implementar mas de una interfaz. En tal caso, debe implementar TODOS los métodos de todas las interfaces. La sintaxis en tal caso será:
public class Nombre_Clase
 implements Interfaz1, Interfaz2,..., InterfazN { ... }
- Una clase puede heredar de otra clase e implementar al mismo tiempo una o varias interfaces. Con esto, implementamos los métodos de las interfaces y además heredamos métodos y atributos de la superclase. En tal caso, la sintaxis será:
public class Nombre_Clase extends SuperClase
 implements Interfaz1, Interfaz2,..., InterfazN { ... }
NOTA: Este será el caso de ejemplo que vamos a desarrollar, que incluye todas las posibilidades que hemos visto en este apartado.
- Una interfaz puede heredar de otras interfaces. No es un caso de Herencia como tal, puesto que solo adquirimos métodos abstractos. Su sintaxis será:
public interface Nombre_Clase
 extends Interface1, Interfaz2,..., InterfazN { ... }

Ejemplo Completo

En este caso vamos a crearnos un proyecto llamado EJ16_InterfaceDatosPersonales, que será la primera interfaz a desarrollar. Dentro del mismo incluiremos Consola.java (a la que tendremos que cambiar el paquete) y el resto de clases de este ejemplo. Gráficamente quedará así:

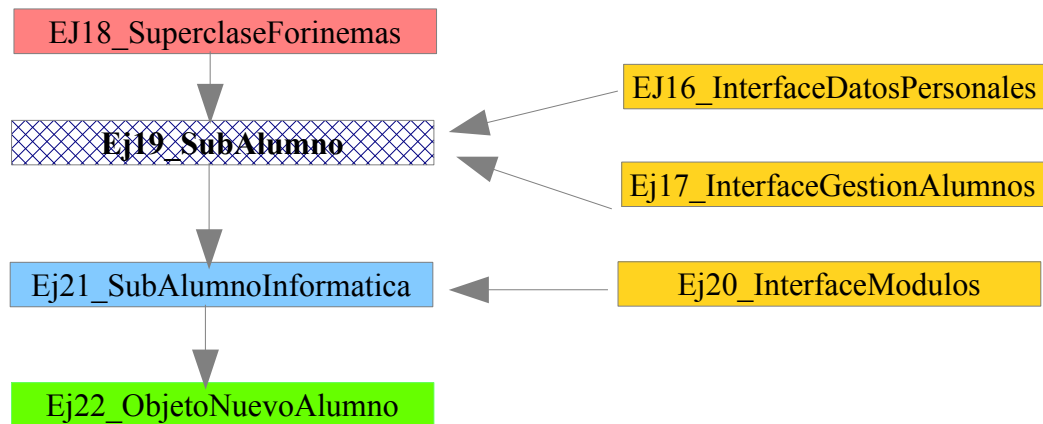
Colores: Rojo -> Superclase;

Amarillo -> Interfaces

Trama -> Subclase – Superclase

Azul -> Clase

Verde -> Clase + Main (con Objeto)



1) EJ16_InterfaceDatosPersonales

```
package ej16_interfacedatospersonales;
/**
 * <h1> Proyecto de Interfaces y Herencia
 * @author Iván Rodríguez
 * @version 1.0
 * Creado: 12 Noviembre 2014
 */
public interface EJ16_InterfaceDatosPersonales {
    // Quitamos el class y ponemos dos métodos abstractos
    public void leeDatosPersonales();
    public void imprimeDatosPersonales();
}
```

2) Ej17_InterfaceGestionAlumnos

```
package ej16_interfacedatospersonales;

public interface EJ17_InterfaceGestionAlumnos {
    // Constante de la interfaz
    public static final String TITULO = "  DATOS ALUMNO  ";
    // Métodos de la interfaz
    public void leeGestion();
    public void imprimeGestion();
}
```

NOTA: Por convenio se ponen las constantes en mayúsculas.

3) **Ej18_SuperclaseForinemas**

```
package ej16_interfacedatospersonales;
```

```
public class Ej18_SuperclaseForinemas {  
    // Observese que creamos el objeto miConsola  
    // en la Superclase, heredandola el resto de clases  
    static Consola miConsola = new Consola();  
  
    // Atributo de la Superclase  
    public int aula;  
  
    public Ej18_SuperclaseForinemas() {  
        aula = 2;  
    }  
}
```

4) **Ej19_SubAlumno**

```
package ej16_interfacedatospersonales;
```

```
public class Ej19_SubAlumno extends Ej18_SuperclaseForinemas  
implements Ej16_InterfaceDatosPersonales, Ej17_InterfaceGestionAlumnos {  
    // Atributos de esta Superclase (que es derivada de Forinemas)  
    public boolean sexo;  
    public String nif;  
    public String nombre;  
    public String apellido1;  
    public int modulo;  
    public boolean activo;  
  
    public Ej19_SubAlumno () {  
        // Constructor por defecto...  
    }  
  
    // Implementamos los métodos de las interfaces  
    @Override  
    public void leeGestion() {  
        super.aula = miConsola.leeEntero(" Escriba Aula (1 3): ");  
        modulo = miConsola.leeEntero(" Escriba Módulo (1 4): ");  
        String respuesta = miConsola.leeCadena(" Está Activo: (SI/NO)");  
        if (respuesta.equals("SI"))  
            activo = true;  
        else  
            activo = false;  
    }  
}
```

```

@Override
public void imprimeGestion() {
    miConsola.imprime(" Aula: " + super.aula);
    miConsola.imprime(" Módulo: " + modulo);
    if (activo)
        miConsola.imprime(" Activo: SI");
    else
        miConsola.imprime(" Activo: NO");
}

```

```

@Override
public void leeDatosPersonales() {
    nombre = miConsola.leeCadena(" Escriba Nombre: ");
    apellido1 = miConsola.leeCadena(" Escriba Apellido1: ");
    nif = miConsola.leeCadena(" Escriba NIF: ");
    String nuevoSexo = miConsola.leeCadena(" Escriba Sexo: (H/M)");
    sexo = nuevoSexo.equalsIgnoreCase("M");
}

```

```

@Override
public void imprimeDatosPersonales() {
    miConsola.imprime(" \n\n");
    // Recojo constante de la interfaz Ej17_InterfaceGestionAlumnos
    miConsola.imprime(TITULO);
    miConsola.imprime(" Nombre: "+ nombre);
    miConsola.imprime(" Apellido1: "+ apellido1);
    miConsola.imprime(" NIF: "+ nif);
    if (sexo == true)
        miConsola.imprime(" Sexo: Mujer");
    else
        miConsola.imprime(" Sexo: Hombre");
}
}

```

5) Ej20_InterfaceModulos

```

package ej16_interfacedatospersonales;

public interface Ej20_InterfaceModulos {
    // Método de la interfaz
    public String nombreModulo (int modulo);
}

```

6) Ej21_SubAlumnoInformatica

```
package ej16_interfacedatospersonales;
public class Ej21_SubAlumnoInformatica extends Ej19_SubAlumno
    implements Ej20_InterfaceModulos {
    public boolean practica;
    @Override // Implementamos el método de la Interfaz
    public String nombreModulo(int modulo) {
        String nombreMod = "";
        switch (modulo) {
            case 1:         nombreMod = "MySQL";    break;
            case 2:         nombreMod = "Java";      break;
            case 3:         nombreMod = "JSP";       break;
            case 4:         nombreMod = "PHP";       break;
        }
        return nombreMod;
    }
    // Constructor por defecto
    public Ej21_SubAlumnoInformatica () {
        practica = false;
    }

    // Pongo los métodos clásicos de Lectura e Impresión
    public void leeAlumnoInf () {
        leeDatosPersonales();
        leeGestion();
        String respuesta =
            miConsola.leeCadena(" Ha presentado las prácticas? (SI/NO)");
        if (respuesta.equals("SI"))
            practica = true;
        else
            practica = false;
    }

    public void impAlumnoInf () {
        imprimeDatosPersonales();
        imprimeGestion();
        if (practica)
            miConsola.imprime(" Practicas Presentadas: SI");
        else
            miConsola.imprime(" Practicas Presentadas: NO");

        miConsola.imprime(" Modulo actual: " + nombreModulo(modulo));
    }
}
```

7) Ej22_ObjetoNuevoAlumno

package ej16_interfacedatospersonales;

```
public class Ej22_ObjetoNuevoAlumno {
    static Ej21_SubAlumnoInformatica alumno1 =
        new Ej21_SubAlumnoInformatica();
    public static void main(String[] args) {
        alumno1.leeAlumnoInf();
        alumno1.impAlumnoInf();
    }
}
```

4.10.3 Interfaces y polimorfismo

Una variable de tipo interfaz puede almacenar cualquier objetos de las clases que la implementan, pudiendo usar esa variable para invocar a los métodos del objeto que han sido declarados en la interfaz e implementados en la clase. Siguiendo con el ejemplo anterior podríamos hacer:

```
Ej16_InterfaceDatosPersonales datos = new Ej19_SubAlumno ();
```

```
datos.leeDatosPersonales();
```

```
datos.imprimeDatosPersonales();
```

Esta capacidad, unida a la flexibilidad de las interfaces lo hacen esenciales para programar.

4.10.4 Interfaces en el J2SE

Los paquetes estándar de Java incluyen interfaces implementadas en las clases J2SE que pueden ser usadas en distintas aplicaciones. Las mas importantes son:

- **java.lang Runnable.** Para aplicaciones multitarea.
- **java.util Enumeration.** Para realizar colecciones de objetos.
- **java.awt.event WindowListener.** Para gestionar eventos en aplicaciones gráficas AWT
- **java.sql.Connection.** Para manejar conexiones a bases de datos.
Esta, en concreto, la estudiaremos en breve.
- **Java.io.Serializable.** Para salidas a ficheros externos.

5 ACCESO A DATOS EN JAVA (JDBC)

Aunque podemos almacenar y recuperar información mediante ficheros de disco, la mayoría de los programas Java emplean para ello los datos almacenados en Bases de Datos. Para el uso de las mismas disponemos de unos estándares que facilitan el trabajo. En Java disponemos del API JDBC para realizar dichas tareas.

5.1 Java Database Connectivity (JDBC)

JDBC proporciona a las aplicaciones Java un mecanismo uniforme para el acceso a datos, enviando instrucciones SQL estándar y recogiendo / tratando los resultados obtenidos.

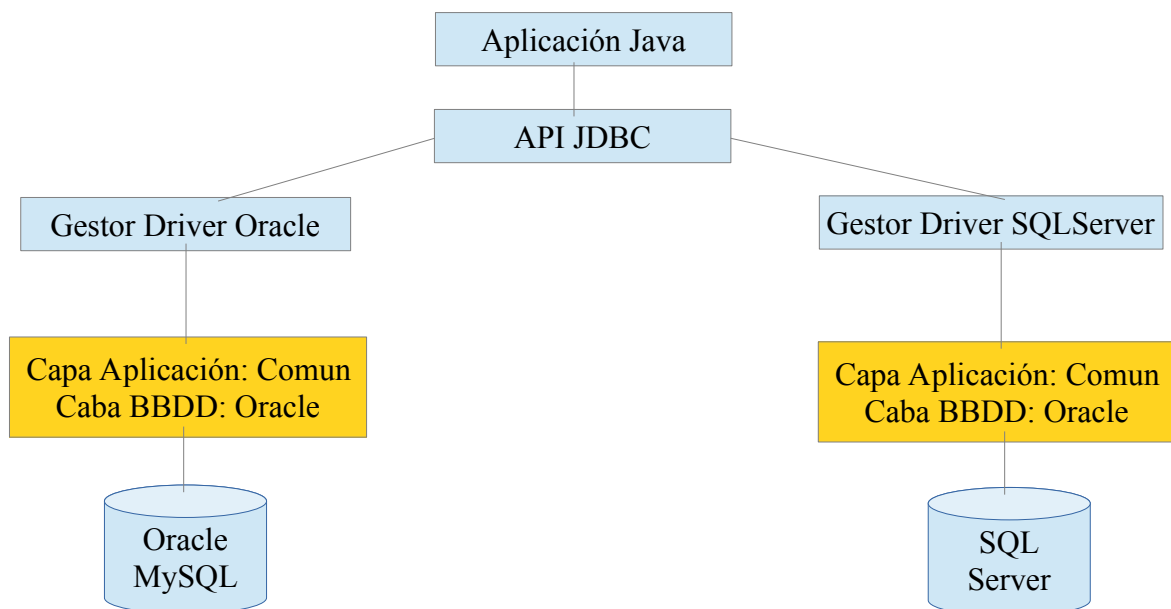
La tecnología JDBC usa un conjunto de clases (API JDBC) que disponen de varios métodos para operar con bases de datos. Estos métodos dirigen las peticiones a un software intermediario llamado Driver JDBC cuyo propósito es traducir estas peticiones a ordenes nativas del Sistema Gestor de Base de Datos (SGBD) que puede ser MySQL 5, Oracle 11g, SQL Server, etc.

5.2 El driver JDBC

El driver JDBC es el encargado de enlazar las aplicaciones Java con el SGBD y juega un papel fundamental en las aplicaciones con acceso a base de datos. Vamos a ver, de forma resumida, su funcionamiento.

5.3 Estructura y funcionamiento

Normalmente son los fabricantes de los SGBD los que distribuyen los driver JDBC específicos para sus propios sistemas. Únicamente tendremos que descargar dichos driver y usarlos en nuestras aplicaciones de bases de datos. Veámoslo de forma gráfica:



En un driver JDBC se distinguen dos capas o interfaces:

- **Capa de aplicación:** es la parte del driver que interactúa con la aplicación. Todos los driver proporcionan la misma interfaz común para esta función.
- **Capa de base de datos (BBDD):** esta capa interactúa con la base de datos por lo que es específica de cada fabricante de SGBD.

5.4 Tipos de driver JDBC

Independientemente del tipo de base de datos para el que haya sido diseñado, cada driver JDBC pertenece a una de las siguientes categorías de tipos:

- Driver JDBC-ODBC
- Driver nativo
- Driver intermedio
- Driver puro-Java

5.4.1 Driver JDBC-ODBC

Fue de los primeros driver JDBC que aparecieron y sirve como intermediario entre el propio driver JDBC y el SGBD (es decir, añade una capa adicional) . En aquel momento el driver ODBC era el mas usado y dado el amplio número de drivers ODBC podemos usar este driver puente para la mayoría de las bases de datos del mercado.

Tiene dos grandes desventajas: añade una capa adicional (empeora el rendimiento) y necesita configurar el DSN (Data Source Name) del equipo que va a ejecutar la aplicación. Es tratada por la clase `sun.jdbc.odbc.JdbcOdbcDriver`.

5.4.2 Driver nativo

El driver nativo convierte las llamadas JDBC en llamadas al API nativo del SGBD. Su principal inconveniente es que hace falta instalar el API nativo en el equipo donde reside la aplicación Java.

5.4.3 Driver intermedio

Similar al ODBC, convierte las llamadas JDBC a un protocolo específico de middleware que a su vez de encarga de conectar con el API nativo del SGBD. La principal ventaja que tiene es que no hace falta instalar el API nativo en el equipo donde reside la aplicación Java

5.4.4 Driver puro-Java

Convierte las llamadas JDBC en protocolo de red usado directamente por el servidor de BBDD, permitiendo llamadas directas entre la máquina cliente (la aplicación Java) y el SGBD. La principal ventaja que tiene es que no hace falta ningún tipo de configuración especial en la máquina cliente.

5.5 Lenguaje SQL

Es Structured Query Language es un estándar usado por todos los fabricantes de SGBD. En este apartado vamos a ver de forma muy resumida la sintaxis de los principales comandos que emplearemos en las conexiones a bases de datos.

Consultas

Su sintaxis es la siguiente:

```
SELECT campo1, campo2..., campoN
```

```
FROM tabla
```

```
[WHERE criterios filtro]
```

```
[ORDER BY campo]
```

Las cláusulas WHERE y ORDER BY son opcionales.

Un ejemplo:

```
SELECT nombre, ciudad  
FROM usuarios  
WHERE ciudad = "Sevilla"  
ORDER BY nombre;
```

Inserción de Registros

Su sintaxis es la siguiente:

```
INSERT INTO tabla (campo1, campo2, ..., campoN)  
VALUES (valor1, valor2, ..., valorN)
```

Cada valor es asignado al campo correspondiente al orden especificado.

Veamos un ejemplo:

```
INSERT INTO usuarios (idUserio, nombre, ciudad)  
VALUES ("1", "Ivan", "Sevilla");
```

Borrado de Registros

Su sintaxis es la siguiente:

```
DELETE FROM tabla WHERE condiciones
```

Veamos un ejemplo:

```
DELETE FROM usuarios WHERE ciudad = "Valencia";
```

Modificar Registros

Su sintaxis es la siguiente:

```
UPDATE table  
SET campo1 = valor1, campo2 = valor2, ... , campoN = valorN  
WHERE condiciones
```

Veamos un ejemplo

```
UPDATE usuarios  
SET ciudad = "Las Palmas de Gran Canaria"  
WHERE ciudd = "Las Palmas";
```

5.6 El API JDBC

Las clases y paquetes que forman parte de este API se encuentran en el paquete java.sql. Veamos los elementos mas importantes de este paquete:

Clase / Interfaz	Función
DriverManager	Establece conexiones a la BBDD a través del driver
Connection	Representa la conexión a la BBDD
Statement	Ejecución de consultas SQL
PreparedStatement	Ejecuta consultas preparadas y procedimientos almacenados
ResultSet	Manipula registros en consultas de tipo SELECT
ResultSetMetadata	Proporciona información sobre la estructura de datos

5.7 Uso de JDBC para acceso a Datos

En toda aplicación que use JDBC para acceder a Bases de datos, se distinguen cuatro fases o pasos a realizar, que son los siguientes:

- Conexión a Base de Datos
- Ejecución de consultas
- Manipulación de registros
- Cierre de la conexión

NOTA: Previamente a estas, debemos tener iniciado el SGBD, creada la Base de datos, las tablas correspondientes y, por supuesto, añadir registros en estas.

En el ejemplo que vamos a ver, usaremos MySQL como SGBD. Instalamos su servidor mediante XAMPP, lo iniciamos y procedemos con las siguiente sentencias SQL:

5.7.1 Conexión a Base de Datos

El procedimiento para añadir el driver al IDE y poder usarlo en nuestros proyectos es el siguiente:

- En primer lugar, debemos bajarnos el conector jdbc en ZIP para mysql en esta página:
<http://dev.mysql.com/downloads/connector/j/5.0.html>
- Descomprimos donde queramos y nos vamos al NetBeans.
- Nos creamos un nuevo proyecto en la carpeta Tema07 llamado **Ej23_EjemploJDBC**
- En la carpeta Bibliotecas del proyecto, Agregar archivo JAR y elegimos el .jar que sale de descomprimir el zip que nos hemos bajado de la web.

Ahora comenzamos con el proceso de conexión a la BBDD que tendrá dos pasos:

1. Carga del Driver
2. Creación de la conexión

Carga del Driver

Con esta acción se prepara el driver para poder ser usado. Para ello emplearemos el método estático `forName()` de la clase `java.lang.Class` cuyo formato es:

`Class.forName(String clase_driver)`

En nuestro caso, la instrucción será:

```
Class.forName("com.mysql.jdbc.Driver");
```

Este método localiza, lee y enlaza dinámicamente el driver, devolviendo un objeto `class` asociado a dicha clase. Hay que tener en cuenta que este método lanza una excepción de tipo `ClassNotFoundException`, por lo que deberemos capturarlo con la aplicación.

Para ello tenemos dos opciones:

1. Lanzar la excepción. Para ello usaremos:
`throws ClassNotFoundException`
2. Capturar completamente la excepción. Para ello usaremos:

```
try {  
    Class.forName("com.mysql.jdbc.Driver");  
}  
  
catch (ClassNotFoundException error) {  
    //...  
}
```

En nuestro ejemplo tomaremos la primera opción, que despeja mucho el código.

Creación de la conexión

Una vez cargado el driver debemos conectarnos a la BBDD empleando el método estático **getConnection()** de la clase **DriverManager** y que meteremos en objeto de tipo **Connection**. Su sintaxis será:

```
Connection objetoConexion =  
    DriverManager.getConnection (String url, usuarioBBDD, claveBBDD);
```

Donde url es la dirección de la base de datos y su formato será:
jdbc:subprotocolo://maquina:puerto

En nuestro caso, será:

```
jdbc:mysql://localhost:3306/<bbdd>
```

Este método, al igual que otros del API JDBC, lanza una excepción de tipo **SQLException** que podremos coger con un **throws** o un **try...catch**.

5.7.2 Ejecución de consultas

Una vez establecido la conexión a la base de datos con el objeto **Connection**, esta se empleará para enviar consultas SQL a la base de datos.

Creación del Objeto Statement

Las consultas SQL se manejan mediante objetos que implementan la interfaz **Statement**, cuya creación se realiza con el método **createStatement()** de la interfaz **Connection**:

```
Statement nombre_statement = nombreConexion.createStatement();
```

Esta operación lanza una excepción de tipo **SQLException**.

Ejecución de la consulta SQL

La interfaz **Statement** proporciona varios métodos para enviar una consulta SQL. Los mas importantes son:

- **boolean execute (String sql):** Devuelve false en caso de consultas de acción (Insert, Update y Delete) y true si son de selección (Select).
- **int executeUpdate (String sql):** Envía una consulta de acción y devuelve el número de registros afectados por dicha acción.
- **ResultSet executeQuery (String sql):** Envía una consulta de selección de registros y devuelve un objeto **ResultSet** (similar a un array) para su manipulación.

Los tres métodos lanzan una excepción de tipo **SQLException**.

5.7.3 Cierre de la conexión

Las consultas de acción no devuelven datos, por lo que una vez ejecutadas debemos proceder al cierre de la conexión. Esto se hace mediante el método **close()** de la interfaz **Connection**:

```
nombreConexion.close ();
```

Este método también lanza una excepción de tipo **SQLException**.

Por otro lado la interfaz **Statement** también dispone de un método **close ()** para liberar el objeto **Statement**, que deberá hacerse antes de cerrar la conexión. Al hacer esto último, de todos modos, se cierran todos los objetos **Statement** abiertos de manera automática.

5.7.4 Manipulación de registros

El envío de una consulta de registros a la BBDD devuelve como resultado un objeto que dispone de un cursor que permite desplazarse por los distintos registros seleccionados a través de sus campos. Este objeto implementa la interfaz ResultSet.

Obtener un objeto ResultSet

Se crea al ejecutar un executeQuery del objeto Statement. Es de solo lectura y permite únicamente el avance por los distintos campos de los registros, que no podrán ser modificados.

Desplazamiento por los registros

Una vez obtenido el ResultSet, el cursor se encuentra en la posición anterior al primer registro. Para realizar el desplazamiento debemos emplear el método next (normalmente dentro de un while, para ir recorriendo cada registro hasta que llegue al final). Comienza el recorrido con el primer campo y acaba la iteración con el último. Su sintaxis:

```
while (resultSet.next ( ))
{
    // sentencias
}
```

Acceso a los campos

La interfaz ResultSet aporta dos grupos de métodos para acceder a los campos del registro actual (el que apunta el cursor). Son los siguientes:

- xxx getXXX (int posición)
- xxx getXXX (String nombre_campo)

En este caso xxx es cualquier tipo básico de Java, mas date, String y Object.

El primer grupo de métodos accede a los campos por su posición (siendo el primero el 1) y el segundo a través de su nombre (que será la opción elegida en este manual).

Otros métodos de la interfaz ResultSet

Además de los anteriores, la interfaz resultSet proporciona los siguientes métodos para gestionar el cursor que, no olvidemos, es de solo avance y solo lectura:

- **boolean isFirst ()**: devuelve true si el cursor está en el primer registro.
- **boolean isBeforeFirst ()**: devuelve true si el cursor está antes del primer registro.
- **boolean isLast ()**: devuelve true si el cursor está en el último registro.
- **boolean isAfterLast ()**: devuelve true si el cursor está después del último registro.
- **int getRow ()**: Devuelve la posición del registro actual, siendo 1 para el primer registro.

Todos los métodos de la interfaz ResultSet lanzan una excepción de tipo SQLException.

Cierre de un ResultSet

El método close () permite cerrar el objeto y liberar recursos para mejorar el rendimiento. Si un mismo objeto Statement se usa para un segundo ResultSet, el primero se cierra de forma implícita. Esto significa que si queremos tener dos ResultSet abiertos al mismo tiempo, tendremos que crearlos sendos objetos Statement.

5.7.5 Información sobre los datos

Además de los datos, el API SQL proporciona una interfaz llamada `ResultSetMetaData` que permite obtener información sobre las características de los datos referenciados por un `ResultSet`.

Obtener un objeto `ResultSetMetaData`

Para ello usaremos el método `getMetaData` de la interfaz `ResultSet`.

Si sintaxis será la siguiente:

```
ResultSetMetaData rmt = rs.getMetaData ( );
```

Acceso a la información

La interfaz `ResultSetMetaData` aporta los siguientes métodos para obtener información de los datos obtenidos de la base de datos:

- `int getColumnCount`: Devuelve el número de campos o columnas del conjunto de registros.
- `String getColumnName (int posicion)`: Permite obtener el nombre de un campo a partir de su posición
- `int getColumnType (int posicion)`: devuelve una constante entera que representa el tipo de dato SQL soportado en el campo. Las constantes de tipos SQL se encuentran definidas en la clase `java.sql.types` según la siguiente tabla:

Constantes <code>java.sql.types</code>	Tipo equivalente Java
SMALLINT	short
INTEGER	int
BIGINT	long
FLOAT	float
DOUBLE	double
BOOLEAN	boolean
VARCHAR	String

- `String getColumnName (int posición)`: Devuelve el nombre del tipo de dato soportado por el campo, según está definido en el SGDB.

Veamos todo lo visto en este tema con un ejemplo completo.

NOTA: Aquí únicamente veremos las consultas de selección. En el siguiente capítulo, mediante **Swing**, veremos las consultas de acción UPDATE, INSERT y DELETE.

Para ello nos creamos un proyecto llamado **Ej23_EjemploJDBC**, incluimos entre sus librerías el driver jdbc, como ya vimos anteriormente y nos vamos creando, por orden, estas clases:

1) Una clase de conexión a base de datos, `ConexionBBDD.java`

En esta clase implementamos todas las posibles acciones a realizar con la BBDD:

```
package ej23_ejemplojdbc;  
import java.sql.Connection;  
import java.sql.DriverManager;  
import java.sql.ResultSet;  
import java.sql.SQLException;  
import java.sql.Statement;
```

```

public class ConexionBBDD {
    // Miembros de la clase (atributos)
    private Connection conexion;
    private String conector, servidorBBDD, BBDD;
    private String usuario, clave;
    private String cadenaConexion;
    private ResultSet resultado;

    public Connection conexionBBDD (String bbdd)
        throws SQLException, ClassNotFoundException {

        // Cargamos el driver
        Class.forName("com.mysql.jdbc.Driver");

        // Usamos el método estático getConnection
        BBDD = bbdd;
        cadenaConexion = conector + servidorBBDD + BBDD;
        conexion =
            DriverManager.getConnection(cadenaConexion, usuario, clave);
        return conexion;
    }
    public ResultSet ejecutaQuery (String consulta, String mensaje)
        throws SQLException {
        Statement ejecutar = conexion.createStatement();
        resultado = ejecutar.executeQuery(consulta);
        System.out.println(mensaje);
        //ejecutar.close();
        return resultado;
    }
}

```

2) Nos creamos una clase llamada **ObjetoUsuario** que será una superclase para definir la estructura de los datos obtenidos de la tabla usuarios:

```

package ej23_ejemplojdbc;
public class ObjetoUsuario {
    // Miembros de la Superclase
    int usuarioID;
    String nombre;
    String clave;
    int edad;
    String Provincia;
    boolean sexo;
    float altura;
    float peso;

    // Constructor por defecto
    public ObjetoUsuario() {}
}

```

3) En la siguiente clase, **SelectUsuarios**, es donde ejecutamos la consulta de Selección y vemos los resultados impresos por pantalla.

```
package ej23_ejemplojdbc;
import java.sql.Connection;
import java.sql.ResultSet;
import java.sql.SQLException;

public class SelectUsuarios extends ConexionBBDD {
    // Los miembros de la clase
    private Connection conexion;
    ResultSet resultado;

    // Constructor predeterminado
    public SelectUsuarios()
        throws SQLException, ClassNotFoundException {
        cargarConsultaUsuarios ();
        verResultado ();
    }

    private void cargarConsultaUsuarios ()
        throws SQLException, ClassNotFoundException{
        // En primer lugar establezco la conexión a la BBDD java
        String bbdd = "java";
        this.conexion = super.conexionBBDD(bbdd);
        // Uso el método ejecutaQuery de la Superclase
        String consulta = "SELECT * FROM usuarios";
        String mensaje = "Se ha ejecutado la consulta!";
        resultado = super.ejecutaQuery(consulta, mensaje);
    }

    private void verResultado () throws SQLException {
        cabecera();

        while (resultado.next()) {           // Mientras haya filas que leer
            ObjetoUsuario nuevoUsuario = new ObjetoUsuario();

            // Meto en cada atributo del objeto el campo
            // que devuelve el ResultSet
            nuevoUsuario.usuarioID = resultado.getInt("usuarioID");
            nuevoUsuario.nombre = resultado.getString("nombre");
            nuevoUsuario.clave = resultado.getString("clave");

            nuevoUsuario.edad = resultado.getInt("edad");
            nuevoUsuario.sexo = resultado.getBoolean("sexo");
            nuevoUsuario.altura = resultado.getFloat("altura");
            nuevoUsuario.peso = resultado.getFloat("peso");
```

```

// Presento los datos...
System.out.print (""+nuevoUsuario.usuarioID+"\t");
System.out.print (""+nuevoUsuario.nombre+"\t");
System.out.print (""+nuevoUsuario.clave+"\t");
System.out.print (""+nuevoUsuario.edad+"\t");
if (nuevoUsuario.sexo==true)
    System.out.print ("Mujer\t");
else
    System.out.print ("Hombre\t");
System.out.print (""+nuevoUsuario.altura+"\t");
System.out.print (""+nuevoUsuario.peso+"\t");
System.out.println ("");
}
}

```

```

private void cabecera () {
    System.out.print ("Id\t");
    System.out.print ("Usuario\t");
    System.out.print ("Clave\t");
    System.out.print ("Edad\t");
    System.out.print ("Sexo\t");
    System.out.print ("Altura\t");
    System.out.print ("Peso\t");
    System.out.println ("");
}
}

```

4) Por último, en la clase **Principal** es donde nos creamos un objeto de la clase anterior y arrancamos la aplicación:

```

package ej23_ejemplojdbc;
import java.sql.SQLException;
public class Principal {
    public static void main(String[] args)
        throws SQLException, ClassNotFoundException{
        new SelectUsuarios();
    }
}

```


6 APLICACIONES BASADAS EN ENTORNO GRÁFICO (SWING)

Hasta ahora todos los ejemplos presentados son via consola. Pero evidentemente las aplicaciones modernas no emplean esta interfaz para interactuar con las aplicaciones Java. Por un lado tenemos aplicaciones basadas en la web (con J2EE que no veremos en este manual) y por otro lado disponemos de aplicaciones de escritorio con sus botones, listas desplegables, menús, ventanas, etc para presentar y solicitar datos al usuario.

En el caso de J2SE disponemos de dos conjuntos de librerías que proporcionan una amplia variedad de componentes gráficos, así como todo el soporte necesario para gestionar la interacción con el usuario. Son AWT y SWING. Nosotros, en este manual, nos centraremos en el segundo, mas moderno, con mas funcionalidades y, por encima de todo, adaptable a distintas plataformas.

6.1 Swing

Swing es una biblioteca gráfica para Java. Incluye widgets para interfaz gráfica de usuario tales como cajas de texto, botones, desplegables y tablas. Es una extensión de la biblioteca AWT a la que complementa (no sustituye).

Arquitectura

Es un framework MVC para desarrollar interfaces gráficas para Java con independencia de la plataforma o sistema operativo. Sigue un simple modelo de programación por hilos, y posee las siguientes características principales:

- **Independencia de plataforma.** Es compatible con multitud de Sistemas operativos, desde Linux hasta MacOS.
- **Extensibilidad:** es una arquitectura altamente particionada: los programadores pueden proveer sus propias implementaciones modificadas para sobrescribir las creadas por defecto.
- **Personalizable:** dado el modelo de representación programático del framework de swing, el control permite representar diferentes estilos de apariencia **"look and feel"** (desde apariencia MacOS hasta apariencia Windows 7 pasando por apariencia GTK+, IBM UNIX o HP UX entre otros). Además, los usuarios pueden proveer su propia implementación de apariencia, que permitirá cambios uniformes en la apariencia existente en las aplicaciones Swing sin efectuar ningún cambio al código de aplicación.

Ventajas respecto a AWT

- El diseño en Java puro posee menos limitaciones de plataforma.
- El desarrollo de componentes Swing es más activo.
- Los componentes de Swing soportan más características e incluyen mas elementos.

A lo largo de este tema vamos a crearnos tres proyectos:

- **Ej24_ElementosSwing:** En este proyecto iremos viendo, uno por uno, los distintos elementos que componen la librería Swing con pequeños ejemplos.
- **Ej25_EventosSwing:** Aquí iremos implementando prácticas con los escuchadores.
- **Ej26_Equipo:** Por último, en este proyecto, veremos un ejemplo completo de aplicación de escritorio con Swing con gestión de bases de datos mediante JDBC.

6.2 Contenedores

En toda aplicación de entorno gráfico el conjunto de controles u objetos visuales que interactúan con el usuario están incluidos en un objeto de jerarquía superior llamado contenedor, cuya principal función es organizar la vista de la aplicación.

Swing (al igual que AWT) proporciona dos tipos principales de contenedores:

- **Ventanas:** Su aspecto es el de una ventana clásica de interfaz gráfica, con su barra de título y cuadro de control de maximizar, minimizar y cerrar. El usuario puede cambiarla de posición y tamaño.
- **Paneles:** son contenedores que no disponen de barra de título ni cuadro de control y su principal función es incluir otros elementos gráficos (botones, menús desplegables, etc) y darles un determinado posicionamiento. Suelen estar incluidos en ventanas o páginas web.

6.2.1 Creación de una ventana

Lo primero que debemos hacer es crear el contenedor principal que representa la vista de la aplicación: la ventana. Para ello usaremos las clases **javax.swing.JFrame** y **javax.swing.JDialog**.

El primero representa una ventana estándar y el segundo un cuadro de diálogo. Los pasos para crear una ventana (del tipo que sea) serán los siguientes:

1. **Creación del objeto.** En este caso, el tipo de ventana elegida (JFrame o JDialog). Para el primer caso podemos usar los siguientes constructores:
 - a. **Frame ():** Ventana sin título
 - b. **Frame (String s):** Crea una ventana con un título.
2. **Definir el tamaño y posición de la ventana.** Para determinar ambos podemos usar los siguientes métodos heredados de la clase Component:
 - a. **void setSize (int ancho, int alto).** Tamaño de la ventana (ancho y alto) en píxeles.
 - b. **void setLocation (int x, int y):** Posicionamiento en píxeles respecto a la esquina superior izquierda de la pantalla. El desplazamiento X es horizontal y el Y es el vertical.
 - c. **void setBounds (int x, int y, int ancho, int alto):** Define al mismo tiempo posición y tamaño del componente.
3. Visualizar la ventana. De forma predeterminada es invisible. Para verla debemos usar:
 - a. **void setVisible (boolean estado).** Si se establece a true se ve la ventana.
4. (Opcional) Controlar el cierre de la Ventana. De forma predeterminada la aplicación continúa corriendo a pesar de cerrar la ventana. Para acabarla al pulsar la [X] debemos usar el método **setDefaultCloseOperation** junto con la constante **EXIT_ON_CLOSE**.

Veamos un ejemplo completo: **Ej2401_HolaMundo**.

NOTA IMPORTANTE: Los próximos ejemplos estarán desarrollados en el mismo proyecto: **Ej24_ElementosSwing**. Como ya sabemos, no podemos tener más de una clase con main dentro del mismo proyecto (al probar no sabríamos cuál es el que realmente se inicia) Por todo ello es importante comentar los main de cada clase al acabar de probarla.

```
package ej24_elementosswing;
/**
 * <h1> Primer ejemplo Swing </h1>
 * @author Iván Rodríguez
 * Creado: 14 Noviembre 2014
 */
```

```

import javax.swing.          ;
import java.awt.FlowLayout;
public class Ej2401_HolaMundo {

    public static void main( String args[] ) {
        JFrame ventana = new JFrame ( "Ventana Hola Mundo");

        // Esta parte lo veremos mas adelante
        // Resumiendo: añadimos una etiqueta y lo añadimos al panel
        JLabel hola = new JLabel( "Hola Mundo!" );
        ventana.getContentPane().setLayout(new FlowLayout());
        ventana.getContentPane().add( hola,"Center" );

        ventana.setBounds(100, 100, 400, 250);
        ventana.setVisible(true);
        ventana.setDefaultCloseOperation
            (WindowConstants.EXIT_ON_CLOSE);
    }
}

```

6.2.2 Personalización de Ventanas

El código anterior es perfectamente válido, pero lo normal es no crear directamente en el main los objetos de la clase JFrame o JDialog. Es mas apropiado definir subclases que encapsulen en su interior el proceso de construcción de la ventana.

Veamos el mismo ejemplo **Ej2401_HolaMundo** reformado:

```

package ej24_elementosswing;
import javax.swing.*;
import java.awt.FlowLayout;
public class Ej2401_HolaMundo {

    public static void main( String args[] ) {
        // Por costumbre siempre uso este nombre ;)
        pintaVentana ventana =
            new pintaVentana ( "Ventana Hola Mundo",100, 100, 400, 250);
    }
}

class pintaVentana extends JFrame {
    // Defino un constructor con todos los parámetros
    public pintaVentana
        (String titulo, int x, int y, int ancho, int alto) {
        super(titulo); // Constructor del JFrame (String titulo)

        // Al heredar JFrame no tenemos que poner ventana.
        setBounds(x, y, ancho, alto);
    }
}

```

```

JLabel hola = new JLabel( "Hola Mundo!" );
getContentPane().setLayout(new FlowLayout());
getContentPane().add( hola,"Center" );

setVisible(true);
setDefaultCloseOperation (WindowConstants.EXIT_ON_CLOSE);
}
}

```

6.3 Uso de Layouts

En java, cuando hacemos ventanas, la clase que decide cómo se reparten los botones (Y demás controles) dentro de la ventana se llama Layout. Esta clase es la que decide en qué posición van los botones y demás componentes, si van alineados, en forma de matriz, cuáles se hacen grandes al agrandar la ventana, etc. Otra cosa importante que decide el Layout es qué tamaño es el ideal para la ventana en función de los componentes que lleva dentro.

Con un layout adecuado, el método pack() de la ventana hará que coja el tamaño necesario para que se vea todo lo que tiene dentro.

```
ventana.pack();
```

Las ventanas vienen con un Layout por defecto. En java hay varios layouts disponibles y podemos cambiar el de defecto por el que queramos.

6.3.1 Layout null

Uno de los Layouts más utilizados por la gente que empieza, por ser el más sencillo, es NO usar layout. Somos nosotros desde código los que decimos cada botón en qué posición va y qué tamaño ocupa

```

contenedor.setLayout(null); // Eliminamos el layout
contenedor.add (boton);      // Añadimos el botón
boton.setBounds (10,10,40,20); // Botón x10,y10, ancho 40px y alto 20px

```

Esto, aunque sencillo, no es recomendable. Si estiramos la ventana los componentes seguirán en su sitio, no se estirarán con la ventana. Si cambiamos de sistema operativo, resolución de pantalla o fuente de letra, tenemos casi asegurado que no se vean bien las cosas: etiquetas cortadas, letras que no caben, etc.

Además, al no haber layout, la ventana no tiene tamaño adecuado. Deberemos dárselo nosotros con un **ventana.setSize(...)**. Y si hacemos que sea un JPanel el que no tiene layout, para que este tenga un tamaño puede que incluso haga falta llamar a **panel.setPreferredSize(...)** o incluso en algunos casos, sobreescribiendo el método panel.getPreferredSize()

Evidentemente, para evitar problemas, debemos impedir el cambio de tamaño de la ventana, que se haría con el siguiente método:

```
ventana.setResizable(false);
```

El tiempo que ahorramos no aprendiendo cómo funcionan los Layouts, lo perderemos echando cuentas con los pixels, para conseguir las cosas donde queremos, sólo para un tipo de letra y un tamaño fijo.

6.3.2 Layout FlowLayout

El FlowLayout es bastante sencillo de usar. Nos coloca los componente en fila. Hace que todos quepan (si el tamaño de la ventana lo permite). Es adecuado para barras de herramientas, filas de botones, etc.

```
contenedor.setLayout(new FlowLayout());
contenedor.add(boton);
contenedor.add(textField);
contenedor.add(checkBox);
```

Veamos un ejemplo completo:

```
package ej02_flowlayout;

import java.awt.Container;           import java.awt.FlowLayout;
import javax.swing.JButton;    import javax.swing.JCheckBox;
import javax.swing.JFrame;           import javax.swing.JLabel;
import javax.swing.JTextField;       import javax.swing.WindowConstants;

public class Ej02_FlowLayout {
    // Atributos de los componentes privados
    private JFrame ventana;           private JButton boton;
    private JTextField campoTexto;    private JCheckBox selector;
    private JLabel etiqueta;          private Container panel;

    // El constructor por defecto
    public Ej02_FlowLayout() {
        pintaVentana ();
    }

    // El método pintaVentana
    private void pintaVentana () {
        ventana = new JFrame("Ejemplo FlowLayout");
        ventana.setLocation(200, 200);
        meteComponentes ();
        ventana.setVisible(true);
        ventana.setDefaultCloseOperation
            (WindowConstants.EXIT_ON_CLOSE);
    }

    private void meteComponentes () {
        // Definimos el panel y el layout
        panel = ventana.getContentPane();
        panel.setLayout(new FlowLayout());

        // Creamos los objetos de los componentes
        boton = new JButton("Botón");
        campoTexto = new JTextField(10);
        selector = new JCheckBox("CheckBox");
        etiqueta = new JLabel("Etiqueta");
```

```

        // Metemos los componentes en el panel
        panel.add(boton);
        panel.add(campoTexto);
        panel.add(selector);
        panel.add(etiqueta);

        // Empaquetamos los componentes
        ventana.pack();
    }

    public static void main(String[] args) {
        new Ej02_FlowLayout();
    }
}

```

6.3.3 Layout BorderLayout

Es como un FlowLayout, pero mucho más completo. Permite colocar los elementos en horizontal o vertical.

```

// Para poner en vertical
contenedor.setLayout(new BorderLayout(contenedor,BorderLayout.Y_AXIS));
contenedor.add(unBoton);
contenedor.add(unaEtiqueta);

```

Veamos un ejemplo completo:

```

import java.awt.Component;

import javax.swing.BoxLayout;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.WindowConstants;

public class PruebaBoxLayout {
    public static void main(String [] args)
    {
        // Se crea la ventana con el BorderLayout
        JFrame v = new JFrame();
        v.getContentPane().setLayout
            (new BorderLayout(v.getContentPane(),BorderLayout.Y_AXIS));

        // Se crea un botón centrado y se añade
        JButton boton = new JButton("B");
        boton.setAlignmentX(Component.CENTER_ALIGNMENT);
        v.getContentPane().add(boton);
    }
}

```

```

        // Se crea una etiqueta centrada y se añade
        JLabel etiqueta = new JLabel("una etiqueta larga");
        etiqueta.setAlignmentX(Component.CENTER_ALIGNMENT);
        v.getContentPane().add(etiqueta);

        // Visualizar la ventana
        v.pack();
        v.setVisible(true);
        v.setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
    }
}

```

BoxLayout.Y_AXIS es para que coloque los componentes en vertical, de arriba a abajo. Con **BoxLayout.X_AXIS** los coloca igual que un **FlowLayout**, de izquierda a derecha. También hay **BoxLayout.LINE_AXIS** e **BoxLayout.PAGE_AXIS**, pero el comportamiento por defecto es igual. El comportamiento es distinto si al contenedor le cambiamos el **setComponentOrientation()**, en cuyo caso podemos conseguir que los componentes vayan de derecha a izquierda según los vamos añadiendo o de abajo a arriba.

6.3.4 Layout GridLayout

Este pone los componentes en forma de matriz (cuadrícula), estirándolos para que tengan todos el mismo tamaño. El **GridLayout** es adecuado para hacer tableros, calculadoras en que todos los botones son iguales, etc.

```

// Creación de los botones
JButton boton[] = new JButton[9];
for (int i=0;i<9;i++)
    boton[i] = new JButton(Integer.toString(i));

// Colocación en el contenedor
contenedor.setLayout (new GridLayout (3,3)); // 3 filas y 3 columnas
for (int i=0;i<9;i++)
    contenedor.add (boton[i]); // Añade los botones de 1 en 1.

```

Aquí podemos ver un ejemplo completo de **GridLayout**:

```

import java.awt.GridLayout;

import javax.swing.JFrame;
import javax.swing.JTextField;
import javax.swing.WindowConstants;

public class PruebaGridLayout {
    private static final int COLUMNAS = 10;
    private static final int FILAS = 5;

```

```

public static void main(String [] args)
{
    JFrame v = new JFrame();
    v.getContentPane().setLayout(new GridLayout(FILAS,COLUMNAS));

    JTextField [][] textField = new JTextField [FILAS][COLUMNAS];
    for (int i=0;i<FILAS;i++)
        for (int j=0;j<COLUMNAS;j++)
        {
            textField[i][j] = new JTextField(1);
            v.getContentPane().add(textField[i][j]);
        }
    v.pack();
    v.setVisible(true);
    v.setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
}
}

```

6.3.5 Layout BorderLayout

El BorderLayout divide la ventana en 5 partes: centro, arriba, abajo, derecha e izquierda. Hará que los componentes que pongamos arriba y abajo ocupen el alto que necesiten, pero los estirará horizontalmente hasta ocupar toda la ventana.

Los componentes de derecha e izquierda ocuparán el ancho que necesiten, pero se les estirará en vertical hasta ocupar toda la ventana. El componente central se estirará en ambos sentidos hasta ocupar toda la ventana.

El BorderLayout es adecuado para ventanas en las que hay un componente central importante (una tabla, una lista, etc) y tiene menús o barras de herramientas situados arriba, abajo, a la derecha o a la izquierda.

Este es el layout por defecto para los JFrame y JDialog.

```

contenedor.setLayout (new BorderLayout());
contenedor.add (componenteCentrallImportante, BorderLayout.CENTER);
contenedor.add (barraHerramientasSuperior, BorderLayout.NORTH);
contenedor.add (botonesDeAbajo, BorderLayout.SOUTH);
contenedor.add (IndiceIzquierdo, BorderLayout.WEST);
contenedor.add (MenuDerecha, BorderLayout.EAST);

```

Por ejemplo, es bastante habitual usar un contenedor (JPanel por ejemplo) con un FlowLayout para hacer una fila de botones y luego colocar este JPanel en el NORTH de un BorderLayout de una ventana. De esta forma, tendremos en la parte de arriba de la ventana una fila de botones, como una barra de herramientas.

```

JPanel barraHerramientas = new JPanel();
barraHerramientas.setLayout(new FlowLayout());
barraHerramientas.add(new JButton("boton 1"));
...
barraHerramientas.add(new JButton("boton n"));

```



```

JFrame ventana = new JFrame();
ventana.getContentPane().setLayout(new BorderLayout()); // No hace falta, por
defecto ya es BorderLayout
ventana.getContentPane().add(barraHerramientas, BorderLayout.NORTH);
ventana.getContentPane().add(componentePrincipalDeVentana, BorderLayout.CENTER);

ventana.pack();
ventana.setVisible(true);

```

Veamos un ejemplo completo:

IMPORTANTE: Tanto para los JPanel como para los Container, **no es aplicable** el SetSize

Para definir tamaños, debemos introducir componentes en su interior.

NOTA: En este ejemplo se añaden botones y JPanel (que se ven mas adelante con detenimiento)

// Se han omitido las importaciones especificas...

```

import java.awt.*;
import javax.swing.*;

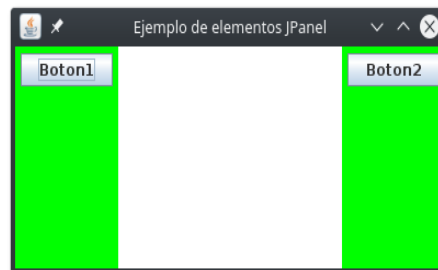
```

```

public class CompPanel {
    JFrame ventana;
    Container panelGeneral;

    public CompPanel() {
        pintaVentana();
    }

```



```

    public void pintaVentana() {
        ventana = new JFrame("Ejemplo de elementos JPanel");
        ventana.setBounds(200, 200, 400, 200);
        componentes();
        ventana.setVisible(true);
        ventana.setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
    }

```

```

    public void componentes() {
        panelGeneral = ventana.getContentPane();
        panelGeneral.setLayout(new BorderLayout());

```

```

        JPanel panelOeste = new JPanel();
        panelOeste.setBackground(Color.GREEN);
        JButton boton = new JButton("Boton1");
        panelOeste.setLayout(new FlowLayout());
        panelOeste.add(boton);
        panelGeneral.add(panelOeste, BorderLayout.WEST);

```

```

        JPanel panelCentro = new JPanel();
        panelCentro.setBackground(Color.WHITE);
        panelGeneral.add(panelCentro, BorderLayout.CENTER);

```

```

    JPanel panelEste = new JPanel();
    panelEste.setBackground(Color.GREEN);
    JButton boton2 = new JButton("Boton2");
    panelEste.setLayout(new FlowLayout());
    panelEste.add(boton2);
    panelGeneral.add(panelEste, BorderLayout.EAST);
}

public static void main(String[] args) {
    CompPanel ejemplo = new CompPanel();
}
}

```

6.3.6 GridBagLayout

Es mucho mas flexible que el GridLayout. Permite, empleando un solo layout, realizar maquetaciones como esta:

Veamos un ejemplo, una calculadora:

```

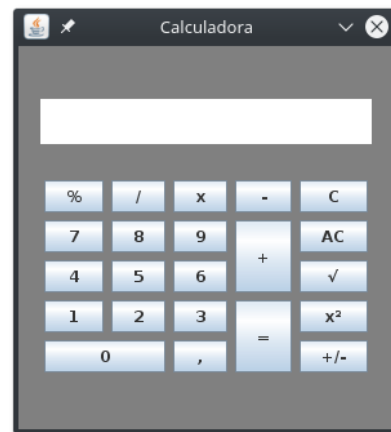
public class Calculadora extends JFrame {
    public Calculadora() {
        pintaVentana();
    }

    public void pintaVentana() {
        Container panel = this.getContentPane();
        panel.setLayout(new GridBagLayout()); // Establecemos el GridBagLayout
        componentes (panel);
        this.setBounds(100, 100, 300, 330);
        this.setResizable(false);
        this.setVisible(true);
        this.setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
    }

    public void componentes (Container panel) {
        // Instanciamos el GridBagConstraint que contiene todos los metodos de
        // GridBagLayout, y en donde indicamos los parámetros "esteticos" de los
        // diferentes elementos que contiene el JFrame
        GridBagConstraints gbc = new GridBagConstraints();

        // Declaracion de los botones
        JButton porcentaje = new JButton("%");
        JButton sumar = new JButton("+");
        JButton restar = new JButton("-");
        JButton multiplicar = new JButton("x");
        JButton division = new JButton("/");
        JButton igual = new JButton("=");
        JButton limpiar = new JButton("C");
        JButton limpiarTodo = new JButton("AC");
    }
}

```



```

// Para obtener el simbolo de la raiz, usamos su carácter ASCII
String raiz = Character.toString ((char)8730);
JButton parentesisInicio = new JButton(raiz);

String cuadrado = Character.toString ((char)0x00B2);
JButton parentesisFin = new JButton("x"+cuadrado);
JButton cambiarSigno = new JButton("+/- ");
JButton coma = new JButton(",");

JButton[] numeros = new JButton[10];
for (int i = 0; i < 10; i++)
    numeros[i] = new JButton(String.valueOf(i));

JTextArea areaTexto = new JTextArea();
areaTexto.setEditable(false);
gbc.ipady = 20;
gbc.gridx = 0; //El area de texto empieza en la columna cero
gbc.gridy = 0; //El area de texto empieza en la fila cero
gbc.gridwidth = 5; //Area texto ocupa TODA la fila (Total=5 Columnas)
gbc.gridheight = 1; // Altura 1 Celda

// Elemento interior ocupa TODO el espacio de la celda
gbc.fill = GridBagConstraints.BOTH;
gbc.insets = new Insets(0, 0, 25, 0); // Margenes (top, right...)
panel.add(areaTexto, gbc);
gbc.ipady = 0;

// Parametros generales de todos los JBUTTON
gbc.insets = new Insets(3, 3, 3, 3); // Margenes
gbc.gridheight = 1; // Ocupan 1x1 celda
gbc.gridwidth = 1;

// PRIMERA FILA
gbc.gridy = 1;
gbc.gridx = 0;    panel.add(porcentaje, gbc);
gbc.gridx = 1;    panel.add(division, gbc);
gbc.gridx = 2;    panel.add(multiplicar, gbc);
gbc.gridx = 3;    panel.add(restar, gbc);
gbc.gridx = 4;    panel.add(limpiar, gbc);

// SEGUNDA FILA
gbc.gridy = 2;
gbc.gridx = 0;    panel.add(numeros[7], gbc);
gbc.gridx = 1;    panel.add(numeros[8], gbc);
gbc.gridx = 2;    panel.add(numeros[9], gbc);
// Cambiamos el numero de "celdas" que ocupa la tecla sumar
gbc.gridheight = 2;
gbc.gridx = 3;    panel.add(sumar, gbc);

```

```

// VALORES POR DEFECTO
gbc.gridheight = 1;
gbc.gridx = 4;    panel.add(limpiarTodo, gbc);

// TERCERA FILA
gbc.gridy = 3;
gbc.gridx = 0;    panel.add(numeros[4], gbc);
gbc.gridx = 1;    panel.add(numeros[5], gbc);
gbc.gridx = 2;    panel.add(numeros[6], gbc);
gbc.gridx = 4;    panel.add(parentesisInicio, gbc);

// CUARTA FILA
gbc.gridy = 4;
gbc.gridx = 0;    panel.add(numeros[1], gbc);
gbc.gridx = 1;    panel.add(numeros[2], gbc);
gbc.gridx = 2;    panel.add(numeros[3], gbc);

// Cambiamos los parámetros para que ocupe dos "celdas"
// como en el caso de suma
gbc.gridheight = 2;
gbc.gridx = 3;    panel.add(igual, gbc);

// VALORES POR DEFECTO
gbc.gridheight = 1;
gbc.gridx = 4;    panel.add(parentesisFin, gbc);

// QUINTA FILA
gbc.gridy = 5;
// En el caso de la tecla 0 ocupa dos "celdas" horizontales
gbc.gridwidth = 2;
gbc.gridx = 0;    panel.add(numeros[0], gbc);

// VALORES POR DEFECTO
gbc.gridwidth = 1;
gbc.gridx = 2;    panel.add(coma, gbc);
gbc.gridx = 4;    panel.add(cambiarSigno, gbc);

// Configuración general de la ventana
panel.setBackground(Color.GRAY);
}

public static void main(String[] args) {
    Calculadora calculadora = new Calculadora();
}
}

```

6.4 Agregar controles a un contenedor

El paquete `javax.swing` incluye una gran variedad de clases para la creación de los elementos gráficos mas comunes que podemos emplear en nuestras aplicaciones. Todas estas clases son subclases de `Component`. En los siguientes subapartados veremos uno por uno cada uno de estos componentes.

6.4.1 JDialog

Representa un cuadro de diálogo que funciona como una ventana especializada para realizar operaciones complejas. Sus constructores son:

JDialog() → Crea una ventana con la configuración normal.

JDialog(Frame propietaria) → Crea un nuevo cuadro de diálogo, indicando como padre la ventana seleccionada

JDialog(Frame propietaria, boolean modal) → Crea un nuevo cuadro de diálogo, indicando como padre la ventana seleccionada. Poniendo a `true` el segundo parámetro, el cuadro de diálogo pasa a ser modal. Una ventana modal obliga a el usuario a contestar al cuadro antes de que pueda continuar trabajando.

JDialog(Frame propietaria, String título) → Crea un cuadro de diálogo perteneciente a la ventana indicada y poniendo el título deseado.

JDialog(Frame propietaria, String título, boolean modal) → Crea un cuadro de diálogo perteneciente a la ventana indicada, poniendo el título deseado e indicando si se desea el cuadro en forma modal.

Métodos mas importantes de `JDialog`:

void toBack() → Coloca la ventana al fondo, el resto de ventanas aparecen por delante

void toFront() → Envía la ventana al frente (además le dará el foco).

void setTitle(String t) → Cambia el título de la ventana

String getTitle() → Obtiene el título de la página

void setResizable(boolean b) → Con `true` la ventana es cambiabile de tamaño, en `false` la ventana tiene tamaño fijo.

void pack() → Ajusta la ventana a tamaño suficiente para ajustar sus contenidos

setLocationRelativeTo(Componente c) → Determina la localización del elemento en función de un componente padre. Si le ponemos `null` se centrará en pantalla.

6.4.2 JLabel, JButton

Para las etiquetas usaremos `JLabel` y para los botones `Jbutton`.

Para ambos casos, tenemos los siguientes métodos:

setText (String mensaje) → Texto en la etiqueta / Botón

setFont (Font fuente) → Fuente del texto

setBounds (ya visto)

setColumns (int numColumnas) → Número de columnas para `JtextField`

Veamos un ejemplo con todo lo anterior:

```
import java.awt.Font;
```

```
import javax.swing.JFrame;
```

```
import javax.swing.JTextField;
```

```
import javax.swing.JPanel;
```

```
import javax.swing.JButton;
```

```
public class CuadrosDialogo{  
    private JFrame ventana;  
    private JPanel panel;  
    private JTextField etiqueta;
```

```

private JTextField campoTexto;
private JButton botonError;
public CuadrosDialogo() {
    pintaVentana ();
}

    private void pintaVentana () {
        ventana = new JFrame ("Ejemplos Cuadros Diálogo");
        ventana.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        ventana.setBounds(200, 200, 800, 600);
        ventana.setFont(new Font("Ubuntu", Font.BOLD, 12));
        panel = new JPanel();
        panel.setLayout(null);
        ventana.setContentPane(panel);
        // Ojo, los componentes van dentro de pintaVentana,
        componentes ();
        ventana.setVisible(true);
    }

public void componentes () {
    etiquetas ();
    botones();
}

public void etiquetas () {
    etiqueta = new JTextField();
    etiqueta.setText("Etiqueta");
    etiqueta.setFont(new Font("Ubuntu", Font.BOLD, 20));
    etiqueta.setBounds(150, 59, 330, 60);
    panel.add(etiqueta);
    etiqueta.setColumns(10);
    campoTexto = new JTextField();
    campoTexto.setFont(new Font("Ubuntu", Font.BOLD, 20));
    campoTexto.setText("Campo Texto");
    campoTexto.setBounds(150, 146, 330, 48);
    panel.add(campoTexto);
    campoTexto.setColumns(10);
}

public void botones () {
    botonError = new JButton("Error");
    botonError.setFont(new Font("Ubuntu", Font.BOLD, 20));
    botonError.setBounds(34, 206, 189, 36);
    panel.add(botonError);
}

public static void main(String[] args) {
    new CuadrosDialogo();
}
}

```

6.4.3 JTextField, JScrollPane y JTextArea

El primero nos sirve para crear campos de texto de una línea, el segundo otro con barra de desplazamiento y el último un campo de texto no adaptable de varias líneas. Como podemos observar, debemos introducir un JTextArea dentro de un JScrollPane para que tenga la barra.

Los métodos son los mismos a los vistos anteriormente. Veamos un ejemplo con ambos:

```
import javax.swing.*;

public class EjemploJscrollPane extends JFrame{
    private JTextField textfield1;
    private JScrollPane scrollpane1;           private JTextArea textarea1;
    public EjemploJscrollPane () {
        setLayout(null);
        textfield1=new JTextField();
        textfield1.setBounds(10,10,200,30);    add(textfield1);
        textarea1=new JTextArea();
        scrollpane1=new JScrollPane(textarea1);
        scrollpane1.setBounds(10,50,400,300);  add(scrollpane1);
    }
    public static void main(String[] ar) {
        EjemploJscrollPane ejemplo1=new EjemploJscrollPane ();
        ejemplo1.setBounds(0,0,540,400);
        ejemplo1.setVisible(true);
    }
}
```

6.4.4 JComboBox

El JComboBox es, en esencia, una lista desplegable. Los métodos son los mismos a los vistos anteriormente, con el añadido de addItem, que nos permite introducir elementos en la lista.

En el siguiente ejemplo hemos incluido un evento de selección de ítem, muy sencillo, que nos servirá de introducción al apartado siguiente:

```
import javax.swing.*;
import java.awt.event.*;

public class EjCombo extends JFrame implements ItemListener{
    private JComboBox combo1;
    public EjCombo() {
        setLayout(null);
        combo1=new JComboBox();
        combo1.setBounds(10,10,80,20);
        add(combo1);
        combo1.addItem("rojo");
        combo1.addItem("verde");
        combo1.addItem("azul");
        combo1.addItem("amarillo");
        combo1.addItem("negro");
        combo1.addItemListener(this);
    }
}
```

```

public void itemStateChanged(ItemEvent e) {
    if (e.getSource()==combo1) {
        String seleccionado=(String)combo1.getSelectedItem();
        setTitle(seleccionado);
    }
}
public static void main(String[] ar) {
    EjCombo combo1=new EjCombo();
    combo1.setBounds(0,0,200,150);
    combo1.setVisible(true);
}
}

```

6.4.5 JCheckBox

Los checkbox se comportan de manera independiente y pueden elegirse varios a la vez. De nuevo, los métodos son muy parecidos a los anteriormente vistos, aunque eso si, los listener deben ser aplicados a cada uno de ellos:

```

public class EjCheckBox extends JFrame implements ChangeListener{
    private JCheckBox check1,check2,check3;
    public EjCheckBox() {
        setLayout(null);
        check1=new JCheckBox("Inglés");
        check1.setBounds(10,10,150,30);
        check1.addChangeListener(this);
        add(check1);
        check2=new JCheckBox("Francés");
        check2.setBounds(10,50,150,30);
        check2.addChangeListener(this);
        add(check2);
        check3=new JCheckBox("Alemán");
        check3.setBounds(10,90,150,30);
        check3.addChangeListener(this);
        add(check3);
    }
    public void stateChanged(ChangeEvent e){
        String cad="";
        if (check1.isSelected()==true) {
            cad=cad+"Inglés ";
        }
        if (check2.isSelected()==true) {
            cad=cad+"Francés";
        }
        if (check3.isSelected()==true) {
            cad=cad+"Alemán ";
        }
        setTitle(cad);
    }
}

```



```

    public static void main(String[] ar) {
        EjCheckBox ej1=new EjCheckBox();
        ej1.setBounds(0,0,300,200);
        ej1.setVisible(true);
    }
}

```

Veamos el ejemplo anterior pero mas modularizado...

```

package componentes;
import java.awt.Color;
import java.awt.FlowLayout;
import javax.swing.BoxLayout;
import javax.swing.JFrame;
import javax.swing.event.ChangeEvent;
import javax.swing.event.ChangeListener;

import java.awt.Container;
import java.awt.Font;
import javax.swing.JCheckBox;

public class CompJCheckBox
    extends JFrame implements ChangeListener {

    // Atributos de la clase
    Container panelGeneral;

    public CompJCheckBox() {
        this.setTitle("Ejemplo de CheckBox");
        pintaVentana();
    }

    public void pintaVentana() {
        panelGeneral = this.getContentPane();
        panelGeneral.setLayout(new BoxLayout(panelGeneral, BoxLayout.Y_AXIS));
        componentes(panelGeneral);
        this.setBounds(200, 200, 400, 300);
        this.setVisible(true);
    }

    public void componentes(Container panelGeneral) {
        Font fuenteCheck = new Font("Times New Roman", Font.BOLD, 20);
        creaCheckBox(panelGeneral, "Inglés", new Color(153, 204, 255),
            Color.black, fuenteCheck);
        creaCheckBox(panelGeneral, "Francés", new Color(0xFFCCCC),
            Color.black, fuenteCheck);
        creaCheckBox(panelGeneral, "Alemán",
            Color.yellow, Color.red, fuenteCheck);
    }

    public void creaCheckBox(Container panel, String titulo,
        Color colorFondo, Color colorFuente, Font nuevaFuente) {
        JCheckBox check = new JCheckBox(titulo);
    }
}

```

```

        check.setBackground(colorFondo);
        check.setForeground(colorFuente);
        check.setFont(nuevaFuente);
        panel.add(check);

        // Añadimos los listener aquí. This se refiere al propio check
        check.addChangeListener(this);
    }

    // NOTA: Cada check creado tiene su PROPIO listener (stateChanged)
    @Override
    public void stateChanged(ChangeEvent evento) {
        JCheckBox check = (JCheckBox) evento.getSource();
        String textoCheck = check.getText();
        if (check.isSelected()) {
            switch (textoCheck) {
                case "Alemán": panelGeneral.setBackground(Color.red); break;
                case "Inglés": panelGeneral.setBackground(Color.green); break;
                case "Francés": panelGeneral.setBackground(Color.blue); break;
            }
        }
        if (!check.isSelected()) {
            panelGeneral.setBackground(Color.white);
        }
    }

    public static void main(String[] args) {
        new CompJCheckBox();
    }
}

```

6.4.6 JRadioButton.

Los JRadioButton funcionan igual que los checkbox, pero deben incluirse en un ButtonGroup.

Veamos un ejemplo de su uso:

```
import javax.swing.*;
import javax.swing.event.*;
public class EjRadio extends JFrame implements ChangeListener{
    private JRadioButton radio1,radio2,radio3;
    private ButtonGroup bg;
    public EjRadio() {
        setLayout(null);
        bg=new ButtonGroup();
        radio1=new JRadioButton("640*480");
        radio1.setBounds(10,20,100,30);
        radio1.addChangeListener(this);
        add(radio1);
        bg.add(radio1);
        radio2=new JRadioButton("800*600");
        radio2.setBounds(10,70,100,30);
        radio2.addChangeListener(this);
        add(radio2);
        bg.add(radio2);
        radio3=new JRadioButton("1024*768");
        radio3.setBounds(10,120,100,30);
        radio3.addChangeListener(this);
        add(radio3);
        bg.add(radio3);
    }
    public void stateChanged(ChangeEvent e) {
        if (radio1.isSelected()) {           setSize(640,480); }
        if (radio2.isSelected()) {           setSize(800,600); }
        if (radio3.isSelected()) {           setSize(1024,768);      }
    }

    public static void main(String[] ar) {
        EjRadio ej1=new EjRadio();
        ej1.setBounds(0,0,350,230);
        ej1.setVisible(true);
    }
}
```

6.4.7 JMenuBar, JMenu y JMenuItem

El primero sirve para incluir barras de herramientas (y dentro sus correspondientes botones). El segundo es para meter menús (los clásicos Archivo, Edición, Ayuda) y el tercero cada uno de los elementos de esos menús (Acerca de, Abrir, Guardar).

Veamos un ejemplo:

```
import java.awt.Color;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

import javax.swing.JFrame;
import javax.swing.JMenu;
import javax.swing.JMenuBar;
import javax.swing.JMenuItem;
import javax.swing.JSeparator;

// En la clase usaré la interfaz ActionListener
public class MenuColores implements ActionListener{

    // Nos creamos los componentes de los Menús y la ventana
    private JMenuBar barraMenus;
    private JMenu menuColores, menuAyuda;
    private JMenuItem opcion1, opcion2, opcion3, opcion4;
    private JSeparator separador;
    JFrame ventana = new JFrame ();

    // Creo el método pintaVentana ()
    public void pintaVentana () {
        ventana.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        ventana.setSize(400, 300);
        ventana.setLocation(200, 200);
        ventana.getContentPane().setBackground
            (new Color(127,255,212));

        // Implementamos los componentes
        barraMenus = new JMenuBar();
        menuColores = new JMenu("Colores");
        menuAyuda = new JMenu ("Ayuda");
        opcion1 = new JMenuItem ("Rojo");
        opcion2 = new JMenuItem ("Verde");
        opcion3 = new JMenuItem ("Azul");
        opcion4 = new JMenuItem ("Salir");
        separador = new JSeparator();
        // Añado los menus a la barra de Menús
        barraMenus.setSize(400, 30);
        barraMenus.add(menuColores);
        barraMenus.add(menuAyuda);
```

```

// Añado cada elemento de Menú a su menú correspondiente
menuColores.add(opcion1);           menuColores.add(opcion2);
menuColores.add(opcion3);           menuColores.add(separador);
menuColores.add(opcion4);

// Asignamos los Listener a los componentes
opcion1.addActionListener(this);
opcion2.addActionListener(this);
opcion3.addActionListener(this);
opcion4.addActionListener(this);

// Por último añadimos la barra de Menús a la ventana...
ventana.setLayout(null);
ventana.add(barraMenus);
ventana.setVisible(true);
}

// Método para controlar el Listener
public void actionPerformed(ActionEvent evento) {
    if (evento.getSource() == opcion1)
        ventana.getContentPane().
            setBackground(new Color (255,0,0));
    if (evento.getSource() == opcion2)
        ventana.getContentPane().
            setBackground(new Color (0,255,0));
    if (evento.getSource() == opcion3)
        ventana.getContentPane().
            setBackground(new Color (0,0,255));

    // Si elijo salir, cierro la ventana
    if (evento.getSource() == opcion4)
        System.exit(0);
}

public static void main(String[] args) {
    MenuColores nuevaVentana = new MenuColores();
    nuevaVentana.pintaVentana();
}
}

```

6.4.8 Pestañas (JTabbedPane)

Para crear pestañas, debemos crear previamente tantos paneles como pestañas queramos introducir. Finalmente, las agregaremos todas a un contenedor específico de tipo **JTabbedPane**.

Veamos un ejemplo completo:

```
package misComponentes;
```

```
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JTabbedPane;
```

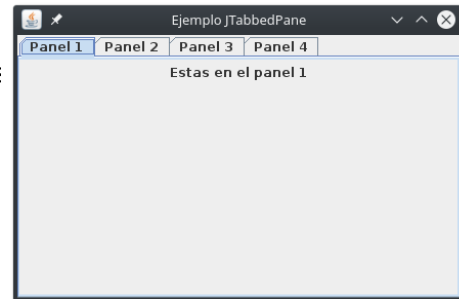
```
/**
 *
 * @author forinemas
 */
public class EjemploTabs extends JFrame {

    public EjemploTabs() {
        super("Ejemplo JTabbedPane");
        pintaVentana();
    }

    public void pintaVentana() {
        //Parametros asociados a la ventana
        meterPestañas();
        setSize(640, 480);
        // La ventana, por defecto, sale maximizada
        setExtendedState(MAXIMIZED_BOTH);
        setVisible(true);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }

    public void meterPestañas() {
        //Creamos el conjunto de pestañas
        JTabbedPane pestañas = new JTabbedPane();

        //Creamos el panel y lo añadimos a las pestañas
        JPanel panel1 = new JPanel();
        //Componentes del panel1
        JLabel et_p1 = new JLabel("Estas en el panel 1");
        panel1.add(et_p1);
        //Añadimos un nombre de la pestaña y el panel
        pestañas.addTab("Panel 1", panel1);
    }
}
```



```

//Realizamos lo mismo con el resto
JPanel panel2 = new JPanel();
pestañas.addTab("Panel 2", panel2);
//Componentes del panel2
JLabel et_p2 = new JLabel("Estas en el panel 2");
panel2.add(et_p2);

JPanel panel3 = new JPanel();
//Componentes del panel3
JLabel et_p3 = new JLabel("Estas en el panel 3");
panel3.add(et_p3);
pestañas.addTab("Panel 3", panel3);

JPanel panel4 = new JPanel();
//Componentes del panel4
JLabel et_p4 = new JLabel("Estas en el panel 4");
panel4.add(et_p4);
pestañas.addTab("Panel 4", panel4);

// Finalmente, agregamos el JTabbedPane al contenedor general
getContentPane().add(pestañas);
}

public static void main(String[] args) {
    EjemploTabs nuevo = new EjemploTabs();
}
}

```

EJERCICIO:

Debemos intentar crear una interfaz gráfica como la siguiente:

NOTA: Debemos usar varios GIFS que podemos encontrar en la carpeta de xampp
La solución se encuentra en el **ANEXO VII**



6.5 El modelo de gestión de Eventos en Java

La plataforma Swing se basa primordialmente para el manejo de eventos en el concepto de escuchado o Listener; la teoría general establece que existe un objeto al que llamaremos disparador que realiza alguna acción sobre otro objeto reactivo. El objeto entonces informa a varios Listener de que algo ha ocurrido en el disparador. Entonces cada uno de los listeners puede realizar alguna acción en base al evento que ocurrió.

Entre los componentes Swing hay una amplia gama de Listener's que se implementan como Interfaces, si queremos que determinada clase responda a los eventos de un objeto es necesario que este implemente la interface Listener adecuada y que se registre con el objeto que genera los reportes de evento. El ejemplo más sencillo es el de los botones comunes y la interfaz ActionListener.

```

JButton boton = new JButton("El boton");
//ActionListener es la clase encargada de escuchar eventos de acción
ActionListener al = new ActionListener() {
    public void actionPerformed(ActionEvent ae) {
        System.out.println("Se ha hecho click en el botón");
    }
}
boton.addActionListener(al);

//Registramos el nuevo objeto como escucha del botón
JButton boton2 = new JButton("Otro boton");
//Podemos agregar un Listener a varios elementos
boton2.addActionListener(al);
//E igualmente podemos registrar mas de un listener por objeto
boton.addActionListener(otro_listener);

//Borramos el registro de escucha cuando deja de interesarnos
boton.removeActionListener(al);
```

Algunos elementos pueden llegar a tener multiples interfaces para utilizar Listeners, como el caso de los JFrame's, que permiten recibir WindowListener, FrameListener, PropertyChangeListener, MouseListener, MouseMotionListener entre otros. En la documentación de Java pueden encontrarse bastantes referencias a los listeners que acepta cada uno de los componente Swing.

Existen casos de Listeners que declaran muchos métodos, como el caso del WindowListener que implementa windowClosed, windowClosing, windowIconified, y otros muchos, estos métodos deben cubrirse completamente en cualquier clase que implemente la interfaz, pero para facilitar las cosas por cada una de estas clases existe una clase Adapter que implementa una funcionalidad vacia en todos estos métodos.

En el **ANEXO III** figuran muchos de los conceptos vistos en este capítulo y algunos métodos nuevos que sería bueno estudiar. Además, se han empleado varios listener entre los que destacan los siguientes:

A) `MouseListener`, en concreto su método `addMouseListener` para conocer la fila y columna pulsada en la tabla de presentación de los equipos en `VerEquipos.java`:

// El listener de la tabla (MUY IMPORTANTE!!)

```
tabla.addMouseListener(new MouseAdapter() {
    public void mouseClicked (MouseEvent evento) {
        int fila = tabla.rowAtPoint(evento.getPoint());
        int columna = tabla.columnAtPoint(evento.getPoint());

        // Sabiendo la Fila y la columna presento el contenido
        System.out.println(modelo.getValueAt(fila, columna));
    }
});
```

Obsérvese que se agrega el listener a la tabla y se abre un paréntesis que no se cierra hasta el final (está puesto en **negrita**). Algo similar sucede con el siguiente listener.

B) Listener básico: `ActionListener` con el método `addActionListener`

En este caso aplicado a un botón.

// Definimos el listener para el botón

```
botonInsertar.addActionListener(new ActionListener () {
    public void actionPerformed(ActionEvent e)
    {
        try {
            insertarJugador();
        } catch (ClassNotFoundException error) {}
    }
});
```

7 EXCEPCIONES, HILOS

7.1 Excepciones

Las excepciones, tanto propias como las definidas por el propio API de Java, nos van a servir para controlar cualquier tipo de error que se produzca en el programa. Debemos diferenciar dos tipos claros de excepciones:

- a) Aquellas que paralizan el programa
- b) Aquellas que presentan un mensaje de error y permiten el resto de la ejecución.

Una vez vista esta primera clasificación debemos observar 2 tipos de excepciones en tiempo de ejecución:

- a) Aquellas que se controlan localmente mediante un **try... catch** (definiendo cada uno de los posibles errores y como controlarlos)
- b) Aquellas que “lanzan y propagan” la excepción, mediante la cláusula **throws**.

Todas las excepciones heredan de Throwable, y se pueden clasificar en los siguientes tipos:

Exception → General, del cual heredan todas las excepciones.

ArithmeticException → Error aritmético, como la división entre 0.

NullPointerException → Error de puntero nulo (el objeto no se ha definido)

ClassCastException → Error por intentar convertir un objeto en otro no válido.

ArrayIndexOutOfBoundsException → Error por exceder del tamaño del array

En el presente ejemplo completo he creado una excepción propia, que será lo mas usual en nuestro trabajo como desarrolladores, además de controlar los posibles errores y dejar que la aplicación siga funcionando.

NOTA: Debemos importar **Consola.java**, que se encuentra en el **ANEXO I**

MiExcepcion.java

```
package excepciones;
```

```
/**
 * @author ilván Rodríguez
 * @version 1.0 2016 09 27
 */
public class MiExcepcion extends Exception{
    public MiExcepcion(String mensaje) {
        super(mensaje);
    }
}
```

Y ahora nos toca emplear esta excepción propia:

```
package excepciones;
import static auxiliares.Consola.*;
import java.util.InputMismatchException;
```

```
/**
 * @author ilván Rodríguez
 * @version 1.0 2016 09 27
 */
```

```

public class MiClase {
    static void verificarPorcentaje(int num)throws MiExcepcion{
        if((num>100)||((num<0))){
            throw new MiExcepcion("Números fuera de rango");
        }
    }

    public static void introducePorcentajes () {
        int porcentaje;
        try {
            porcentaje =leeEntero("Escriba porcentaje de aciertos");
            verificarPorcentaje(porcentaje);
        }catch(InputMismatchException error){
            imprimeMensaje("Se han introducido caracteres no numéricos");
            porcentaje = 0;
        }catch(MiExcepcion error){
            porcentaje = 0;
            imprimeMensaje(error.getMessage());
        }
        imprimeMensaje("\n El Porcentaje guardado es:"+porcentaje);
        menu();
    }

    public static void menu() {
        int opcion = 0;
        imprimeMensaje("\n Programa Introducir Porcentajes ");
        imprimeMensaje(" 1. Introducir valor ");
        imprimeMensaje(" 2. Salir ");
        opcion = leeEntero(" Escriba opción: ");
        switch (opcion) {
            case 1: introducePorcentajes (); break;
            default: imprimeMensaje("Salida Programa");
        }
    }

    public static void main(String[] args) {
        menu();
    }
}

```

7.2 HILOS

Para manejar distintas ejecuciones al mismo tiempo emplearemos Hilos (**Threads**).

Para ello, tenemos dos opciones:

- a) Implementar la interfaz **Runnable**
- b) Heredar de **Thread**

En ambos casos, para definir que es lo que realizará el hilo usaremos el método **run()**.

Además, tenemos los siguientes métodos:

- **start()** → Inicia el hilo
- **setPriority(int)** → Asigna prioridad a los hilos
- **getPriority()** → Devuelve la prioridad del hilo
- **setName(String)** → Asigna un nombre al hilo

Veamos un ejemplo completo (son 3 clases, MiHilo1, MiHilo2, MiClase)

```
public class MiHilo1 extends Thread {  
  
    String mensaje;  
  
    public MiHilo1(String mensaje) {  
        super(mensaje);  
    }  
  
    public void run() {  
        mensaje = "Inicia el "+this.getName();  
        for (int i = 0; i < 5; i++) {  
            System.out.println(mensaje);  
            try {  
                Thread.sleep(1000);  
                setMensaje("Mensaje del Hilo 1 > "+(i+1));  
            } catch (InterruptedException ex) {}  
        }  
        System.out.println("Este proceso ha terminado:" + this.getName());  
    }  
  
    public void setMensaje(String mensaje) {  
        this.mensaje = mensaje;  
    }  
}
```

MiHilo2 es prácticamente igual...

```
public class MiHilo2 extends Thread{  
  
    String mensaje;  
  
    public MiHilo2(String mensaje) {  
        super(mensaje);  
    }  
  
    public void run() {
```

```

    mensaje = "Inicia el "+this.getName();
    for (int i = 0; i < 5; i++) {
        System.out.println(mensaje);
        try {
            Thread.sleep(3000);
            setMensaje("Este es el mensaje del Hilo 2 > "+(i+1));
        } catch (InterruptedException ex) {}
    }
    System.out.println("Este proceso ha terminado:" + this.getName());
}

```

```

public void setMensaje(String mensaje) {
    this.mensaje = mensaje;
}
}

```

Por último, la clase con el Main

```

public class MiClase {

    static public void main(String args[]) {
        MiHilo1 hilo1 = new MiHilo1("Hilo 1");
        MiHilo2 hilo2 = new MiHilo2("Hilo 2");

        hilo1.start();
        hilo2.start();
    }
}

```

ANEXO 1: Consola

Para la lectura e impresión de datos para las aplicaciones, vamos a emplear una clase especial que emplea Scanner. Este es su código al completo:

```
import java.util.Scanner;
/** <h3>Clase para gestionar métodos de Entrada/Salida</h3>
 * @version 1.0 2014 11 19
 * @author Iván Rodríguez
 */
import java.util.Scanner;
public class Consola {
    // Creamos una instancia de la clase Scanner
    public Scanner teclado = new Scanner (System.in);

    //      METODOS DE IMPRESIÓN      //
    public void imprimeMensaje (String mensaje) {
        System.out.println(mensaje);
    }

    //      METODOS DE LECTURA      //
    public int leeEntero (String mensaje) {
        imprimeMensaje(mensaje);
        int entero = teclado.nextInt();
        return entero;
    }

    public long leeLong (String mensaje) {
        imprimeMensaje(mensaje);
        long enteroLargo = teclado.nextLong();
        return enteroLargo;
    }

    public char leeCaracter (String mensaje) {
        imprimeMensaje(mensaje);
        char caracter = teclado.next().charAt(0);
        return caracter;
    }

    public String leeCadena (String mensaje) {
        imprimeMensaje(mensaje);
        String cadena = teclado.nextLine();
        return cadena;
    }

    public float leeFloat (String mensaje) {
        imprimeMensaje(mensaje);
        float flotante = teclado.nextFloat();
        return flotante;
    }
}
```

```
public double leeDouble (String mensaje) {  
    imprimeMensaje(mensaje);  
    double decimal = teclado.nextDouble();  
    return decimal;  
}  
  
public boolean leeBooleano (String mensaje) {  
    imprimeMensaje(mensaje);  
    boolean booleano = teclado.nextBoolean();  
    return booleano;  
}  
}
```

ANEXO 2: Base de Datos (JDBC y Swing)

Para el acceso a Bases de datos con JDBC y Swing, vamos a usar una base de datos llamada java con una única tabla llamada usuarios y 3 registros. Las sentencias para crearla son:

NOTA: Estas sentencias pueden ser agregadas mediante:

- Consola
- Workbench
- PHPMyAdmin

1) Creación de la Base de datos

```
CREATE DATABASE java;
```

2) Creación de la tabla usuarios

```
CREATE TABLE usuarios
(
    usuarioID TINYINT NOT NULL AUTO_INCREMENT,
    nombre    VARCHAR (30) NOT NULL,
    clave     VARCHAR (30) NOT NULL,
    edad      INT (2) NOT NULL,
    sexo      BOOLEAN NOT NULL,
    altura    FLOAT(3,2) NOT NULL,
    peso      FLOAT(3,1) NOT NULL,

    PRIMARY KEY (usuarioID),
    KEY nombreUsuario (nombre)
) ENGINE=InnoDB
DEFAULT CHARSET=latin1;
```

3) Inserción de 3 registros en la tabla usuarios

```
INSERT INTO usuarios
(nombre,clave,edad,sexo,altura,peso)
VALUES
("Iván","profe",38,false,1.78,86),
("Cristina","admin",37,true,1.68,62),
("Javier","ferro",44,false,1.82,90);
```


ANEXO III: Practica completa de Swing – Ej26_Equipo

- 1) En primer lugar nos creamos **Equipo.java**, donde, desde Java, nos creamos la Base de datos y la tabla que vamos a emplear para el proyecto:

```
package ej26_equipo;
```

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;
```

```
public class Equipo {
```

```
    // Miembros de la clase (atributos)
```

```
    private Connection conexion;
    private String conector;
    private String servidorBBDD;
    private String BBDD;
    private String usuario;
    private String clave;
    private String cadenaConexion;
```

```
    private void conexionBBDD ()
```

```
        throws SQLException, ClassNotFoundException {
    Class.forName("com.mysql.jdbc.Driver");
        conector = "jdbc:mysql:";
        servidorBBDD = "///localhost:3306/"; // ~ 127.0.0.1:<puerto>
        usuario = "root";
        clave = "root";
        cadenaConexion = conector + servidorBBDD;
        conexion =
            DriverManager.getConnection(cadenaConexion, usuario, clave);
    }
```

```
    private void conexionBBDDCreada ()
```

```
        throws SQLException, ClassNotFoundException {

        // Cargamos el driver
    Class.forName("com.mysql.jdbc.Driver");
        conector = "jdbc:mysql:";
        servidorBBDD = "///localhost:3306/"; // ~ 127.0.0.1:<puerto>
        BBDD = "java";
        usuario = "root";
        clave = "root";
        cadenaConexion = conector + servidorBBDD + BBDD;
    conexion =
        DriverManager.getConnection(cadenaConexion, usuario, clave);
    }
```

```

private void crearBBDD ()
    throws SQLException,ClassNotFoundException {

    conexionBBDD ();
    String cadenaCrearBBDD =
        "CREATE DATABASE IF NOT EXISTS java ";
    Statement ejecutar = conexion.createStatement();
    ejecutar.executeUpdate(cadenaCrearBBDD);
    System.out.println("La BBDD se creó correctamente");
    ejecutar.close(); // Cerrar la conexión!
}

private void crearTabla()
    throws SQLException,ClassNotFoundException {
    conexionBBDDCreada ();

    // Definimos la cadena para crear la Tabla
    String cadenaCrearTabla =
        "CREATE TABLE IF NOT EXISTS Equipo" +
        "(EquipoID TINYINT(2)      NOT NULL AUTO_INCREMENT," +
        " NombreEquipo VARCHAR(30) NOT NULL UNIQUE," +
        " Estadio      VARCHAR(30) NOT NULL UNIQUE," +
        " Poblacion VARCHAR(30) NOT NULL," +
        " Provincia VARCHAR(30) NOT NULL," +
        " Cod_Postal SMALLINT(5) UNSIGNED NOT NULL," +
        " PRIMARY KEY (EquipoID)" +
        ")      ENGINE = innnoDB " +
        "CHARSET = 'Latin1'," +
        "COLLATE = 'latin1_spanish_ci'";

    Statement ejecutar = conexion.createStatement();
    ejecutar.executeUpdate(cadenaCrearTabla);
    System.out.println("La Tabla equipo se creó correctamente");
    ejecutar.close();
}

public Equipo() throws SQLException,ClassNotFoundException {
    crearBBDD ();
    crearTabla();
}

// No olvidar comentar este main al crear la BBDD
public static void main(String[] args)
    throws SQLException,ClassNotFoundException {
    new Equipo();
}
}

```

- 2) Ahora nos creamos la clase **ConexionBBDD.java** que será esencial para realizar las distintas conexiones a la BBDD (aunque hemos implementado el método ConexionBBDD en la primera clase, mejor usaremos esta para el resto del proyecto):

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class ConexionBBDD {
    // Miembros de la clase (atributos)
    private Connection conexion;
    private String conector, servidorBBDD, BBDD;
    private String usuario, clave;
    private String cadenaConexion;
    private ResultSet resultado;

    // Constructor Predefinido
    public ConexionBBDD() {
        conector = "jdbc:mysql:";
        servidorBBDD = "///localhost:3306/"; // ~ 127.0.0.1:<puerto>
        usuario = "root";
        clave = "root";
    }

    public Connection conexionBBDD (String bbdd)
        throws SQLException, ClassNotFoundException {
        // Cargamos el driver
        Class.forName("com.mysql.jdbc.Driver");

        // Usamos el método estático getConnection
        BBDD = bbdd;
        cadenaConexion = conector + servidorBBDD + BBDD;
        conexion =
            DriverManager.getConnection(cadenaConexion, usuario, clave);
        return conexion;
    }

    public void ejecutaUpdate (String accion, String mensaje)
        throws SQLException {
        Statement ejecutar = conexion.createStatement();
        ejecutar.executeUpdate(accion);
        System.out.println(mensaje);
        ejecutar.close();
    }
}
```

```

public ResultSet ejecutaQuery (String consulta, String mensaje)
    throws SQLException {
    Statement ejecutar = conexion.createStatement();
    resultado = ejecutar.executeQuery(consulta);
    System.out.println(mensaje);
    //ejecutar.close();
    return resultado;
}
}

```

- 3) Ahora creamos la tabla jugadores a partir de la clase **Jugadores.java** (esta clase ya empleará el ConexionBBDD.java que hemos visto antes):

```

import java.sql.Connection;
import java.sql.SQLException;
import java.sql.Statement;
public class Jugadores extends ConexionBBDD {
    // El único miembro de la clase será el objeto Connection
    private Connection conexion;

    // El Constructor
    public Jugadores()
        throws SQLException, ClassNotFoundException {
        String bbdd = "java";
        // Con super llamo al constructor Predefinido del Padre
        // Luego, llamo al método que me devuelve la conexión a la BBDD
        this.conexion = super.conexionBBDD(bbdd);
        borrarTablaJugadores();
        crearTablaJugadores();
    }

    private void crearTablaJugadores ()
        throws SQLException {
        String crearTabla =
            "CREATE TABLE IF NOT EXISTS Jugadores" +
            "(JugadorID SMALLINT(3) NOT NULL AUTO_INCREMENT, " +
            "EquipoID TINYINT(2) NOT NULL," +
            "NombreJugador VARCHAR(30) NOT NULL," +
            "Dorsal TINYINT(2) NOT NULL," +
            "Edad TINYINT(2) NOT NULL," +
            "PRIMARY KEY (JugadorID)," +
            "CONSTRAINT fk_EquipoID " +
            "FOREIGN KEY (EquipoID) REFERENCES Equipo (EquipoID)" +
            ") ENGINE = innodb " + // Ojo al espacio tras innodb
            "CHARSET = 'Latin1'," +
            "COLLATE = 'latin1_spanish_ci'";
        String mensaje = "La tabla Jugadores se creó Correctamente!";
        ejecutaAccion (crearTabla, mensaje);
    }
}

```

```

private void borrarTablaJugadores () throws SQLException {
    String borrarTabla = "DROP TABLE IF EXISTS Jugadores ";
    String mensaje = "La tabla Jugadores se borró Correctamente!";
    ejecutaAccion (borrarTabla, mensaje);
}

private void ejecutaAccion (String accion, String mensaje)
    throws SQLException {
    Statement ejecutar = conexion.createStatement();
    ejecutar.executeUpdate(accion);
    System.out.println(mensaje);
    ejecutar.close();
}

// No olvidar comentar este main al crear la tabla
public static void main(String[] args)
    throws SQLException, ClassNotFoundException {
    new Jugadores();
}
}

```

- 4) Ahora nos creamos la clase **InsertarDatos.java** para incluir algunos registros en las tablas Equipos y Jugadores (todo desde Java y usando ConexionBBDD):

```

import java.sql.Connection;
import java.sql.SQLException;
public class InsertarDatos extends ConexionBBDD {
    // El único miembro de la clase será el objeto Connection
    private Connection conexion;

    public InsertarDatos()
        throws SQLException ,ClassNotFoundException{
        String bbdd = "java";
        this.conexion = super.conexionBBDD(bbdd);
        borrarDatosEquipos();
        borrarDatosJugadores();
        cargarDatosEquipos ();
        cargarDatosJugadores ();
    }

    private void borrarDatosEquipos () throws SQLException {
        String borraEquipos = "DELETE FROM Equipo";
        String mensaje =
            "Se ha borrado el contenido de la tabla Equipo";
        super.ejecutaUpdate(borraEquipos, mensaje);
    }
}

```

```

private void borrarDatosJugadores () throws SQLException {
    String borraJugadores = "DELETE FROM Jugadores";
    String mensaje =
        "Se ha borrado el contenido de la tabla Jugadores";
    super.ejecutaUpdate(borraJugadores, mensaje);
}

private void cargarDatosEquipos () throws SQLException {
    String insertarEquipos =
"INSERT INTO Equipo" +
"(NombreEquipo, Estadio, Poblacion, Provincia, Cod_Postal)" +
"VALUES" +
"('Real Betis' , 'Benito Villamarín' , 'Sevilla' , 'Sevilla',41012)," +
"('Sevilla FC','Ramón Sanchez Pizjuan','Sevilla','Sevilla',41008 )" +
"('Recreativo Huelva','Nuevo Colombino','Huelva','Huelva', 21007)," +
"('Málaga CF','La Rosaleda','Málaga','Málaga', 29011 )";
    String mensaje = "Se han cargado los Equipos";
    super.ejecutaUpdate(insertarEquipos, mensaje);
}

private void cargarDatosJugadores () throws SQLException {
    // Usamos StringBuilder por ser mas eficiente
    StringBuilder sb = new StringBuilder("INSERT INTO Jugadores");
    sb.append(" (EquipID, NombreJugador, Dorsal, Edad)");
    sb.append(" VALUES"); // Con append concatenamos cadenas
    sb.append("(1 , 'Rubén Castro', 24 , 31 ),");
    sb.append("(2 , 'Álvaro Negredo', 9 , 26 ),");
    sb.append("(3 , 'Ezequiel Calvente', 9 , 21 ),");
    sb.append("(4 , 'Willy Caballero', 13 , 32 )");
    String insertarJugadores = sb.toString();
    String mensaje = "Se han cargado los Jugadores";
    super.ejecutaUpdate(insertarJugadores, mensaje);
}

```

// No olvidar comentar este main al introducir los Registros

```

public static void main(String[] args)
    throws SQLException, ClassNotFoundException {
    new InsertarDatos();
}

```

NOTA: Como puede verse en el método cargarDatosJugadores hemos empleado la clase `StringBuilder` creada en Java5 que permite trabajar con cadenas, como `String`, pero a diferencia de este NO es inmutable, con lo que no genera un nuevo objeto en cada concatenación de cadenas. Luego asignamos el conteido del objeto `StringBuilder` a la cadena y se pasa al parámetro, reduciendo mucho el contenido de memoria.

- 5) La siguiente clase, **ObjetoEquipo.java**, nos permitirá trabajar en la clase posterior con arrays de objetos. Es la que nos dará el "molde" para los datos de los equipos:

```
public class ObjetoEquipo {

    // Miembros de la Superclase
    int EquipoID;
    String NombreEquipo;
    String Estadio;
    String Poblacion;
    String Provincia;
    int Cod_Postal;

    public ObjetoEquipo() {          }

}
```

- 6) La clase **VerEquipos.java**, contiene algunos elementos esenciales tanto de la programación básica en Java como del framework Swing. En concreto:

- ArrayList o arrays de Objetos
- Tablas, Modelos para tablas y personalizaciones (Swing).

```
import java.awt.BorderLayout;          import java.awt.Font;
import java.awt.event.MouseAdapter;import java.awt.event.MouseEvent;
import java.util.ArrayList;            import java.util.List;
import java.sql.Connection;            import java.sql.ResultSet;
import java.sql.SQLException;
import javax.swing.JFrame;              import javax.swing.JScrollPane;
import javax.swing.JTable;              import javax.swing.table.DefaultTableModel;
import javax.swing.table.TableColumn;
```

```
public class VerEquipos extends ConexionBBDD {
    // Los miembros de la clase
    private Connection conexion;
    ResultSet resultado;                // ResultSet ~ mysqli_fetch_assoc (PHP)
    static List <ObjetoEquipo> equipos =
        new ArrayList <ObjetoEquipo> ();

    // Defino los objetos para presentar la tabla en la ventana
    // Plantilla de la tabla
    DefaultTableModel modelo = new DefaultTableModel();
    JTable tabla = new JTable (modelo);

    // Constructor predeterminado
    public VerEquipos() throws SQLException, ClassNotFoundException {
        cargarConsultaEquipos ();
        crearArrayList ();
        pintaVentana();
    }
}
```

```

private void cargarConsultaEquipos ()
    throws SQLException, ClassNotFoundException {
    // En primer lugar establezco la conexión a la BBDD java
    String bbdd = "java";
    this.conexion = super.conexionBBDD(bbdd);

    // Uso el método ejecutaQuery de la Superclase
    String consulta = "SELECT * FROM Equipo";
    String mensaje = "Se ha ejecutado la consulta!";
    resultado = super.ejecutaQuery(consulta, mensaje);
}
private void crearArrayList () throws SQLException {
    while (resultado.next()) {           // Mientras haya filas que leer
        ObjetoEquipo nuevoEquipo = new ObjetoEquipo();

        // Meto en cada atributo del objeto el campo que devuelve el ResultSet
        nuevoEquipo.EquipoID = resultado.getInt("EquipoID");
        nuevoEquipo.NombreEquipo = resultado.getString("NombreEquipo");
        nuevoEquipo.Estadio = resultado.getString("Estadio");
        nuevoEquipo.Poblacion = resultado.getString("Poblacion");
        nuevoEquipo.Provincia = resultado.getString("Provincia");
        nuevoEquipo.Cod_Postal = resultado.getInt("Cod_Postal");
        // Meto el objeto creado en el ArrayList
        equipos.add(nuevoEquipo);
    }
}

private void pintaVentana () throws SQLException {
    JFrame ventanaEquipos = new JFrame ("Listado de Equipos");
    ventanaEquipos.setBounds(200, 200, 1024, 350);
    ventanaEquipos.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    tabla.setRowHeight(30);                               // Altura de cada fila
    tabla.setFont(new Font ("Arial", Font.ITALIC, 20)); // Tipo letra

    // Ponemos los títulos de las Columnas
    String nombresColumnas[] = {"ID Equipo", "Nombre Equipo",
                                "Estadio", "Población", "Provincia", "Cod. Postal"};
    for (int i=0; i<nombresColumnas.length; i++)
        modelo.addColumn(nombresColumnas[i]);

    // Al fin, metemos los datos del ArrayList en la tabla. Iremos metiendo
    // los datos FILA POR FILA en el MODELO (NO en la tabla). Si lo metiésemos
    // en la tabla directamente tendría que ser Object[][]
    for (int i=0; i<equipos.size(); i++) {
        Object fila [] = new Object [nombresColumnas.length];
        fila[0] = equipos.get(i).EquipoID;
        fila[1] = equipos.get(i).NombreEquipo;
        fila[2] = equipos.get(i).Estadio;
    }
}

```



```

        fila[3] = equipos.get(i).Poblacion;
        fila[4] = equipos.get(i).Provincia;
        fila[5] = equipos.get(i).Cod_Postal;
        // Una vez metido los dato en la fila[] lo meto en el modelo
        modelo.addRow(fila);
    }
    personalizaColumnas ();

    // El listener de la tabla (MUY IMPORTANTE!!)
    tabla.addMouseListener(new MouseAdapter() {
        public void mouseClicked (MouseEvent evento) {
            int fila = tabla.rowAtPoint(evento.getPoint());
            int columna = tabla.columnAtPoint(evento.getPoint());

            // Sabiendo la Fila y la columna presento el contenido
            System.out.println(modelo.getValueAt(fila, columna));
        }
    });

    // Añadimos una Barra de desplazamiento con la tabla
    // incluida a la Ventana
    JScrollPane barraDesp = new JScrollPane (tabla);
        ventanaEquipos.getContentPane().add(barraDesp,
            BorderLayout.CENTER);
    ventanaEquipos.setVisible(true);
}

// Si queremos personalizar el ancho de cada columna de la tabla...
public void personalizaColumnas () {
    TableColumn columna1 = tabla.getColumnModel().getColumn(0);
    columna1.setPreferredWidth(40);
    TableColumn columna2 = tabla.getColumnModel().getColumn(1);
    columna2.setPreferredWidth(140);
    TableColumn columna3 = tabla.getColumnModel().getColumn(2);
    columna3.setPreferredWidth(180);
    TableColumn columna4 = tabla.getColumnModel().getColumn(3);
    columna4.setPreferredWidth(40);
    TableColumn columna5 = tabla.getColumnModel().getColumn(4);
    columna5.setPreferredWidth(40);
    TableColumn columna6 = tabla.getColumnModel().getColumn(5);
    columna6.setPreferredWidth(40);
}

    // No olvidar comentar este main al introducir los Registros
    public static void main(String[] args)
        throws SQLException, ClassNotFoundException {
        new VerEquipos();
    }
}

```

- 7) En la siguiente clase **InsertarJugadores.java** vamos a ver un ejemplo de inserción de registro (uno solo) mediante un formulario, llamando previamente a la Base de datos para rellenar una lista desplegable.

NOTA IMPORTANTE: Para que salga bien el ejemplo, nos debemos bajar un icono de Guardar en formato PNG, a ser posible de tamaño 32x32 pixeles y situarlo en el paquete junto al .java.

```
import java.awt.Color;
import java.awt.Font;
import java.awt.SystemColor;
import java.awt.event.ActionListener;
import java.sql.ResultSet;
import java.sql.Statement;

import java.awt.Component;
import java.awt.Rectangle;
import java.awt.event.ActionEvent;
import java.sql.Connection;
import java.sql.SQLException;
```

```
import javax.swing.DefaultComboBoxModel;
import javax.swing.ImageIcon;
import javax.swing.JComboBox;
import javax.swing.JLabel;
import javax.swing.JScrollPane;
import javax.swing.SwingConstants;
import javax.swing.border.BevelBorder;
import javax.swing.border.LineBorder;

import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.JTextField;
```

```
public class InsertarJugadores extends ConexionBBDD {
    // Miembros de la clase...
    private JPanel contentPane;
    private JFrame ventana;
    private JTextField campoNombre;
    private JTextField campoDorsal;
    private JTextField campoEdad;

    // Otros miembros IMPORTANTES
    private Connection conexion;
    private JLabel etiqaMensaje;
    private int equipoID;
    private String nombre;
    private int dorsal;
    private int edad;
    private JButton botonCerrar;

    // Para poner una lista desplegable
    private JScrollPane scrollLista;
    private JComboBox lista;
    private DefaultComboBoxModel modeloLista;
    private ResultSet resultado;

    public InsertarJugadores() throws ClassNotFoundException{
        pintaVentana ();
    }
}
```

```

private void pintaVentana () throws ClassNotFoundException {
    ventana = new JFrame ("Insertar Jugador");
    ventana.setBounds(new Rectangle(100, 100, 800, 600));
    ventana.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    contentPane = new JPanel();
    contentPane.setFont(new Font("Ubuntu", Font.BOLD, 20));
    contentPane.setBackground(new Color(240, 230, 140));
    contentPane.setBorder(new BevelBorder
        (BevelBorder.RAISED, new Color(255, 0, 0),
         null, null, new Color(0, 0, 128)));
    ventana.setContentPane(contentPane);
    contentPane.setLayout(null);

    JLabel etiqTitulo = new JLabel("Insertar Jugadores");
    etiqTitulo.setHorizontalAlignment(SwingConstants.CENTER);
    etiqTitulo.setFont(new Font("Dialog", Font.BOLD, 25));
    etiqTitulo.setForeground(new Color(0, 0, 128));
    etiqTitulo.setOpaque(true);
    etiqTitulo.setBackground(new Color(135, 206, 250));
    etiqTitulo.setBounds(155, 45, 500, 70);
    contentPane.add(etiqTitulo);

    JLabel etiqEquipID = new JLabel("Selecciona Equipo");
    etiqEquipID.setOpaque(true);
    etiqEquipID.setHorizontalAlignment(SwingConstants.LEFT);
    etiqEquipID.setForeground(SystemColor.activeCaption);
    etiqEquipID.setFont(new Font("Dialog", Font.BOLD, 20));
    etiqEquipID.setBackground(new Color(255, 127, 80));
    etiqEquipID.setBounds(100, 180, 200, 40);
    contentPane.add(etiqEquipID);

    JLabel etiqNombre = new JLabel("Nombre");
    etiqNombre.setOpaque(true);
    etiqNombre.setHorizontalAlignment(SwingConstants.LEFT);
    etiqNombre.setForeground(SystemColor.activeCaption);
    etiqNombre.setFont(new Font("Dialog", Font.BOLD, 20));
    etiqNombre.setBackground(new Color(255, 127, 80));
    etiqNombre.setBounds(100, 232, 200, 40);
    contentPane.add(etiqNombre);
    JLabel etiqDorsal = new JLabel("Dorsal");
    etiqDorsal.setOpaque(true);
    etiqDorsal.setHorizontalAlignment(SwingConstants.LEFT);
    etiqDorsal.setForeground(SystemColor.activeCaption);
    etiqDorsal.setFont(new Font("Dialog", Font.BOLD, 20));
    etiqDorsal.setBackground(new Color(255, 127, 80));
    etiqDorsal.setBounds(100, 284, 200, 40);
    contentPane.add(etiqDorsal);

```

```

JLabel etiqEdad = new JLabel("Edad");
etiqEdad.setOpaque(true);
etiqEdad.setHorizontalAlignment(SwingConstants.LEFT);
etiqEdad.setForeground(SystemColor.activeCaption);
etiqEdad.setFont(new Font("Dialog", Font.BOLD, 20));
etiqEdad.setBackground(new Color(255, 127, 80));
etiqEdad.setBounds(100, 336, 200, 40);
contentPane.add(etiqEdad);

campoNombre = new JTextField();
campoNombre.setFont(new Font("Dialog", Font.PLAIN, 25));
campoNombre.setColumns(10);
campoNombre.setBounds(350, 232, 350, 40);
contentPane.add(campoNombre);

campoDorsal = new JTextField();
campoDorsal.setFont(new Font("Dialog", Font.PLAIN, 25));
campoDorsal.setColumns(10);
campoDorsal.setBounds(350, 284, 350, 40);
contentPane.add(campoDorsal);

campoEdad = new JTextField();
campoEdad.setFont(new Font("Dialog", Font.PLAIN, 25));
campoEdad.setColumns(10);
campoEdad.setBounds(350, 336, 350, 40);
contentPane.add(campoEdad);

JButton botonInsertar = new JButton("Insertar Jugador");
botonInsertar.setBorder(new LineBorder
    (new Color(0, 0, 255), 2, true));
botonInsertar.setBounds(150, 420, 250, 40);
contentPane.add(botonInsertar);

// Definimos el listener para el botón
botonInsertar.addActionListener(new ActionListener () {
    public void actionPerformed(ActionEvent e)
    {
        try {
            insertarJugador();
        } catch (ClassNotFoundException error) {}
    }
});

JLabel etiqAutor = new JLabel("Realizado por Iván Rodríguez");
etiqAutor.setIcon(new ImageIcon
    (InsertarJugadores.class.getResource("guardar.png")));
etiqAutor.setHorizontalAlignment(SwingConstants.CENTER);
etiqAutor.setForeground(new Color(128, 0, 128));
etiqAutor.setFont(new Font("Ubuntu", Font.BOLD, 20));

```

```

etiqaAutor.setBorder(new BevelBorder
    (BevelBorder.RAISED, new Color(255, 0, 0),
        null, null, new Color(0, 0, 128)));
etiqaAutor.setOpaque(true);
etiqaAutor.setBackground(new Color(230, 230, 250));
etiqaAutor.setAlignmentY(Component.BOTTOM_ALIGNMENT);
etiqaAutor.setAlignmentX(Component.RIGHT_ALIGNMENT);
etiqaAutor.setBounds(200, 130, 400, 40);
contentPane.add(etiqaAutor);

etiqaMensaje = new JLabel("");
etiqaMensaje.setFont(new Font("Dialog", Font.BOLD, 14));
etiqaMensaje.setForeground(new Color(255, 0, 0));
etiqaMensaje.setBounds(250, 500, 400, 40);
contentPane.add(etiqaMensaje);

botonCerrar = new JButton("Cerrar Ventana");
botonCerrar.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        // cierra la ventana pero NO la aplicacion
        ventana.dispose();
    }
});

botonCerrar.setBorder(new LineBorder
    (new Color(0, 0, 255), 2, true));
botonCerrar.setBounds(430, 420, 250, 40);
contentPane.add(botonCerrar);

// Para añadir la lista desplegable
scrollLista = new JScrollPane();
scrollLista.setBounds(350, 180, 350, 40);
contentPane.add(scrollLista);

lista = new JComboBox();
lista.setForeground(new Color(255, 0, 0));
lista.setFont(new Font("Dialog", Font.BOLD, 20));
scrollLista.setViewportView(lista);
modeloLista = new DefaultComboBoxModel();

verEquipos();
try {
    while (resultado.next())
        modeloLista.addElement
            (resultado.getString("NombreEquipo"));
} catch (SQLException e1) {e1.printStackTrace(); }

```

```

// Una vez metido en el modelo TODOS los equipos
// se lo aplico a la Lista
lista.setModel(modeloLista);
ventana.setVisible(true);
}

// Para sacar los equipos hago un método
private void verEquipos () throws ClassNotFoundException{
    try {
        // Establezco la conexión a la BBDD
        this.conexion = super.conexionBBDD("java");
        String cadenaConsulta =
            "SELECT NombreEquipo FROM Equipo ORDER BY EquipoID";
        Statement ejecutaConsulta = conexion.createStatement();
        resultado = ejecutaConsulta.executeQuery(cadenaConsulta);
    } catch (SQLException error) {
        error.printStackTrace();
    }
}

private void insertarJugador () throws ClassNotFoundException{
    try {
        this.conexion = super.conexionBBDD("java");
        // Recojo el valor elegido en el formulario
        String nombreEquipo = (String) lista.getSelectedItemAt();
        String cadenaVerEquipo =
            "SELECT EquipoID FROM Equipo " +
            "WHERE NombreEquipo = '" + nombreEquipo + "'";

        Statement ejecutarConsulta = conexion.createStatement();
        resultado = ejecutarConsulta.executeQuery(cadenaVerEquipo);
        while (resultado.next())
            equipoID = Integer.parseInt
                (resultado.getString("EquipoID"));

    } catch (SQLException error) {
        error.printStackTrace();
    }

    nombre = campoNombre.getText();
    dorsal = Integer.parseInt(campoDorsal.getText());
    edad = Integer.parseInt(campoEdad.getText());

    String cadenaInsertar =
        "INSERT INTO Jugadores" +
        "(EquipoID, NombreJugador, Dorsal, Edad)" +
        "VALUES " +
        "(" + equipoID + ", '" + nombre + "', " +
        dorsal + ", " + edad + ")";
}

```

```

        try {
            Statement ejecutar = conexion.createStatement();
            ejecutar.executeUpdate(cadenaInsertar);
            etiqMensaje.setText("Jugador Insertado Correctamente!");
            ejecutar.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }

    // No olvidar comentar este main al insertar el jugador
    public static void main(String[] args)
        throws ClassNotFoundException{
        new InsertarJugadores();
    }
}

```

- 8) En la siguiente clase, **ActualizarJugadores.java** realizamos un UPDATE sobre la tabla jugadores. Es importante reseñar que, a la hora de probarlo, debemos realizar una elección en la primera lista desplegable (se ha hecho así para simplificar código):

```

import java.awt.Color;
import java.awt.Rectangle;
import java.awt.event.ActionListener;
import java.sql.ResultSet;
import java.sql.Statement;

import java.awt.Font;
import java.awt.event.ActionEvent;
import java.sql.Connection;
import java.sql.SQLException;

import javax.swing.DefaultComboBoxModel;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.JTextField;

import javax.swing.JComboBox;
import javax.swing.JLabel;
import javax.swing.JScrollPane;
import javax.swing.SwingConstants;

public class ActualizarJugadores extends ConexionBBDD {
    // Miembros de la clase
    private JFrame ventana;
    private JPanel panel;
    private JButton botonActualizar;
    private JButton botonCerrar;

    private JTextField campoNombre;
    private JTextField campoDorsal;
    private JTextField campoEdad;
    private JLabel etiqMensaje;

    private int jugadorID;
    private String nombreJugador;
    private String nombreEquipo;
    private int dorsal;
    private int edad;
}

```

```

// Otros miembros de la clase
private Connection conexion;
private ResultSet resultadoEquipos;
private ResultSet resultadoJugadores;
private ResultSet resultadoCampos;

// Definimos las variables para los combos
private JScrollPane scrollEquipo;
private JComboBox listaEquipos;
private DefaultComboBoxModel modeloEquipo;
private JScrollPane scrollJugador;
private JComboBox listaJugadores;
private DefaultComboBoxModel modeloJugadores;

// El constructor
public ActualizarJugadores() throws ClassNotFoundException{
    pintaVentana ();
}

// Método que pinta la ventana y sus componentes
public void pintaVentana () throws ClassNotFoundException{
    ventana = new JFrame ("Actualizar Jugador");
    ventana.setBounds(new Rectangle(100, 100, 800, 600));
// Si es ventana secundaria al cerrar pasamos a la ventana principal
    ventana.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);

    panel = new JPanel();
    ventana.setFont(new Font("Ubuntu", Font.BOLD, 20));
    panel.setBackground(new Color(240, 230, 140));
    // Definimos el Panel principal de ventana
    ventana.setContentPane(panel);
    // El panel tiene Absolute Layout
    panel.setLayout(null);

    // Declaro y defino las Etiquetas...
    JLabel etiqTitulo = new JLabel("Actualizar Jugador");
    etiqTitulo.setOpaque(true);
    etiqTitulo.setForeground(new Color(0, 0, 255));
    etiqTitulo.setFont(new Font("Ubuntu", Font.BOLD, 25));
    etiqTitulo.setHorizontalAlignment(SwingConstants.CENTER);
    etiqTitulo.setBounds(155, 45, 500, 70);
    panel.add(etiqTitulo);

    JLabel etiqEquipo = new JLabel(" Selecciona Equipo");
    etiqEquipo.setOpaque(true);
    etiqEquipo.setHorizontalAlignment(SwingConstants.LEFT);
    etiqEquipo.setForeground(Color.BLUE);
    etiqEquipo.setFont(new Font("Ubuntu", Font.BOLD, 15));

```



```
etiEquipo.setBounds(100, 125, 200, 40);
panel.add(etiqEquipo);
JLabel etiqJugador = new JLabel(" Selecciona Jugador");
etiJugador.setOpaque(true);
etiJugador.setHorizontalAlignment(SwingConstants.LEFT);
etiJugador.setForeground(Color.BLUE);
etiJugador.setFont(new Font("Ubuntu", Font.BOLD, 15));
etiJugador.setBounds(100, 177, 200, 40);
panel.add(etiqJugador);
```

```
JLabel EtiqNombre = new JLabel(" Nombre");
EtiqNombre.setOpaque(true);
EtiqNombre.setHorizontalAlignment(SwingConstants.LEFT);
EtiqNombre.setForeground(Color.BLUE);
EtiqNombre.setFont(new Font("Ubuntu", Font.BOLD, 15));
EtiqNombre.setBounds(100, 230, 200, 40);
panel.add(EtiqNombre);
```

```
JLabel EtiqDorsal = new JLabel(" Dorsal");
EtiqDorsal.setOpaque(true);
EtiqDorsal.setHorizontalAlignment(SwingConstants.LEFT);
EtiqDorsal.setForeground(Color.BLUE);
EtiqDorsal.setFont(new Font("Ubuntu", Font.BOLD, 15));
EtiqDorsal.setBounds(100, 282, 200, 40);
panel.add(EtiqDorsal);
```

```
JLabel EtiqEdad = new JLabel(" Edad");
EtiqEdad.setOpaque(true);
EtiqEdad.setHorizontalAlignment(SwingConstants.LEFT);
EtiqEdad.setForeground(Color.BLUE);
EtiqEdad.setFont(new Font("Ubuntu", Font.BOLD, 15));
EtiqEdad.setBounds(100, 334, 200, 40);
panel.add(EtiqEdad);
```

```
// Meto la etiqueta con el mensaje de confirmación
etiMensaje = new JLabel("<...>");
etiMensaje.setFont(new Font("Ubuntu", Font.BOLD, 15));
etiMensaje.setForeground(new Color(255, 0, 0));
etiMensaje.setBounds(250, 500, 400, 40);
panel.add(etiqMensaje);
```

```
// Declaro y defino los campos
campoNombre = new JTextField();
campoNombre.setFont(new Font("Ubuntu", Font.BOLD, 20));
campoNombre.setBounds(350, 230, 350, 40);
panel.add(campoNombre);
campoNombre.setColumns(10);
```

```

campoDorsal = new JTextField();
campoDorsal.setFont(new Font("Ubuntu", Font.BOLD, 20));
campoDorsal.setColumns(10);
campoDorsal.setBounds(350, 282, 350, 40);
panel.add(campoDorsal);

campoEdad = new JTextField();
campoEdad.setFont(new Font("Ubuntu", Font.BOLD, 20));
campoEdad.setColumns(10);
campoEdad.setBounds(350, 334, 350, 40);
panel.add(campoEdad);

// Meto un botón y su Listener
botonActualizar = new JButton("Actualizar Jugador");
botonActualizar.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        try {
            actualizarJugador();
        } catch (ClassNotFoundException error) {}
    }
});
botonActualizar.setBounds(150, 420, 250, 40);
panel.add(botonActualizar);

// Meto el botón cerrar Ventana y su listener
botonCerrar = new JButton("Cerrar Ventana");
botonCerrar.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        ventana.dispose();
    }
});
botonCerrar.setBounds(448, 420, 250, 40);
panel.add(botonCerrar);

// Creo ambos combos y sus correspondientes modelos
listaEquipos = new JComboBox();
modeloEquipo = new DefaultComboBoxModel ();
listaEquipos.setModel(modeloEquipo);

listaJugadores = new JComboBox();
listaJugadores.setFont(new Font("Ubuntu", Font.BOLD, 20));
modeloJugadores = new DefaultComboBoxModel ();
listaJugadores.setModel(modeloJugadores);

// El primer combo y su listener
scrollEquipo = new JScrollPane();
scrollEquipo.setBounds(350, 127, 350, 40);
panel.add(scrollEquipo);

```

```

listaEquipos.setFont(new Font("Ubuntu", Font.BOLD, 20));
// Una vez creado el primer combo meto los datos (equipos)
verEquipos();

listaEquipos.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        String nombreEquipo =
            (String) listaEquipos.getSelectedItem();

        try {
            verJugadores(nombreEquipo);
        } catch (ClassNotFoundException error) {}
    }
});
scrollEquipo.setViewportView(listaEquipos);

// El Segundo Combo y su listener
scrollJugador = new JScrollPane();
scrollJugador.setBounds(350, 180, 350, 40);
panel.add(scrollJugador);

listaJugadores.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        String nombreJugador =
            (String) listaJugadores.getSelectedItem();

        try {
            mostrarCampos(nombreJugador);
        }
        catch (ClassNotFoundException error) {}
    }
});
scrollJugador.setViewportView(listaJugadores);
ventana.setVisible(true);
}

// Implemento un método para cargar los equipos en el primer combo
private void verEquipos () throws ClassNotFoundException{
    try {
        this.conexion = super.conexionBBDD("java");
        String cadenaConsulta =
            "SELECT NombreEquipo " +
            "FROM Equipo " +
            "ORDER BY EquipoID";
        Statement ejecutarConsulta = conexion.createStatement();
        resultadoEquipos =
            ejecutarConsulta.executeQuery(cadenaConsulta);
    }
}

```

```

        while (resultadoEquipos.next())
            modeloEquipo.addElement
                (resultadoEquipos.getString("NombreEquipo"));
    } catch (SQLException e) {        e.printStackTrace();        }
}

// Método para ver los jugadores del EQUIPO SELECCIONADO
private void verJugadores (String nombreEquipo)
    throws ClassNotFoundException{
    try {
        this.conexion = super.conexionBBDD("java");
        String cadenaConsulta =
            " SELECT NombreJugador FROM Jugadores, Equipo " +
            " WHERE NombreEquipo = '" + nombreEquipo + "' " +
            " AND Jugadores.EquipoID = Equipo.EquipoID";
        Statement ejecutarConsulta = conexion.createStatement();
        resultadoJugadores =
            ejecutarConsulta.executeQuery(cadenaConsulta);

        // Meto los jugadores en el modelo.
        // Borro el contenido del combo antes de nada...
        modeloJugadores.removeAllElements();
        while (resultadoJugadores.next())
            modeloJugadores.addElement
                (resultadoJugadores.getString("NombreJugador"));
    } catch (SQLException e) {        e.printStackTrace();        }
}

private void mostrarCampos (String nombreJugador)
    throws ClassNotFoundException{
    try {
        this.conexion = super.conexionBBDD("java");
        String cadenaConsulta =
            " SELECT * FROM Jugadores " +
            " WHERE NombreJugador = '" + nombreJugador + "'";
        Statement ejecutarConsulta = conexion.createStatement();
        resultadoCampos =
            ejecutarConsulta.executeQuery(cadenaConsulta);

        // Meto los datos en los campos y en la variable jugadorID
        while (resultadoCampos.next()) {
            jugadorID = resultadoCampos.getInt("JugadorID");
            campoNombre.setText
                (resultadoCampos.getString("NombreJugador"));
            campoDorsal.setText(resultadoCampos.getString("Dorsal"));
            campoEdad.setText(resultadoCampos.getString("Edad"));
        }
    } catch (SQLException e) {        e.printStackTrace();        }
}

```

```

private void actualizarJugador () throws ClassNotFoundException{
    if (jugadorID == 0)
        etiqMensaje.setText("No ha elegido ningún jugador!");
    else
    {
        // Recogemos los datos de los campos
        nombreJugador = campoNombre.getText();
        dorsal = Integer.parseInt(campoDorsal.getText());
        edad = Integer.parseInt(campoEdad.getText());

        String cadenaUpdate =
            " UPDATE Jugadores " +
            " SET" +
            " NombreJugador = '" + nombreJugador + "', " +
            " Dorsal = " + dorsal + "," +
            " Edad = " + edad +
            " WHERE JugadorID = " + jugadorID;

        try {
            this.conexion = super.conexionBBDD("java");
            Statement ejecutarUpdate =
                conexion.createStatement();
            ejecutarUpdate.executeUpdate(cadenaUpdate);
            etiqMensaje.setText(" Jugador Actualizado!");
            ejecutarUpdate.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}

// No olvidar comentar este main al actualizar el Jugador
public static void main(String[] args)
    throws ClassNotFoundException {
    new ActualizarJugadores();
}
}

```

- 9) En el siguiente ejemplo, **Imprimir.java**, creamos una clase para imprimir lo que tengamos en la ventana de la aplicación. Usaremos la interfaz Printable:

```

import java.awt.Graphics;
import java.awt.print.PageFormat;
import java.awt.print.PrinterException;
import javax.swing.JFrame;

import java.awt.Graphics2D;
import java.awt.print.Printable;

public class Imprimir implements Printable{
    //miembro de clase:
    private JFrame ventana;

```

```

//constructor por defecto.
public Imprimir() {}
//Constructor definido.
public Imprimir(JFrame nuevaVentana){
    this.ventana = nuevaVentana;
}

public int print(Graphics grafico, PageFormat formato, int pagina){
    if(pagina==0){
        //Convertimos la ventana en un gráfico 2D.
        Graphics2D grafico2d = (Graphics2D) grafico;
        //Cogemos el tamaño de la ventana y lo pasamos a gráfico2d.
        grafico2d.translate(formato.getImageableX(),
            formato.getImageableY());
        //imprimimos:
        ventana.printAll(grafico2d);
        return PAGE_EXISTS;//devuelve un 1.
    }
    else
        return NO_SUCH_PAGE;//devuelve un 0.
    }
}

```

10) Por último, creamos una clase llamada **Borrarjugador.java** donde usaremos el DELETE y de paso llamamos, mediante botones, a Imprimir y a Actualizarjugador.

```

import java.sql.Connection;
import java.sql.SQLException;
import java.awt.Color;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.print.PrinterException;
import java.awt.print.PrinterJob;

import java.sql.ResultSet;
import java.sql.Statement;
import java.awt.Font;

import javax.swing.DefaultComboBoxModel;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JOptionPane;
import javax.swing.JScrollPane;

import javax.swing.JComboBox;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.SwingConstants;

public class BorrarJugador extends ConexionBBDD {
    //Miembros de la clase
    private JFrame ventana;
    private JPanel panel;
    private JLabel etiqMensaje;
    private JComboBox listaEquipos;
    private JComboBox listaJugadores;

```

```

private DefaultComboBoxModel modeloEquipos;
private DefaultComboBoxModel modeloJugadores;
private JButton botonBorrar;
private JButton botonCerrar;
private JButton botonActualizar;
private JButton botonImprimir;
private String nombreJugador;

// Definimos las variables para conectar la BBDD
private Connection conexion;
private ResultSet resultadoEquipos;
private ResultSet resultadoJugadores;

public BorrarJugador() throws ClassNotFoundException{
    pintaVentana();
}

private void pintaVentana() throws ClassNotFoundException{
    ventana = new JFrame("Borrar Jugador");
    ventana.setFont(new Font("Ubuntu", Font.BOLD, 12));
    ventana.setBounds(100,100,800,600);
    ventana.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);

    panel = new JPanel();
    panel.setBackground(new Color(176, 224, 230));
    ventana.setContentPane(panel);
    panel.setLayout(null);

    componentes();
    verEquipos();
    ventana.setVisible(true);
}

/* En este caso hemos separado los componentes de la interfaz*/
private void componentes() {
    etiquetas();
    combos();
    botones();
    listeners();
}

private void etiquetas() {
    JLabel etiqTitulo = new JLabel("BORRAR JUGADOR");
    etiqTitulo.setHorizontalAlignment(SwingConstants.CENTER);
    etiqTitulo.setHorizontalTextPosition(SwingConstants.LEADING);
    etiqTitulo.setFont(new Font("Ubuntu", Font.BOLD, 20));
    etiqTitulo.setForeground(new Color(0, 0, 139));
    etiqTitulo.setOpaque(true);

```

```

etiqtitulo.setBackground(new Color(0, 250, 154));
etiqtitulo.setBounds(100, 45, 600, 70);
panel.add(etiqtitulo);

JLabel etiqEquipo = new JLabel("Selecciona Equipo");
etiqEquipo.setOpaque(true);
etiqEquipo.setHorizontalTextPosition(SwingConstants.LEADING);
etiqEquipo.setHorizontalAlignment(SwingConstants.CENTER);
etiqEquipo.setForeground(new Color(238, 232, 170));
etiqEquipo.setFont(new Font("Ubuntu", Font.BOLD, 20));
etiqEquipo.setBackground(new Color(154, 205, 50));
etiqEquipo.setBounds(100, 160, 200, 40);
panel.add(etiqEquipo);

JLabel etiqJugador = new JLabel("Selecciona Jugador");
etiqJugador.setOpaque(true);
etiqJugador.setHorizontalTextPosition(SwingConstants.LEADING);
etiqJugador.setHorizontalAlignment(SwingConstants.CENTER);
etiqJugador.setForeground(new Color(238, 232, 170));
etiqJugador.setFont(new Font("Ubuntu", Font.BOLD, 20));
etiqJugador.setBackground(new Color(154, 205, 50));
etiqJugador.setBounds(100, 240, 200, 40);
panel.add(etiqJugador);

etiqMensaje = new JLabel("<...>");
etiqMensaje.setOpaque(true);
etiqMensaje.setHorizontalTextPosition(SwingConstants.LEADING);
etiqMensaje.setHorizontalAlignment(SwingConstants.CENTER);
etiqMensaje.setForeground(new Color(255, 0, 0));
etiqMensaje.setFont(new Font("Ubuntu", Font.BOLD, 20));
etiqMensaje.setBackground(new Color(176, 224, 230));
etiqMensaje.setBounds(155, 500, 522, 40);
panel.add(etiqMensaje);
}

private void combos() {
    JScrollPane scrollPane = new JScrollPane();
    scrollPane.setBounds(350, 160, 350, 40);
    panel.add(scrollPane);

    listaEquipos = new JComboBox();
    listaEquipos.setFont(new Font("Ubuntu", Font.BOLD, 20));
    scrollPane.setViewportView(listaEquipos);

    JScrollPane scrollPane_1 = new JScrollPane();
    scrollPane_1.setBounds(350, 240, 350, 40);
    panel.add(scrollPane_1);
}

```



```

listaJugadores = new JComboBox();
listaJugadores.setFont(new Font("Ubuntu", Font.BOLD, 20));
scrollPane_1.setViewportView(listaJugadores);

//Definimos los modelos
modeloEquipos = new DefaultComboBoxModel();
modeloJugadores = new DefaultComboBoxModel();
listaEquipos.setModel(modeloEquipos);
listaJugadores.setModel(modeloJugadores);
}

private void botones() {
    botonBorrar = new JButton("Borrar Jugador");
    botonBorrar.setBounds(155, 368, 250, 40);
    panel.add(botonBorrar);

    botonCerrar = new JButton("Cerrar Ventana");
    botonCerrar.setBounds(427, 420, 250, 40);
    panel.add(botonCerrar);

    botonActualizar = new JButton("Actualizar Jugador");
    botonActualizar.setBounds(427, 368, 250, 40);
    panel.add(botonActualizar);

    botonImprimir = new JButton("Imprimir");
    botonImprimir.setBounds(155, 420, 250, 40);
    panel.add(botonImprimir);
}

private void listeners() {
    listaEquipos.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            String nombreEquipo = (String) listaEquipos.getSelectedItem();
            try {
                verJugador(nombreEquipo);
            } catch (ClassNotFoundException error) {}
        }
    });

    listaJugadores.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            nombreJugador = (String) listaJugadores.getSelectedItem();
        }
    });
}

```

```

    botonActualizar.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            try {
                ActualizarJugadores ventanaActualizar =
                    new ActualizarJugadores();
            }
            catch (ClassNotFoundException error) {}
        }
    });

    botonBorrar.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            try {
                cuadroBorrar ();
            } catch (ClassNotFoundException error) {}
        }
    });

    botonCerrar.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            ventana.dispose();
        }
    });

    botonImprimir.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            imprimir();
        }
    });
}

private void verEquipos () throws ClassNotFoundException{
    try {
        this.conexion = super.conexionBBDD("java");
        String cadenaEquipos = " SELECT * FROM Equipo";
        Statement ejecutar = conexion.createStatement();
        resultadoEquipos = ejecutar.executeQuery(cadenaEquipos);

        while (resultadoEquipos.next())
            modeloEquipos.addElement
                (resultadoEquipos.getString("NombreEquipo"));
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

```

```

        private void verJugador(String nombreEquipo)
            throws ClassNotFoundException{
        try {
            this.conexion = super.conexionBBDD("java");
            String cadenaJugadores =
            " SELECT * FROM Jugadores, Equipo " +
            " WHERE Equipo.EquipoID = Jugadores.EquipoID " +
            " AND NombreEquipo = '" + nombreEquipo + "'";
            Statement ejecutarConsulta = conexion.createStatement();
                resultadoJugadores =
                ejecutarConsulta.executeQuery(cadenaJugadores);

                // borro el combo
            modeloJugadores.removeAllElements();
            while (resultadoJugadores.next())
                modeloJugadores.addElement
                    (resultadoJugadores.getString("NombreJugador"));
            } catch (SQLException e) { e.printStackTrace(); }
        }

        private void cuadroBorrar () throws ClassNotFoundException{
            int opcion = JOptionPane.showConfirmDialog
                (ventana, " ¿Desea borrar a " + nombreJugador + " ?",
                "Confirmar borrado", JOptionPane.OK_CANCEL_OPTION);
            if (opcion == JOptionPane.OK_OPTION)
                borrarJugador ();
            else
                etiMensaje.setText("Opción cancelada!!!");
        }

        private void borrarJugador () throws ClassNotFoundException{
            try {
                this.conexion = super.conexionBBDD("java");
                String cadenaBorrado =
                    " DELETE FROM Jugadores " +
                    " WHERE NombreJugador = '" + nombreJugador + "'";
                Statement ejecutarDelete = conexion.createStatement();
                ejecutarDelete.executeUpdate(cadenaBorrado);
                etiMensaje.setText
                    ("El jugador " + nombreJugador + " ha sido borrado");
            } catch (SQLException e) { e.printStackTrace(); }
        }
    }

```

```

private void imprimir(){
    //definimos un trabajo de impresión
    PrinterJob impresion = PrinterJob.getPrinterJob();
    //definimos lo que vamos a imprimir, llamando a la clase creada.
    impresion.setPrintable(new Imprimir(ventana));

    if(impresion.printDialog()==true){
        try {
            impresion.print();
            etiqMensaje.setText("La impresión se ha realizado");
        }
        catch (PrinterException e) {e.printStackTrace();}
    }
    else
        etiqMensaje.setText("La impresión ha sido cancelada");
}

    public static void main(String[] args)
        throws ClassNotFoundException{
new BorrarJugador();
}
}

```

ANEXO IV: Practica completa de ArrayList con Objetos – Ej05_Libros

- 1) En primer lugar nos creamos **Libro.java**, donde vamos a crear el molde para los objetos libro que iremos introduciendo en el ArrayList:

NOTA: La clase auxiliar Consola deberá ser modificada en su método leeCadena con un next(). Así solo hacemos la lectura de los títulos con una sola palabra, para simplificar código.

```
import clasesAuxiliares.Consola;
```

```
public class Libro {
    // Definimos los atributos privados
    private String titulo;
    private long isbn;
    private float precio;
    private boolean disponible;

    // Creo un objeto Consola
    static Consola teclado = new Consola ();

    // Constructor definido
    public Libro(String titulo, long isbn,
        float precio, boolean disponible) {
        this.titulo = titulo;
        this.isbn = isbn;
        this.precio = precio;
        this.disponible = disponible;
    }

    // Constructor por defecto
    public Libro() {
    }

    public String getTitulo() {
        return titulo;
    }

    public void setTitulo(String titulo) {
        this.titulo = titulo;
    }

    public long getIsbn() {
        return isbn;
    }

    public void setIsbn(long isbn) {
        this.isbn = isbn;
    }
}
```

```

public float getPrecio() {
    return precio;
}

public void setPrecio(float precio) {
    this.precio = precio;
}

public boolean getDisponible() {
    return disponible;
}

public void setDisponible(boolean disponible) {
    this.disponible = disponible;
}

public void leeDatos () {
    String nuevoTitulo = teclado.leeCadena("Escriba título:");
    long nuevolsbn = teclado.leeLong("Escriba isbn:");
    float nuevoPrecio = teclado.leeFloat("Escriba Precio:");
    boolean disponible =
        teclado.leeBooleano("Disponible (true/false):");
    setTitulo(nuevoTitulo);
    setIsbn(nuevolsbn);
    setPrecio(nuevoPrecio);
    setDisponible(disponible);
}

public void imprimeDatos () {
    teclado.imprimeMensaje(" Título: "+getTitulo());
    teclado.imprimeMensaje(" ISBN: "+getIsbn());
    teclado.imprimeMensaje(" Precio: "+getPrecio());
    if (getDisponible())
        teclado.imprimeMensaje(" Disponible: Si");
    else
        teclado.imprimeMensaje(" Disponible: No");
}
}

```

- 2) Ahora, en la clase **Ej05_Libros** (la que crea por defecto el proyecto) introducimos en el main la creación del ArrayList y, mediante métodos, usaremos los principales métodos de esta clase (add, remove, indexOf y get).

```

import clasesAuxiliares.Consola;
import java.util.ArrayList;
/**
 * @version 1.2 201411 20
 * @author Iván Rodríguez
 */

```

```

public class Ej05_Libros {

    static Consola teclado = new Consola ();
    // Nos creamos el arrayList de Libros
    static ArrayList <Libro> biblioteca = new ArrayList <Libro> ();
    //static Libro miLibro = new Libro ();

    public static int menuOpciones () {
        int opcion = 0;

        teclado.imprimeMensaje("  MENU LIBROS  ");
        teclado.imprimeMensaje(" 1. Añadir Libro ");
        teclado.imprimeMensaje(" 2. Borrar Libro ");
        teclado.imprimeMensaje(" 3. Buscar Libro ");
        teclado.imprimeMensaje(" 4. Imprimir Libros ");
        teclado.imprimeMensaje(" 5. Salir ");
        opcion = teclado.leeEntero(" Teclee Opción: ");
        return opcion;
    }

    public static void usoMenus (int opcion) {
        switch (opcion) {
            case 1: introLibro (); break;
            case 2: borraLibro (); break;
            case 3: buscaLibro (); break;
            case 4: imprimeBiblioteca (); break;
        }
    }

    public static void despedida () {
        teclado.imprimeMensaje("Gracias por usar esta aplicación");
        teclado.imprimeMensaje("Realizado por: Iván Rodríguez");
    }

    public static void introLibro () {
        Libro miLibro = new Libro ();
        miLibro.leeDatos();
        biblioteca.add(miLibro);
    }

    public static void borraLibro () {
        int indice = teclado.leeEntero("Escriba indice Libro");
        biblioteca.remove(indice);oi
    }
}

```

```

public static void buscaLibro () {
    Libro miLibro;
    String titulo = teclado.leeCadena("Escriba titulo");

    for (int i=0; i<biblioteca.size(); i++)
    {
        miLibro = biblioteca.get(i);
        if (miLibro.getTitulo().equals(titulo)) {
            miLibro.imprimeDatos();
        }
    }
}

public static void imprimeBiblioteca (){
    Libro miLibro;
    for (int i=0; i<biblioteca.size(); i++) {
        teclado.imprimeMensaje("  DATOS LIBRO["+i+"]  ");
        miLibro = biblioteca.get(i);
        miLibro.imprimeDatos();
    }
}

public static void main(String[] args) {
    int opcion=0;
    do {
        opcion = menuOpciones();
        usoMenus(opcion);
    } while (opcion!=5);

    despedida();
}
}

```


ANEXO V: Practica completa de HashMap – Ej05_Diccionario

```
import java.util.HashMap;

public class Ej04_Diccionario {

    // Creamos un objeto consola y Hashmap
    static Consola teclado = new Consola();
    static HashMap diccionario = new HashMap();

    static void llenarDiccionario () {
        diccionario.put("Archivo", "File");
        diccionario.put("Abrir", "Open");
        diccionario.put("Guardar", "Save");
        diccionario.put("Guardar Como", "Save As");
        diccionario.put("Imprimir", "Print");
        diccionario.put("Cerrar", "Close");
        diccionario.put("Deshacer", "Undo");
        diccionario.put("Copiar", "Copy");
        diccionario.put("Cortar", "Cut");
        diccionario.put("Pegar", "Paste");
        diccionario.put("Buscar", "Find");
        diccionario.put("Reemplazar", "Replace");
        diccionario.put("Ayuda", "Help");
        diccionario.put("Acerca de", "About");
    }

    static String traduceEspEng (String cadenaEsp) {
        String cadena = "";
        cadena = (String) diccionario.get(cadenaEsp);
        return cadena;
    }

    public static void main(String[] args) {
        String palabra = "";
        String traducida = "";
        llenarDiccionario ();
        palabra =
            teclado.leePalabra("Escriba palabra Esp a traducir:");
        traducida = traduceEspEng (palabra);
        teclado.imprimeMensaje(traducida);
    }
}
```

ANEXO VI: Código a UML

<http://wiki.netbeans.org/NetbeansUML>

<http://deadlock.netbeans.org/hudson/job/uml/lastSuccessfulBuild/artifact/build/updates/updates.xml>

Los pasos para realizarlo son los siguientes:

1. Herramientas > Plugins > [Plugins disponibles]
- 2.

<https://www.youtube.com/watch?v=k-3KAMfkr14>

ANEXO VII: Ejercicio de Interfaz Gráfica

```
public class InterfazGrafica extends JFrame implements ActionListener {
    static int anchoPantalla;
    //static int  altoPantalla;                NO SE USA AQUI

    public InterfazGrafica() {
        super("Interfaz Gráfica");
        Dimension tamPantalla = Toolkit.getDefaultToolkit().getScreenSize();
        anchoPantalla = tamPantalla.width;
        //altoPantalla = tamPantalla.height;
        pintaVentana();
    }

    public void pintaVentana() {
        Container panelGeneral = this.getContentPane();
        panelGeneral.setLayout
            (new BoxLayout(panelGeneral, BoxLayout.Y_AXIS));

        creaMenus(panelGeneral);
        crearBarraHerramientas(panelGeneral);
        meterPestañas(panelGeneral);
        meteBarraEstado(panelGeneral);

        setSize(640, 480);
        setExtendedState(MAXIMIZED_BOTH);
        setVisible(true);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }

    public void creaMenus(Container panel) {
        JMenuBar barra;
        barra = new JMenuBar();
        barra.setMaximumSize(new Dimension(anchoPantalla, 100));
        JMenu archivo, ayuda, coches, furgonetas;
        JMenuItem introCoche, verCoche,
            introFurgo, verFurgo, salir;

        archivo = new JMenu("Archivo");

        coches = new JMenu("Coches");
        introCoche = new JMenuItem("Introducir");
        introCoche.setName("introCoche");
        verCoche = new JMenuItem("Ver");
        verCoche.setName("verCoche");
        coches.add(introCoche);
        coches.add(verCoche);
```

```

furgonetas = new JMenu("Furgonetas");
introFurgo = new JMenuItem("Introducir");
introFurgo.setName("introFurgo");
verFurgo = new JMenuItem("Ver");
verFurgo.setName("verFurgo");
furgonetas.add(introFurgo);
furgonetas.add(verFurgo);

salir = new JMenuItem("Salir");
archivo.add(coches);
archivo.add(furgonetas);
archivo.add(salir);
ayuda = new JMenu("Ayuda");
barra.add(archivo);
barra.add(ayuda);
panel.add(barra);

// Añadimos los listener
introCoche.addActionListener(this);
introFurgo.addActionListener(this);
verCoche.addActionListener(this);
verFurgo.addActionListener(this);
salir.addActionListener(this);
}

public void crearBarraHerramientas(Container panel) {
    JMenuBar barraHerramientas = new JMenuBar();
    barraHerramientas.setMaximumSize(new Dimension(anchoPantalla, 40));
    barraHerramientas.setSize(400, 80);
    agregarBotones(barraHerramientas, "Rojo", "close.gif");
    agregarBotones(barraHerramientas, "Amarillo", "important.gif");
    agregarBotones(barraHerramientas, "Verde", "ring.gif");
    panel.add(barraHerramientas);
}

public void agregarBotones(JMenuBar barra, String texto, String nombrelcono)
{
    Image icono1;
    JButton boton = new JButton(texto);
    try {
        icono1 = ImageIO.read(getClass().getResource(nombrelcono));
        boton.setIcon(new ImageIcon
            (icono1.getScaledInstance(40, 40, Image.SCALE_SMOOTH)));
    } catch (IOException ex) {}
    boton.setHorizontalTextPosition(SwingConstants.CENTER);
    boton.setVerticalTextPosition(SwingConstants.BOTTOM);
    barra.add(boton);
}

```

```

public void meterPestañas(Container panel) {
    //Creamos el conjunto de pestañas
    JTabbedPane pestañas = new JTabbedPane();

    //Creamos el panel y lo añadimos a las pestañas
    JPanel panel1 = new JPanel();
    //Componentes del panel1
    JLabel etiq1 = new JLabel(" PANEL COCHES ");
    panel1.add(etiq1);
    //Añadimos un nombre de la pestaña y el panel
    pestañas.addTab("Coches", panel1);

    //Realizamos lo mismo con el resto
    JPanel panel2 = new JPanel();
    pestañas.addTab("Furgonetas", panel2);
    //Componentes del panel2
    JLabel etiq2 = new JLabel(" PANEL FURGONETAS");
    panel2.add(etiq2);

    // Finalmente, agregamos el JTabbedPane al contenedor general
    panel.add(pestañas);
}

public void meteBarraEstado(Container panel) {
    JPanel barraEstado = new JPanel();
    barraEstado.setMaximumSize(new Dimension(anchoPantalla, 100));
    JLabel etiq = new JLabel(" BARRA DE ESTADO");
    barraEstado.add(etiq);
    panel.add(barraEstado);
}

@Override
public void actionPerformed(ActionEvent evento) {
    JMenuItem elementoMenu = (JMenuItem) evento.getSource();
    switch (elementoMenu.getName()) {
        case "verCoche": this.setTitle("Ver Coche");
        case "introCoche": this.setTitle("Introducir Coche");
        case "verFurgo": this.setTitle("Ver Furgoneta");
        case "introFurgo": this.setTitle("Introducir Furgoneta");
    }
}

public static void main(String[] args) {
    InterfazGrafica nuevo = new InterfazGrafica();
}
}

```